

SOPC 技术基础

实验指导书

(适用 AC620 开发板)

第 2 版

吴朝晖

安徽师范大学物电学院

2024. 01

目录

第一部分 AC620 开发板和功能模块	- 1 -
一、AC620 开发板功能特点	- 1 -
1.1 布局及组件	- 1 -
1.2 AC620V2 开发板器件资源	- 3 -
1.3 核心板接口资源	- 4 -
二、开发板使用	- 4 -
2.1 开发板烧写配置	- 4 -
2.2 轻触按键	- 5 -
2.3 用户 LED	- 5 -
2.4 时钟输入	- 6 -
2.5 GPIO 接口	- 7 -
2.6 SDRAM	- 8 -
2.7 USB to UART	- 8 -
2.8 无源蜂鸣器	- 9 -
2.9 7 段 8 位数码管	- 10 -
第二部分 基础实验	- 11 -
实验一 SOPC 开发流程	- 11 -
一、实验任务	- 11 -
二、实验简介	- 11 -
三、硬件设计	- 11 -
四、软件设计	- 28 -
实验二 PIO IP 核	- 33 -
一、实验任务	- 33 -
二、实验简介	- 33 -
三、硬件设计	- 33 -
四、软件设计	- 37 -
实验三 UART IP 核	- 39 -
一、实验任务	- 39 -
二、实验简介	- 39 -
三、硬件设计	- 39 -
四、软件设计	- 42 -
实验四 SDRAM IP 核	- 45 -
一、实验任务	- 45 -
二、实验简介	- 45 -
三、硬件设计	- 46 -
四、软件设计	- 55 -
实验五 uCOS 实验	- 57 -
一、实验任务	- 57 -
二、实验简介	- 57 -
三、硬件设计	- 58 -
四、软件设计	- 59 -

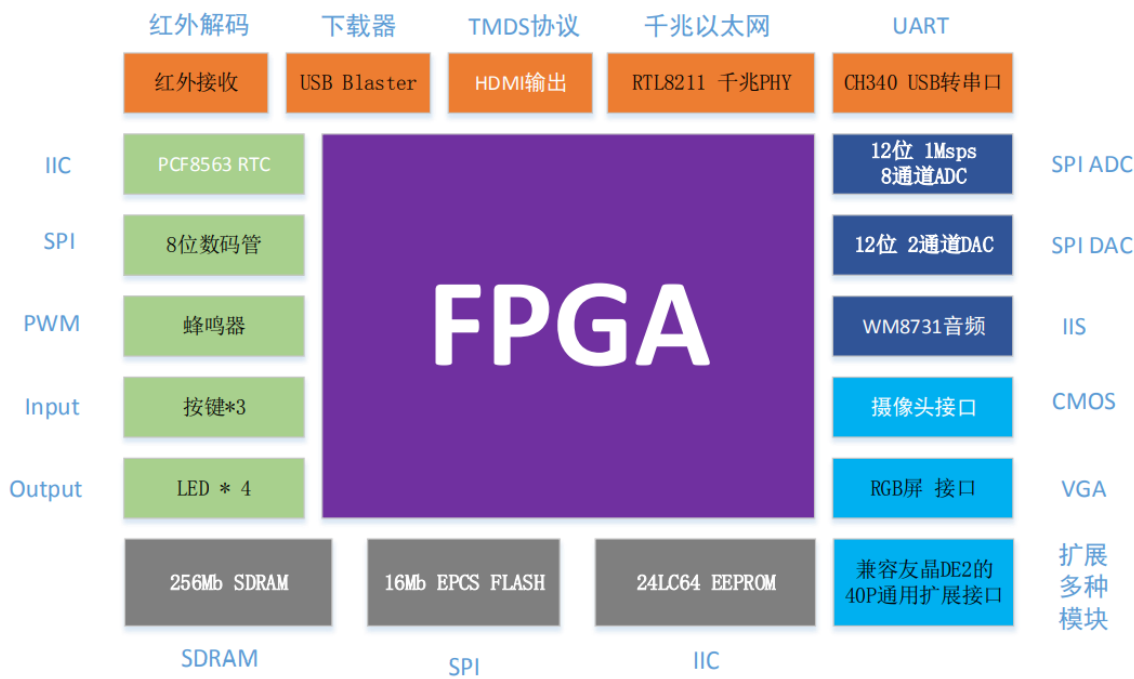
实验六 自定义 IP 核	- 65 -
一、实验任务	- 65 -
二、实验简介	- 65 -
三、硬件设计	- 67 -
四、软件设计	- 73 -

第一部分 AC620 开发板和功能模块

AC620 开发板是武汉芯路恒小梅哥团队结合当下 FPGA 在常见嵌入式应用场合的一些基本应用，设计开发的一款资源丰富，功能全面的 FPGA 教学实验板。

一、AC620 开发板功能特点

本节说明了 AC620V2 开发板的功能以及设计特点。



1.1 布局及组件

图 1-1 展示了 AC620 核心开发板的图片，它描绘了开发板布局以及一些接插件和关键元件的位置信息。

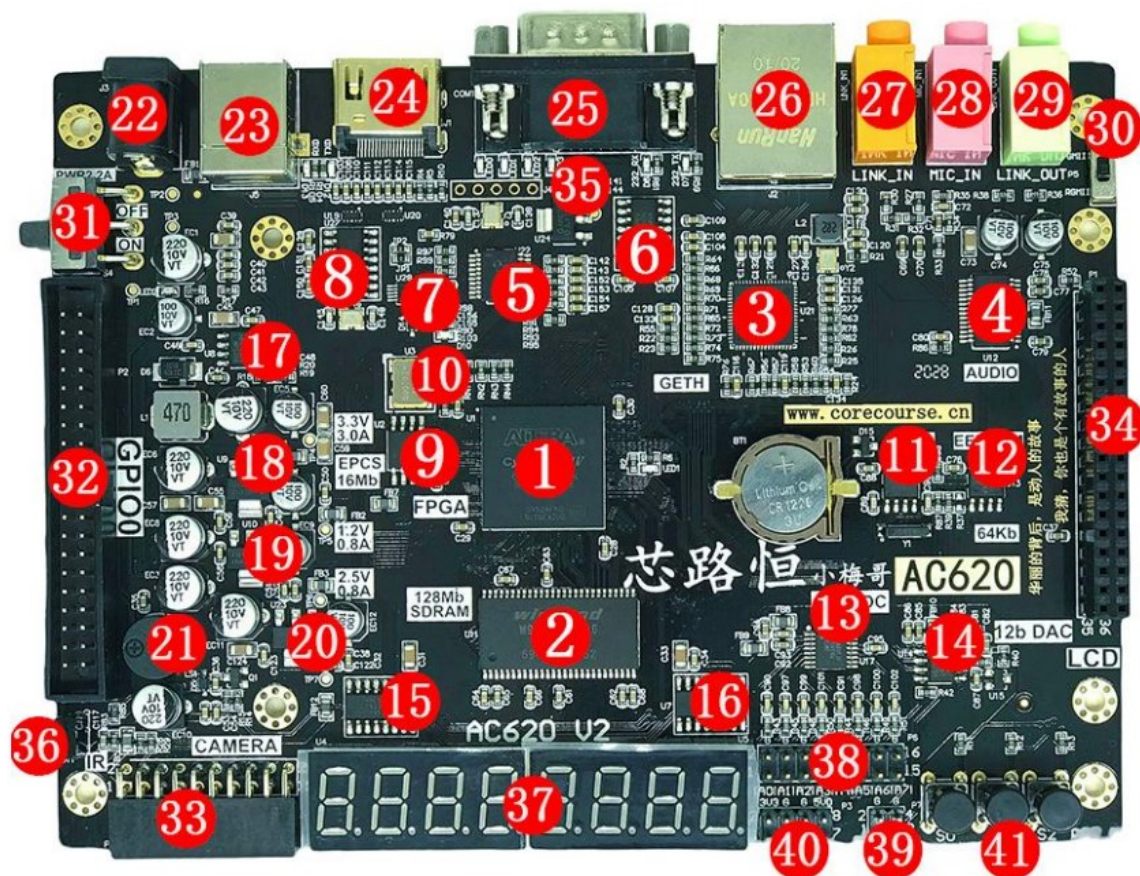


图 1-1 芯路恒 AC620V2 开发板顶层视图

1	EP4CE10F17 (FPGA)	17	MP1484 3.3V	33	摄像头接口
2	W9825G6KH-6 (SDRAM)	18	AMS1117-1.2	34	显示屏接口
3	RTL8211EG (网络芯片)	19	AMS1117 2.5	35	用户 LED
4	WM8731S (音频芯片)	20	AMS1117-3.3	36	HS38B (红外
5	USB Blaster 芯片	21	无源蜂鸣器	37	数码管
6	SP3232 (232 通信芯)	22	5V 电源输入	38	ADC 输入接口
7	CH340E (USB 转串口)	23	Blaster/UART	39	DAC 输出接口
8	SL2.1A (usb hub 芯片)	24	HDMI 输出接口	40	电源输出接口
9	W25Q16 (EPCS 芯片)	25	RS232 接口	41	用户按键
10	50MHz 有源时钟晶振	26	RJ45 网口		
11	PCF8563 (RTC 时钟)	27	音频输入接口		
12	24LC64 (EEPROM)	28	麦克风输入接口		
13	ADC128S102 (模数转换)	29	音频输出接口		
14	TLV5618 (数模转换)	30	MII/RGMII 切换		
15	74HC595 串转并芯片	31	电源开关		
16	74HC595 串转并芯片	32	通用扩展 I/O		

AC620V2 开发板拥有非常丰富的功能, 从简单逻辑到各种各样的多媒体项目, 允许使用者在一个较为宽广的领域内进行数字逻辑设计。

1.2 AC620V2 开发板器件资源

FPGA 主芯片 EP4CE10F17C8N, 拥有 10K 的逻辑单元, 两个独立锁相环, 180 个用户 IO 管脚, 423936bit 嵌入式 RAM, 46 个 9 位嵌入式硬件乘法器.....资源丰富, 完全可以满足绝大部分中小型设计的资源需求。	电可擦除只读存储器 (EEPROM) 24LC64, 拥有 64K 位的存储容量, 使用两线制 IIC 接口, 可读可写, 掉电数据不丢失, 可用来存储各种配置数据 (如触摸屏校准参数)
动态刷新随机存取存取存储器 (SDRAM) W9825G6KH-6, 拥有 256Mbit 的存储器资源, 最高运行速度 166MHz, 即可使用逻辑直接驱动, 用来进行大量数据存储 (视频图像数据, ADC 采集数据), 也可作为 NIOS II 处理器运行内存, 用来运行较大的 NIOS II 软件系统。	串行 FLASH 芯片 W25Q16, 拥有 2M 字节的存储空间, 使用 SPI 接口, 默认作为 FPGA 芯片的上电配置器件, 当使用 NIOS II 软核时, 还可以作为软件代码的存储空间。同时, 在 NIOS II 中, 还可以使用 EPCS 控制器, 将该存储器当做通用型 FLASH 进行读写, 以存储运行过程中产生的各种数据。
HDMI/DVI 输出, 开发板上使用 FPGA 逻辑和 IO 口直接实现基于 TMDS 编码规范的 HDMI/DVI 输出功能, 输出分辨率可达 1280*720, 刷新速率可达 60Hz。	USB Blaster, 开发板使用成熟的 USB Blaster 实现方案, 实现了板载 USB Blaster 功能, 在最精简的模式下, 用户只需要使用一根 USB 数据线与开发板相连, 就能实现开发和调试了。
RS232 收发器 RS3232, 3.3V 供电的 RS232 收发器, 实现 TTL 电平和 RS232 电平的互转。RS232 接口作为标准的工业设备接口, 存在于当前在大部分工业设备上, 使用该接口, AC620 可以实现对带 RS232 接口的设备控制。	以太网收发器 RTL8211, 千兆以太网收发器, 以太网以其便捷的组网特性, 几乎存在于我们生活的每个角落, 使用 FPGA 实现以太网, 可以方便的实现采集数据的便捷传输。
ADC 芯片 ADC128S102, TI 公司生产的 12 位 8 通道 ADC 芯片, 采样速率高达 1Msps, 采用标准的 SPI 串行接口, 可实现对模拟信号的采集, 实现对模拟信号的分析处理。	DAC 芯片 TLV5618, TI 公司生产的 12 位 2 通道 DAC 芯片, 采用标准的 SPI 串行接口, 可实现数字信号到模拟信号的转换, 以实现数控系统的设计。
USB 转串口芯片 CH340, 实现 USB 转串口协议, 使得 AC620 开发板能够使用一根普通的 USB mini 数据线即可与 PC 进行通信, 实现方便灵活的通信和控制。	LED 灯, 最基础的输出设备, 单个 LED 能够显示两种不同的状态, 4 个 LED 组合能够最多显示 16 种状态, 方便设计和调试。

音频编解码器 WM8731 ，IIS 总线作为一个简单高效的音频传输总线，目前已经成了各种多媒体设备中进行音频数据传输的标配，学习使用 IIS 接口的音频编解码器，实现对音频数据的处理。	红外遥控接收 ，开发板自带红外遥控接收器，可接收 38KHz 的红外遥控信号，从而为开发板提供红外遥控功能，另外，当开发板上用户按键不够用时，也可以使用红外遥控来作为开发板的扩展键盘。
蜂鸣器 ，当用户有输入请求或者系统有输出请求时，使用一个蜂鸣器发声来提示用户输入或者输出有效。以获得更加直观的输入输出体验。	串行移位寄存器 74HC595 ，使用 SPI 接口，实现串行数据到并口数据的转换，通过 3 个 IO 口扩展得到更多的输出引脚。适用于 IO 需求量巨大的应用，如大屏 LED 显示、多位数码管驱动。AC620 上使用两片 74HC595 驱动 8 位数码管。
实时时钟（RTC）芯片 PCF8563 ，IIC 接口的实时时钟芯片，带后备电池，为系统提供精准的时钟。	独立按键 ，三个独立按键，可以作为用户控制按键，实现对系统运行状态的控制

1.3 核心板接口资源

同时，为了尽量提升板卡的可扩展性和实用性，AC620V2 开发板上提供了多个通用或专用接插口，用来方便扩展各种外设模块，接插口主要有：

一个 40Pin 的排针接口 ，该接口与友晶 DE2 的单个 40Pin 接口完全兼容，每个接插口提供 36 个通用 IO 口，一个 5V 供电和一个 3.3V 供电。用户可以使用该接口连接一些扩展模块，以实现自定义功能。	一个 LCD 液晶屏的接口 ，5 寸 TFT 触摸液晶模组、VGA 输出模块支持该接口，可直接插接在此接口上，进行图像的显示。同时该接口也能兼容正点原子开发的 STM32 开发板配套液晶屏，方便使用手头现有的液晶屏插接到开发板上进行数据显示，
一个 CMOS 摄像头的摄像头接口 ，该接口兼容市面上主流的 CMOS 摄像头，方便需要进行图像采集处理的插接摄像头进行项目开发（OV7670，OV7725，OV2640，OV5640 全兼容）	一组 电源接口 ，方便直接从开发板上取电，给外挂模块供电。

二、开发板使用

本节给出了 AC620 开发板的使用说明，同时详细描述了开发板的部分组件。

2.1 开发板烧写配置

AC620V2 开发板提供了一个串行配置设备用来对 Cyclone IV E FPGA 芯片进行配置，上电以后配置数据可以自动从配置设备加载到 FPAG，使用 Quartus 软件，我们也可以在任何时候重新配置 FPGA，当然也可以改变串行非易失串行存储设备，下面将描述这两种类型的配置。

1.JTAG 配置：这种编程方法将配置比特流直接下载到了开发板上的 Cyclone IV E FPGA 芯

片中，在上电状态下，FPGA 将保留此配置，逻辑功能可以正常运行，但断电以后，配置信息将会丢失，需要上电以后重新进行配置。

2.JIC 配置：Altera 允许在 JTAG 下载模式下配置 EPCS 器件，通过 Quartus 软件自带的转换工具，可以将编译生成的.sof 文件转换为.jic 文件，然后通过 JTAG 口下载到 EPCS 器件，这样就可以做到配置数据掉电不丢失。

2.2 轻触按键

如图 2-1 所示，AC620V2 开发板提供了三个轻触按键，三个按键分别被称为 S0、S1、S2。此三个引脚直接与 FPGA 管脚相连，并接上了上拉电阻。在没有按键按下的时候，每个按键端输出的都是高电平，当按键按下的时候，被按下的按键端会输出低电平。

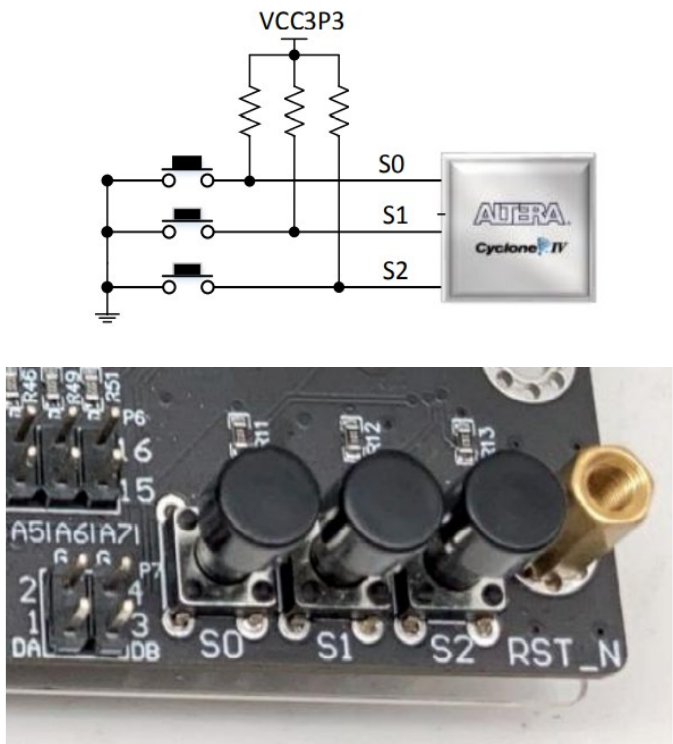


图 2-1 轻触按键和 Cyclone IV E FPGA 连接关系示例
按键管脚分配表

信号名	引脚锁定
KEY S0	M16
KEY S1	E15
KEY S2(reset)	E16

2.3 用户 LED

开发板提供了四个红色的 LED 调试灯，所有的 LED 灯都是通过 Cyclone IV E FPGA 直接驱动;当 FPGA 输出低电平时,LED 点亮,当 LED 输出高电平时,LED 灯熄灭。图 2-2 展示了 LED 和 FPGA 的连接关系。

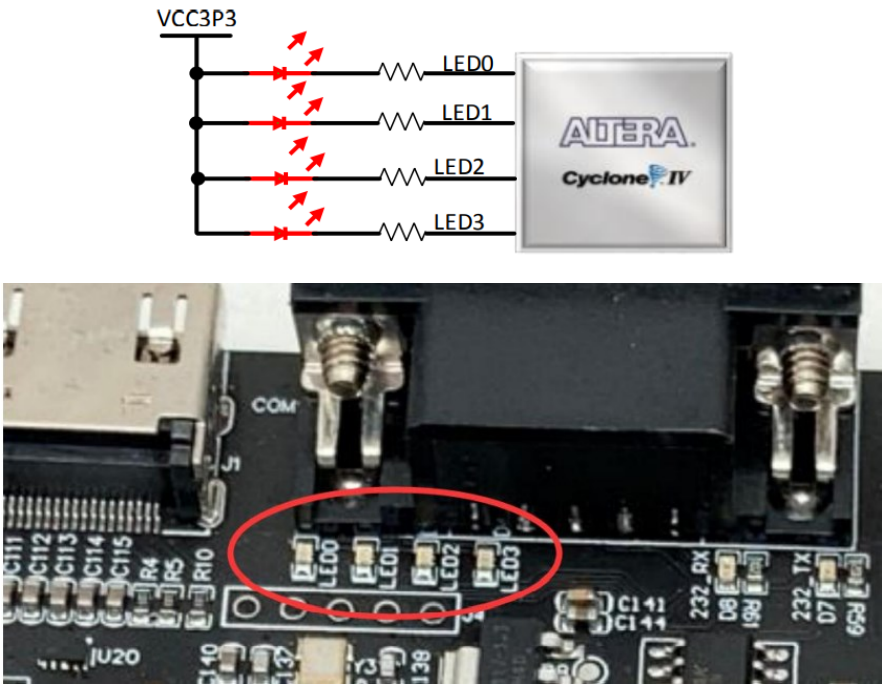


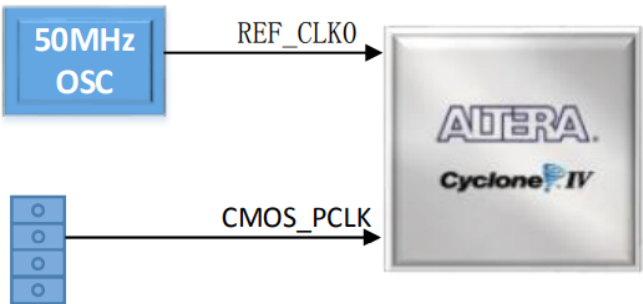
图 2-2 流水灯和 Cyclone IV E FPGA 连接示例

LED 管脚分配表

信号名	引脚锁定
LED0	PIN_A2
LED1	PIN_B3
LED2	PIN_A4
LED3	PIN_A3

2.4 时钟输入

AC620V2 开发板设计了 2 路时钟源，第一路由板载 50MHz 有源晶振提供，另外第二路集成在 CMOS 摄像头接口中(一般用于输入 CMOS 摄像头时钟像素时钟)。2 路时钟均通过 Cyclone IV E 专用的时钟引脚进行输入，保证了最好的时钟质量，并保证可以配置到对应的全局时钟链路上。图 2-3 展示了时钟和 FPGA 的连接关系。



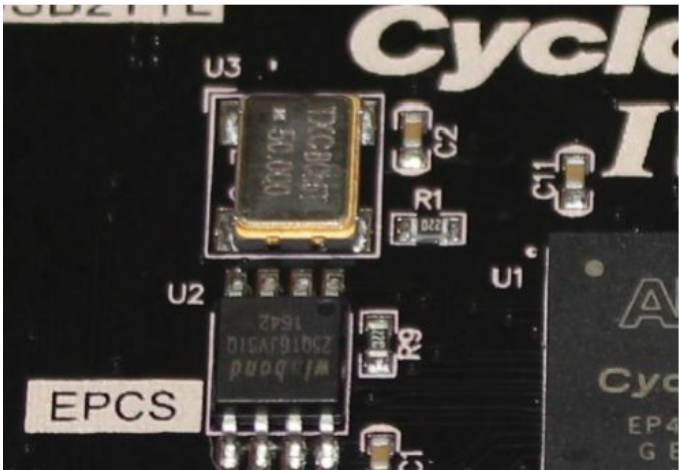


图 2-3 晶振和 Cyclone IV E FPGA 连接示例

时钟管脚分配表

信号名	引脚锁定
REF_CLK0	PIN_E1
CMOS_PCLK	PIN_M2

2.5 GPIO 接口

AC620V2 开发板提供了 1 个 40Pin 的 GPIO 接口，端口使用标准的 IDC3-40 接口。该端口除了有 36 个引脚直接连到了 Cyclone IV E FPGA 以外,还输出了 DC+5V (VCC5) ，DC+3.3V (VCC3P3)，和两个接地的引脚。端口名为 GPIO0。图 2-4 展示了 GPIO0 的端子 FPGA 管脚连接关系。

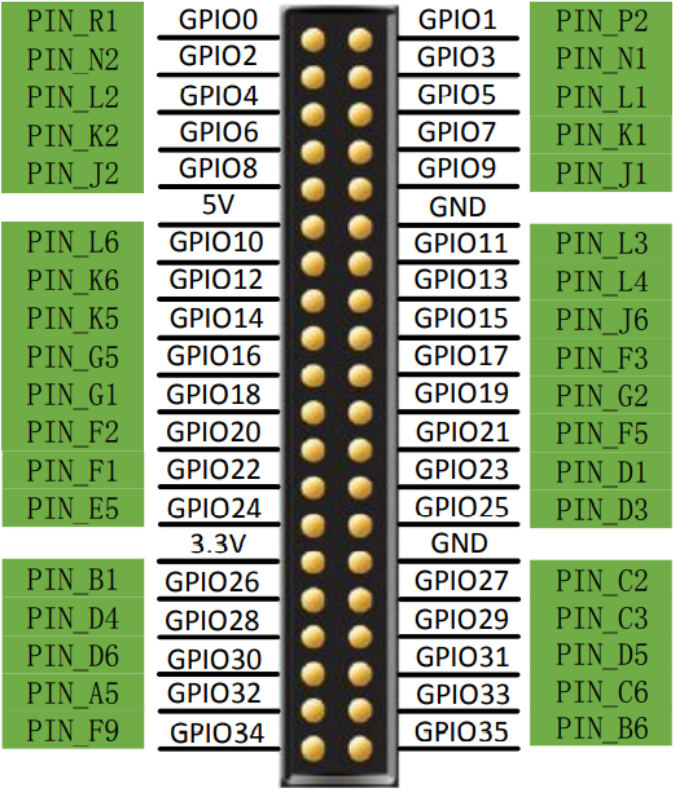




图 2-4 GPIO0 和 Cyclone IV E FPGA 连接示例

2.6 SDRAM

AC620V2 开发板设计了一片存储容量高达 256Mbits 的 SDRAM 存储器, 该芯片与 FPGA 相连的数据总线位宽为 16bit, 采用 3.3V 供电, 最高运行频率可达 166MHz。图 2-5 展示了 SDRAM 和 FPGA 之间的连接关系。

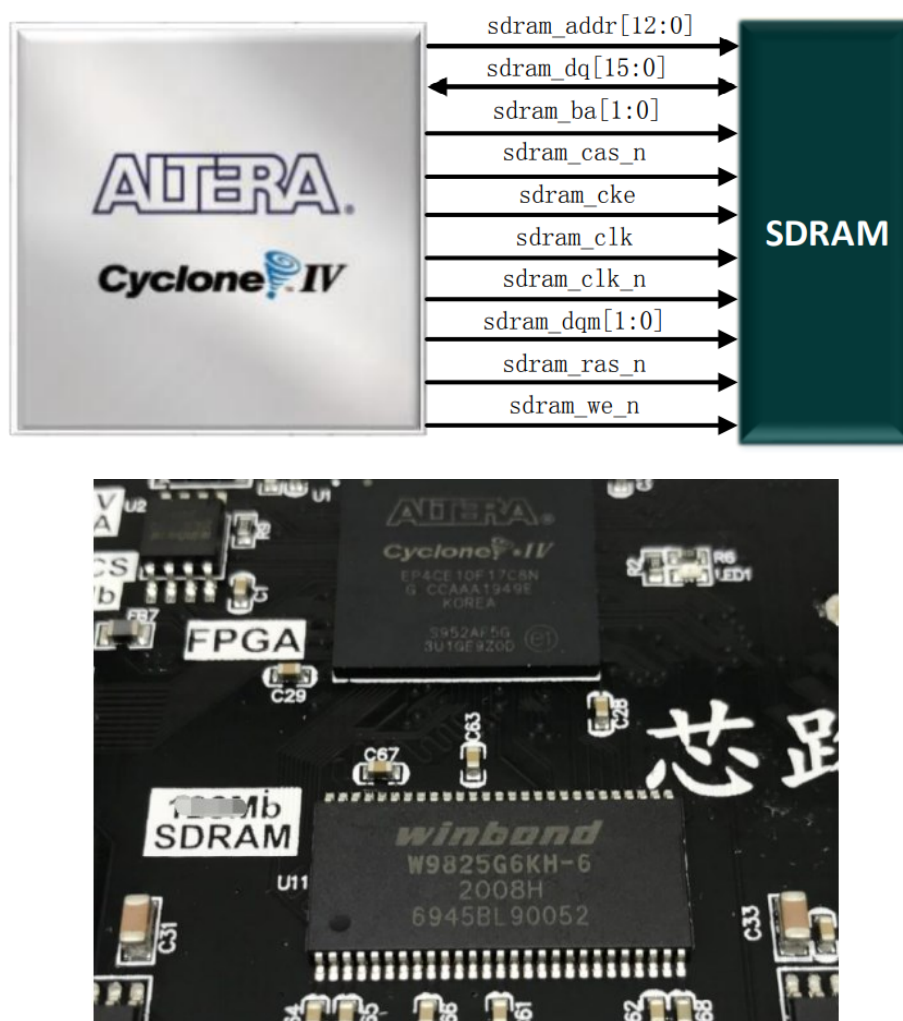


图 2-5 SDRAM 和 Cyclone IV E FPGA 连接示例

2.7 USB to UART

为了方便开发板与 PC 机相连, AC620V2 上提供了一个 USB 转串口的电路, 该 USB 可以直接作为开发板供电端口, 也可以作为一个串口电路使用, 通过串口 (UART), 可以很方便的实现开发板与计算机之间的通信。图 2-6 给出了该转换电路和 FPGA 连接关系。

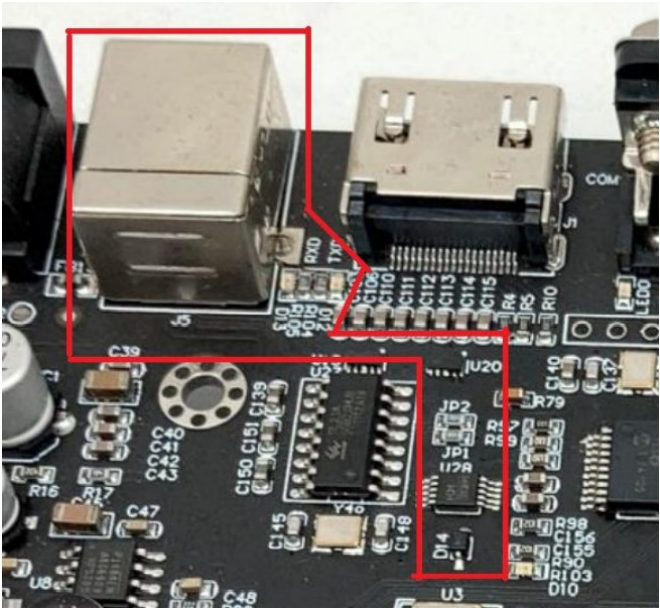
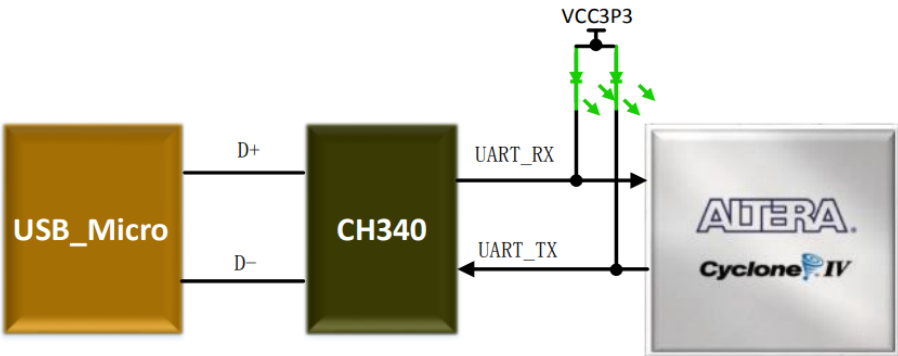


图 2-6 USB、CH340 和 Cyclone IV E FPGA 连接示例

UART 管脚分配表

信号名	引脚锁定
UART_RX	B5
UART_TX	A6

2.8 无源蜂鸣器

AC620V2 开发板为用户配备了发声装置—无源蜂鸣器。用户可以根据自己的喜好播放音乐，也可以将蜂鸣器作为报警装置，在某些时刻发出警报声。图 2-7 给出了无源蜂鸣器和 FPGA 的连接关系。



图 2-7 Beep 和 Cyclone IV E FPGA 连接示例

蜂鸣器管脚分配表

信号名	引脚锁定
BEEP	L16

2.9 7 段 8 位数码管

为了提供一种最简单直观的显示，AC620V2 开发板上提供了一个 7 段 8 位共阳极数码管电路，为了减少对 FPGA 引脚资源的占用，AC620V2 开发板上的数码管采用串行移位寄存器芯片将串行数据转化为 16 位并行数据后进行驱动。Cyclone IV E 通过 3 根数据线，连接到两片级联的串行移位器芯片 74HC595 上，再由 74HC595 将每次 16 位串行的数据转化为 16 位并行的数据，分别用以驱动 7 段 8 位数码管的段选和位选，图 2-8 为 74HC595 与 Cyclone IV E 的连接关系。

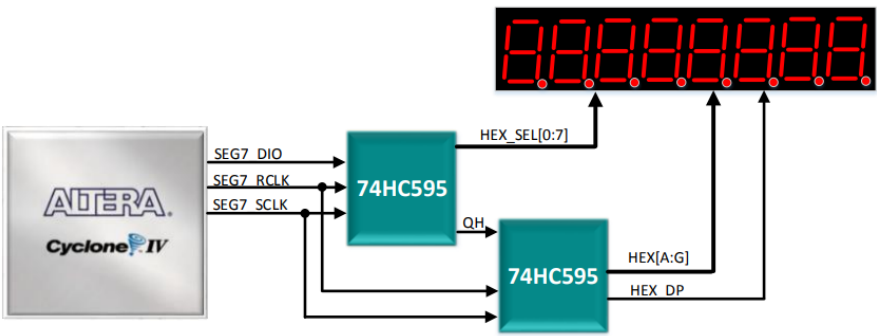
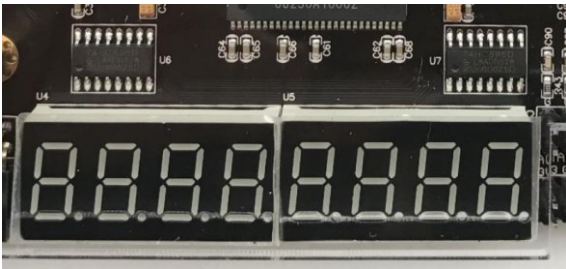


图 2-8 74HC595 与 Cyclone IV E 的连接关系



74HC595 数码管驱动电路管脚分配表

信号名	引脚锁定
SEG7_SCLK	F6
SEG7_DIO	E6
SEG7_RCLK	B4

此外开发板还配置有红外接收，IIC 总线，通用显示扩展接口，摄像头接口，电源拓展接口，音频编解码电路，以太网收发器，HDMI/DVI 输出，12 位 8 通道 ADC，12 位 2 通道 DAC 等。具体可参考 AC620V2 FPGA 开发板用户手册。

第二部分 基础实验

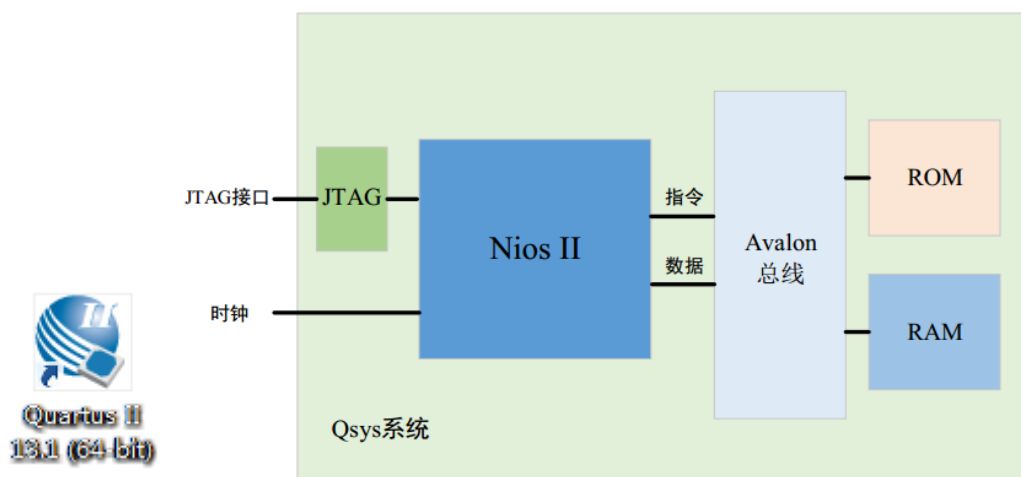
实验一 SOPC 开发流程

一、实验任务

1. 熟悉 FPGA 开发板硬件系统，了解各模块的作用，了解主板 I/O 分布情况。
2. 在 Qsys 中搭建最基础的硬件系统，并基于 Nios II 处理器编写软件，要求电脑能够通过串口接收开发板发送的数据“Hello from Nios II!”。

二、实验简介

本次实验的系统框图。如图所示，Nios II 处理器通过 Avalon 总线与 RAM 连接，其中 Avalon 总线用来传输指令与数据。除此之外，Qsys 系统还提供了一个 JTAG 接口，供用户下载和调试程序。

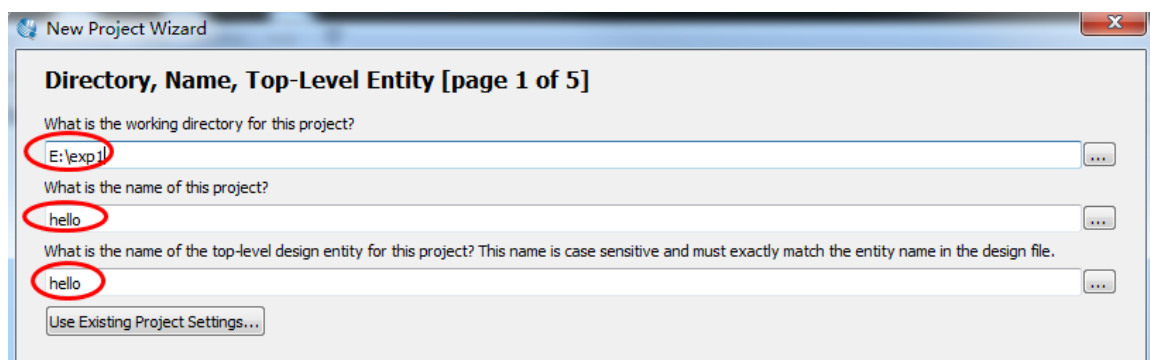


三、硬件设计

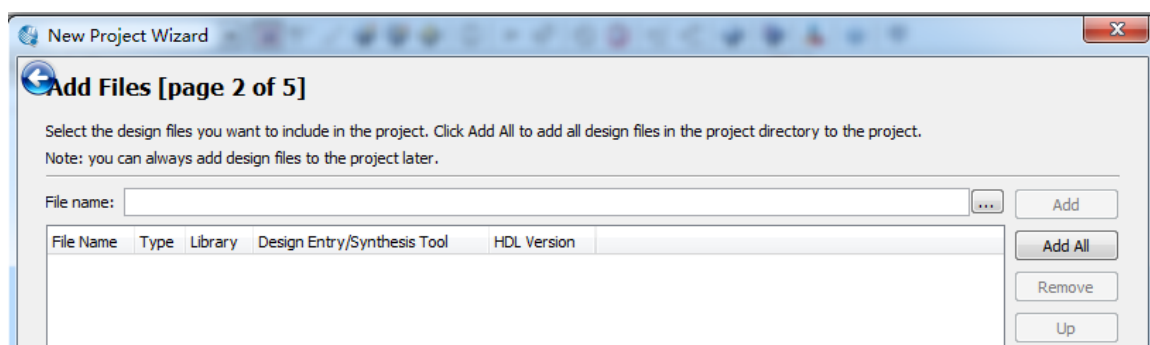
1. 创建 Quartus 工程

(1) 建立文件夹：建立一个文件夹，用于存放自己设计的工程文件，注意文件夹名用英文，不要带空格，例如可建立 E:\exp1。

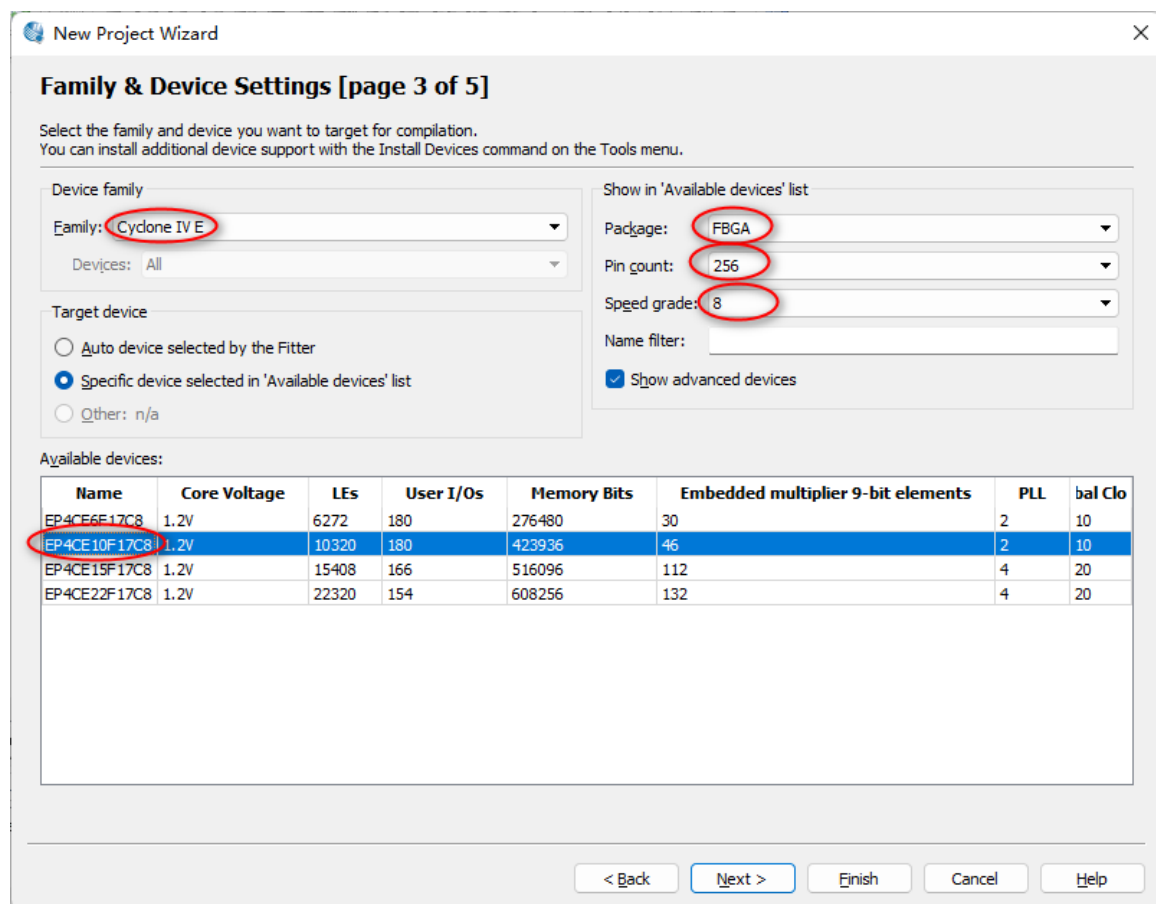
(2) 建立工程：打开“Quartus II”软件，选菜单 **【File】->【New Project Wizard...】**，选 **【Next】**，选择工程文件存放路径 E:\exp1（文件夹名不能用中文），输入工程名称 hello。顶层设计实体的名称默认与工程名称一致，设计时可修改。



选【Next】，如下图。添加设计文件到工程中，由于还未编写任何文件，这一步跳过。



继续选【Next】，如下图，选择目标芯片，此处要与开发板 FPGA 芯片一致，选择 Cyclone IV E 系列的 EP4CE10F17C8。



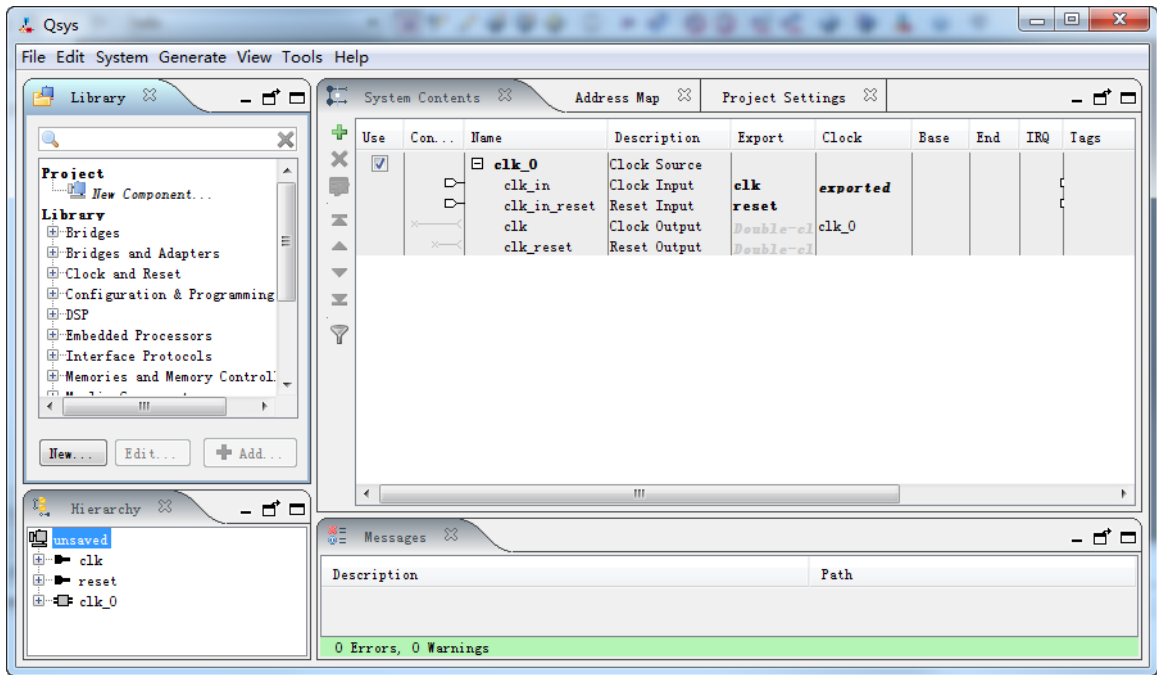
继续选【Next】，点击【Finish】完成工程创建向导。

2.创建 Qsys 系统

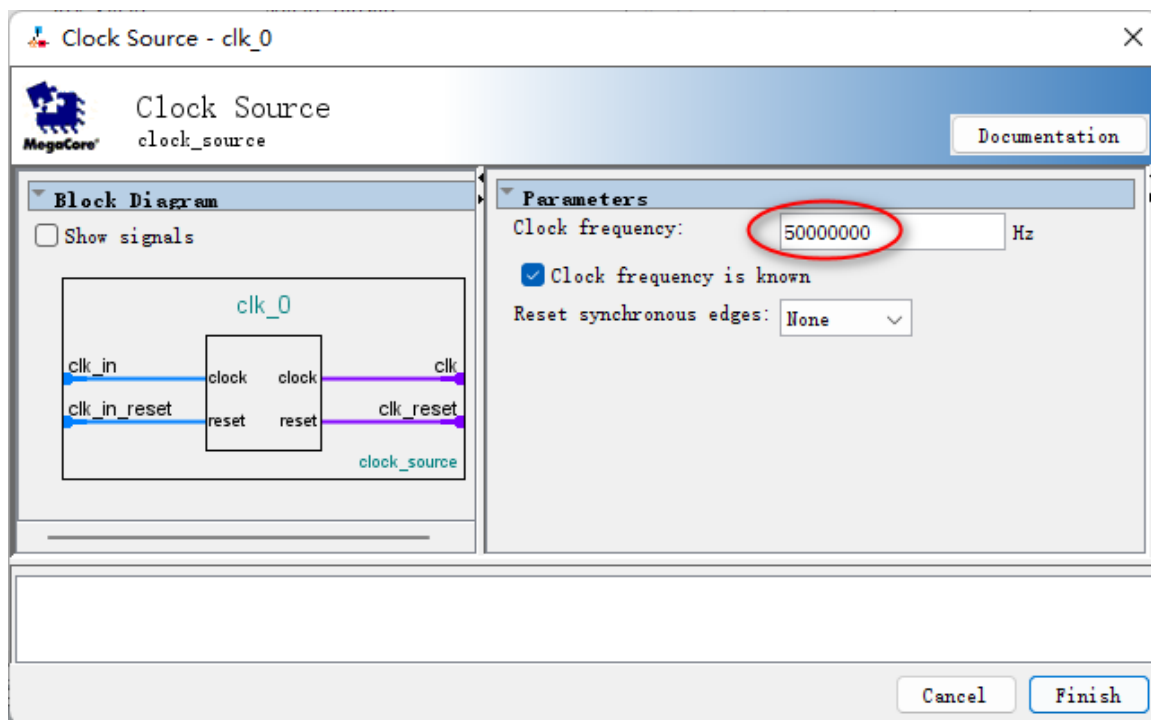
在 Quartus II 软件的菜单栏中选择【Tools】→【Qsys】选项来启动工具 Qsys。或者点击  按钮，打开 Qsys。出现 Qsys 工具界面。

(1) 时钟 clk IP 核

图中在“System Contents”标签页中可以看到已经包含时钟（clk）IP 核了。

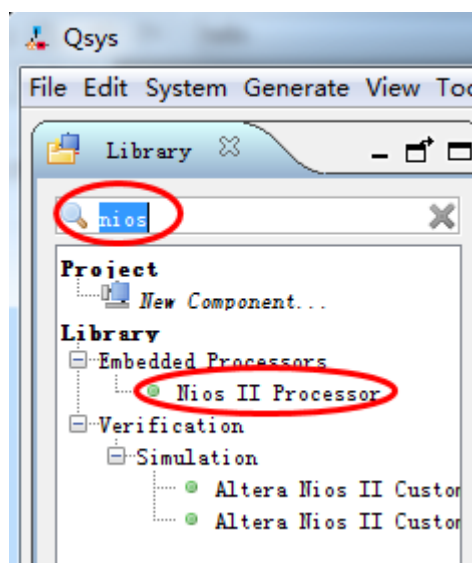


双击图中的 clk IP 核，设置时钟频率为 50M。

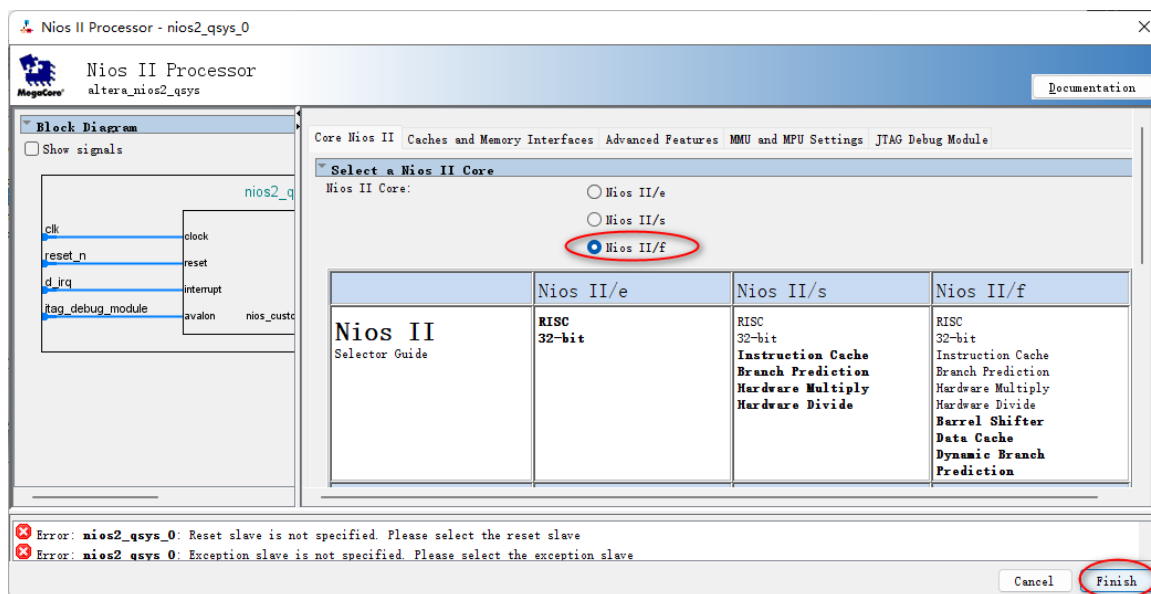


(2) 添加 Nios II IP 核

在 Qsys 工具左侧“Library”一栏的搜索框中输入“Nios”，然后在下方的搜索结果中找到“Nios II Processor”选项。

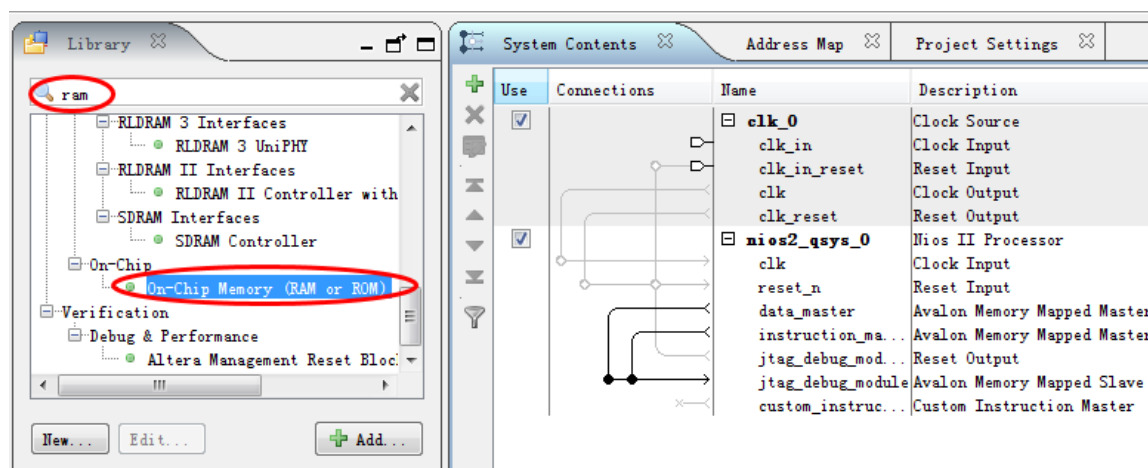


双击“Nios II Processor”选项，就会出现 Nios II 配置界面。可以看到 Nios II Core 已默认选择了“Nios II/F”内核。然后点击【Finish】。

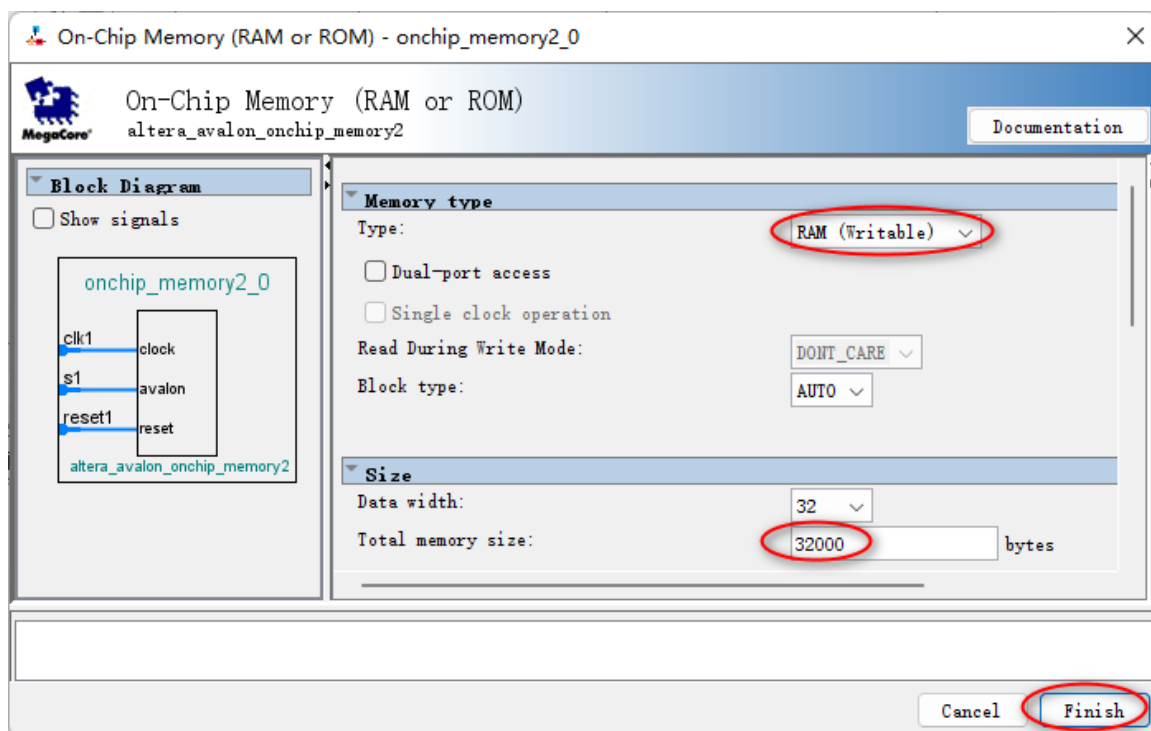


(3) 添加 On-Chip RAM 核

在 Qsys 工具左侧“Library”一栏的搜索框中输入“ram”，然后在下方的搜索结果中找到“On-Chip Memory (RAM or ROM)”选项。

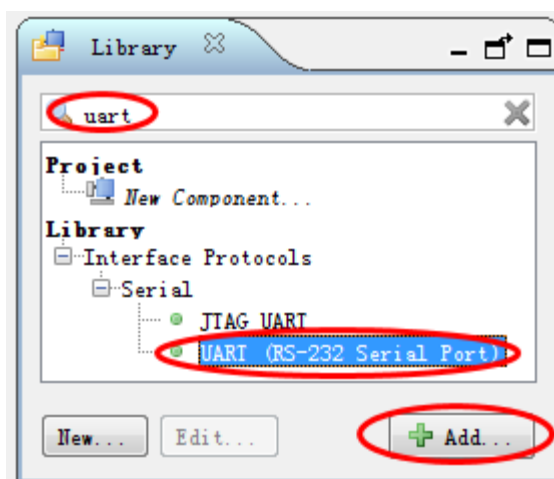


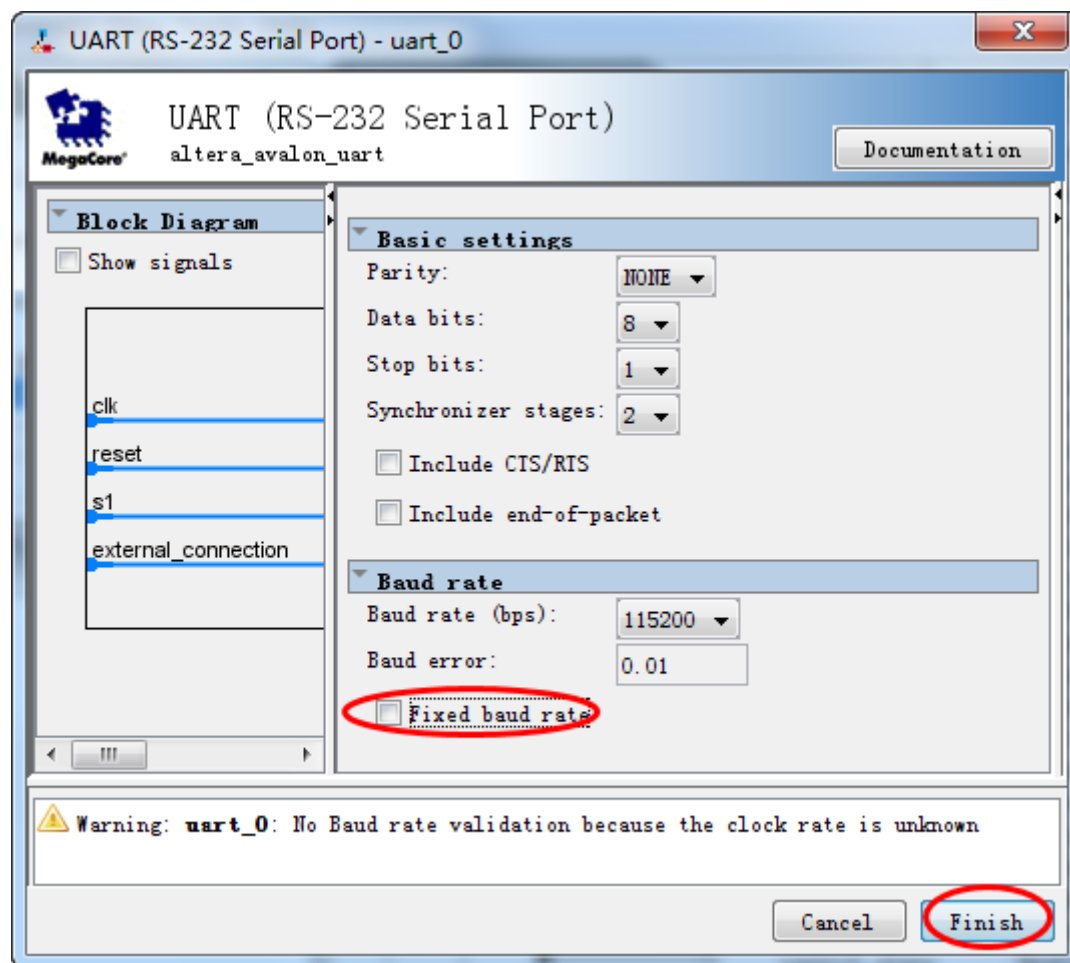
双击“On-Chip Memory (RAM or ROM)”这个选项，然后会弹出它的配置页面。在“Memory Type”一栏将“Type”选项设置为“RAM (Writable)”，然后将“Size”一栏中的“Total memory size”修改为“32000”，即约 32KB 内存。



(4) 添加 UART IP 核

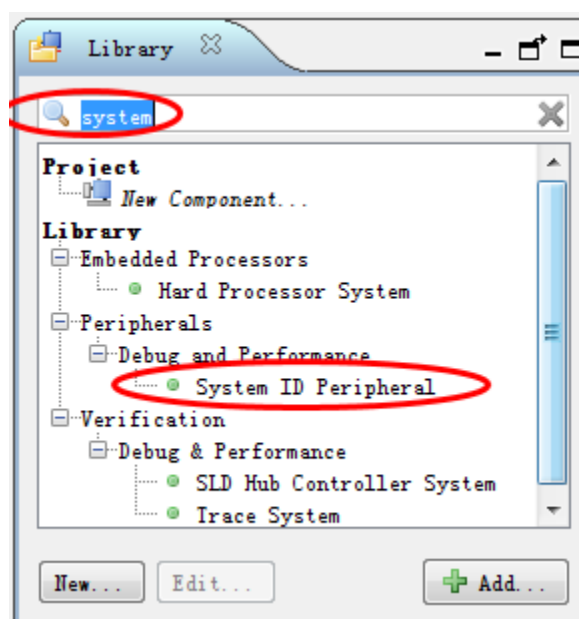
在“Library”库中找到“UART (RS-232 Serial Port)”IP 核。双击“UART”，将会弹出 UART 设置界面。按照默认设置，不作修改。直接点击【Finish】即可。

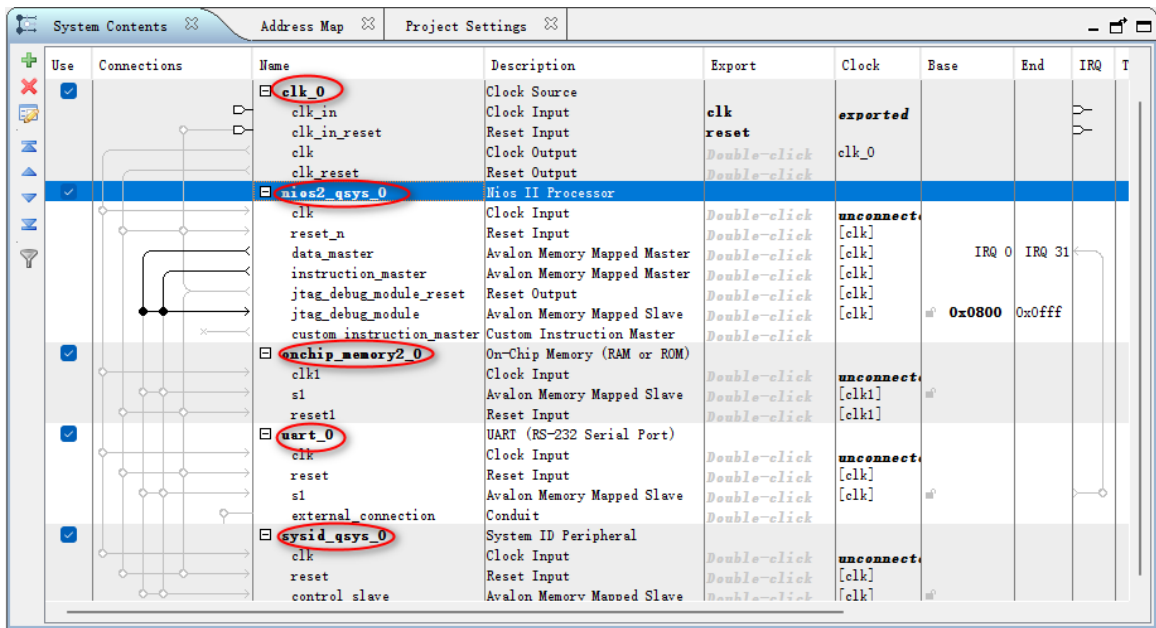




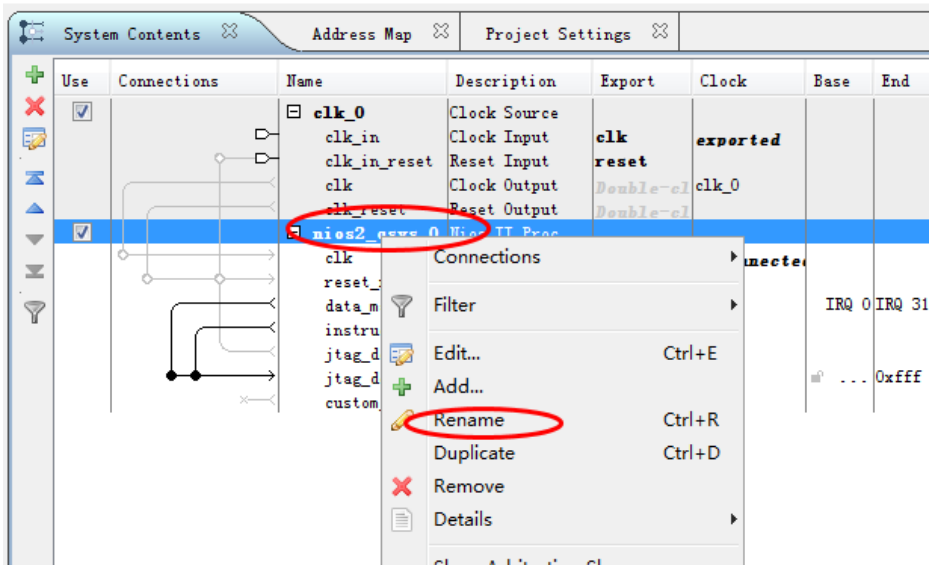
(5) 添加 System ID IP 核

在 Library 里搜索找到“System ID Peripheral”IP 核。双击该 IP 核，可以看到设置界面。按照默认设置，不作修改。直接点击【Finish】即可。

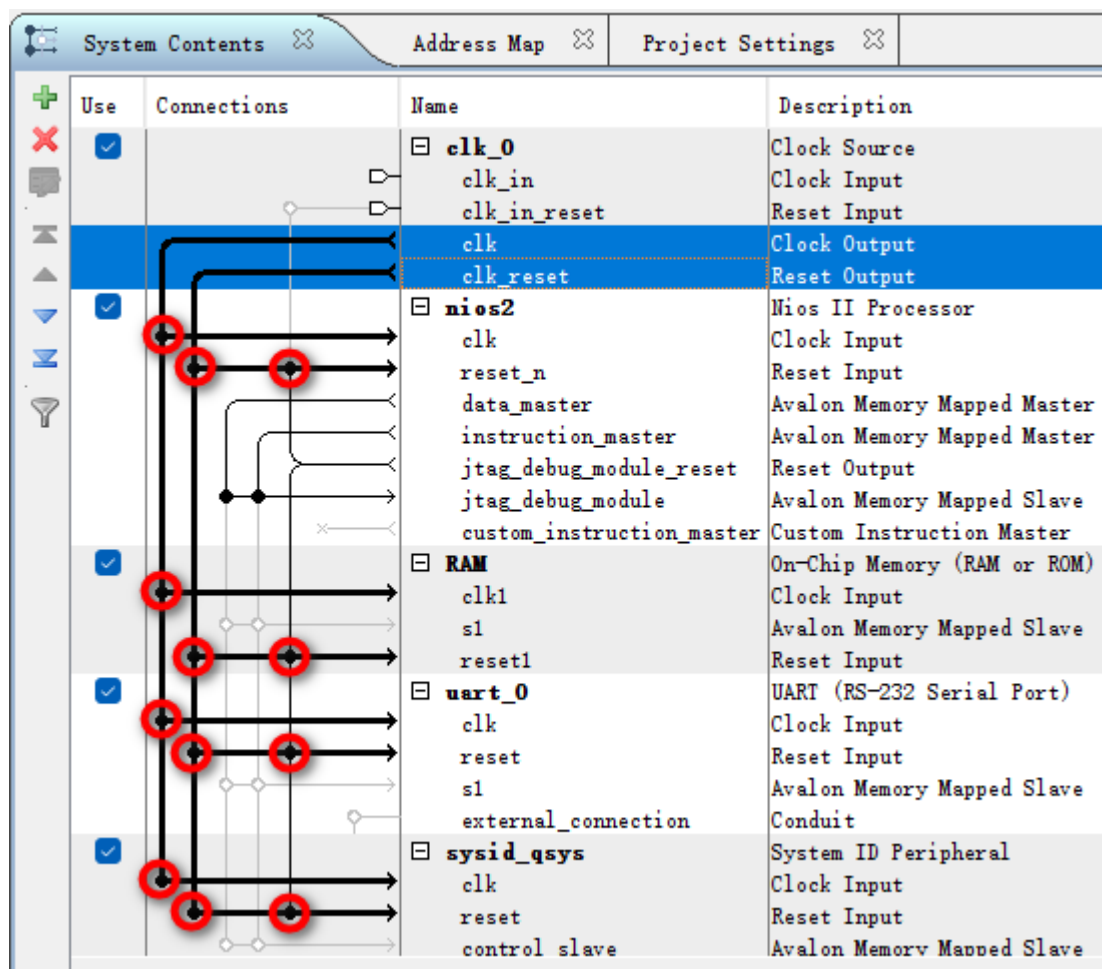




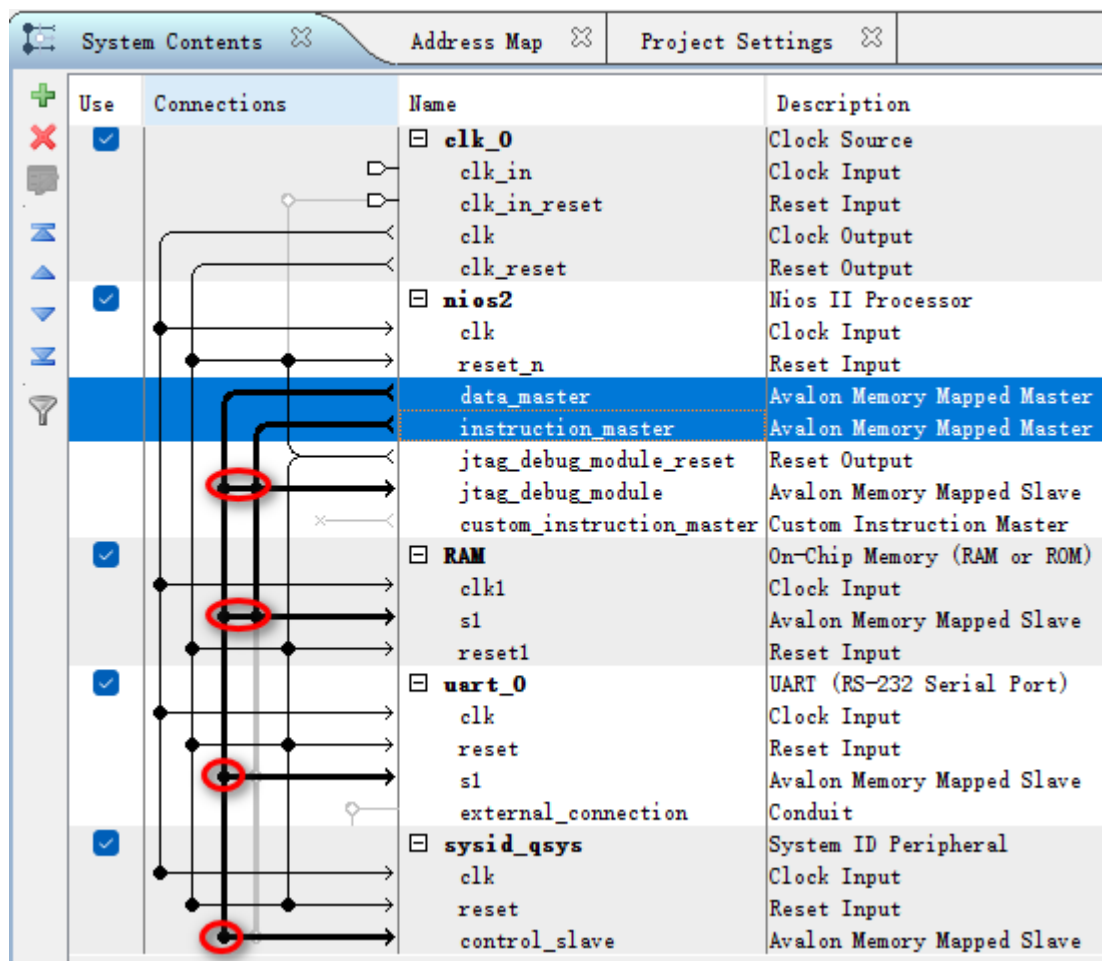
为了能够更直观地看出各 IP 核的功能，可以对这些 IP 核进行重命名。首先在“Name”一栏找到各 IP 核的名称(黑体部分)，用鼠标单击选中需要重命名的 IP 核，然后右键选择【Rename】；最后在文本框中修改名称，修改完成后按 Enter 键确认。



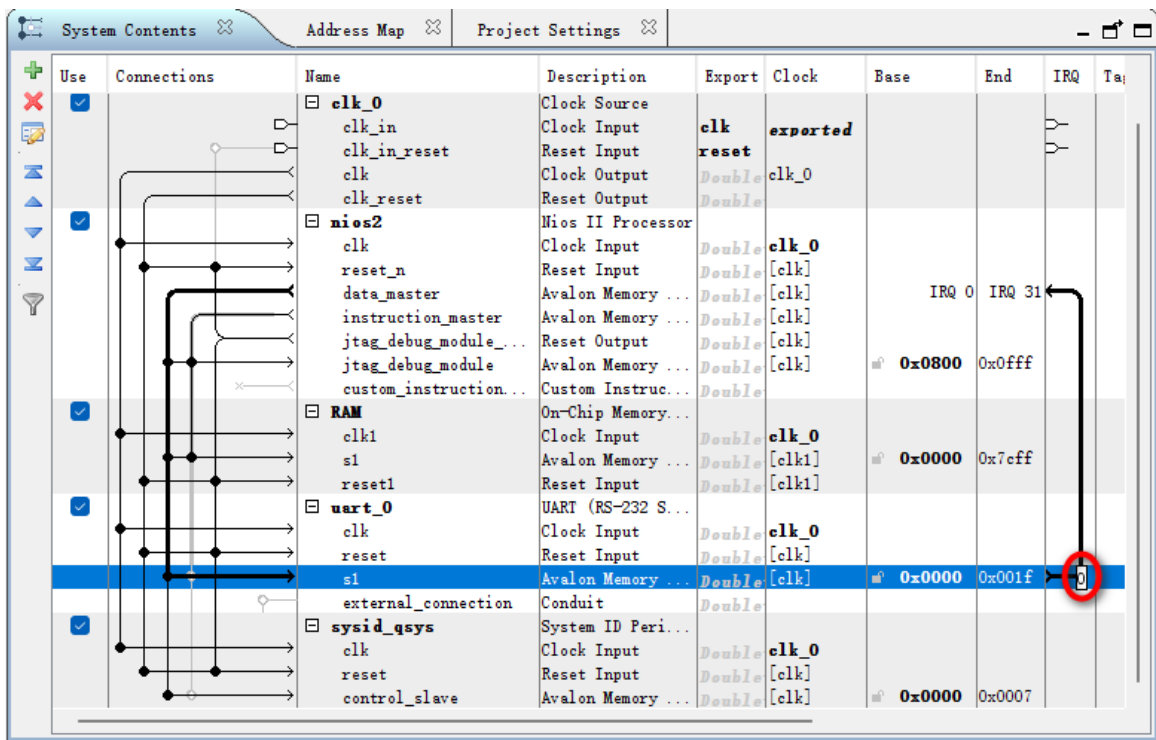
名字修改后，如下图所示。



接下来可以连接 Nios II 处理器 IP 核的数据主端口“data_master”和指令主端口“instruction_master”。这两个端口的连线遵循以下规则：数据主端口与所有的外设 IP 核连接，而指令主端口只连接存储器 IP 核。也就是说，如果是非存储器 IP 核，比如 System ID，只需要将它们的 Avalon Memory Mapped Slave 端口连接到 Nios II 处理器核的 data_master 端口上即可。而对于存储器 IP 核，比如说 onchip_RAM 等，需要将其 Avalon Memory Mapped Slave 端口同时连接到 Nios II 处理器核的 data_master 和 instruction_master 端口上。

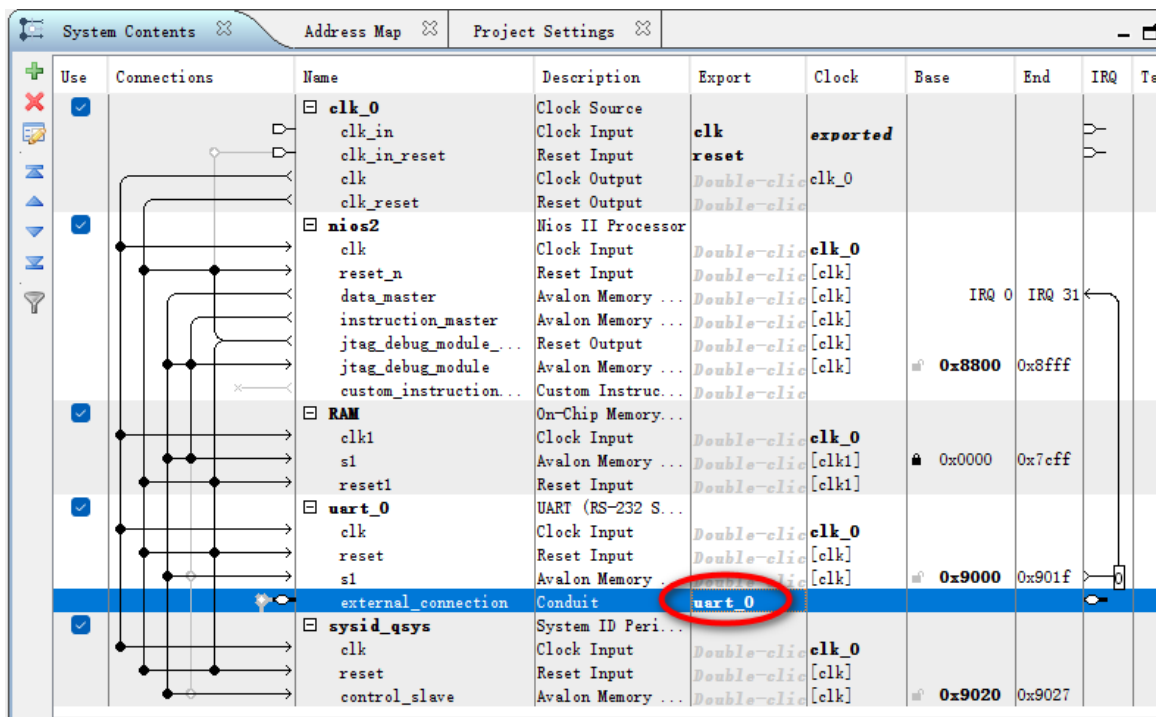


最后还要连接 Nios II 处理器 IP 核的“jtag_debug_module_reset”信号，并在“IRQ”一栏将 uart IP 核的中断信号与 Nios II 处理器连接起来。在连接中断 IRQ 的时候，Qsys 会自动为中断分配一个优先级。



(7) uart 串口信号导出

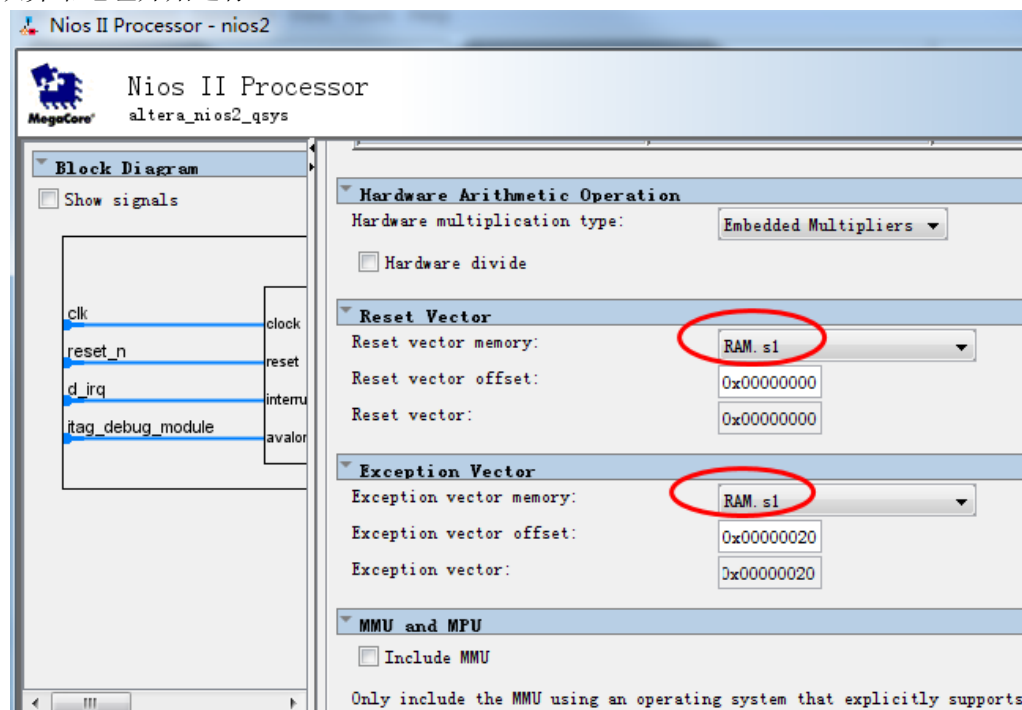
uart IP 核的输入输出信号的导出。双击 uart_0 IP 核 external_connection 的 Export 列，取名 uart_0。



(8) 设置复位向量和异常向量

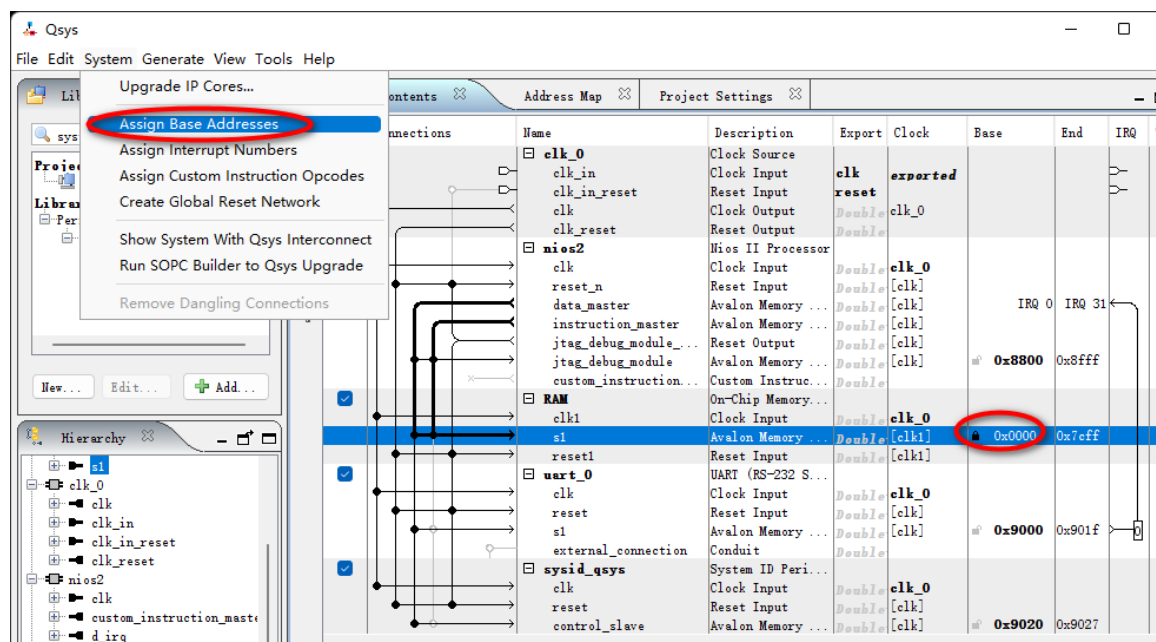
双击 Nios II IP 核，打开 IP 核的配置页面。复位向量是由所选存储器的基地址和偏移地址“Reset vector offset”共同计算出来的，在复位后，程序从复位地址开始运行。异常向量同样是由所选存储器的基地址和偏移地址“Exception vector offset”共同计算出来的，在异常事件发生时，

程序从异常地址开始运行。



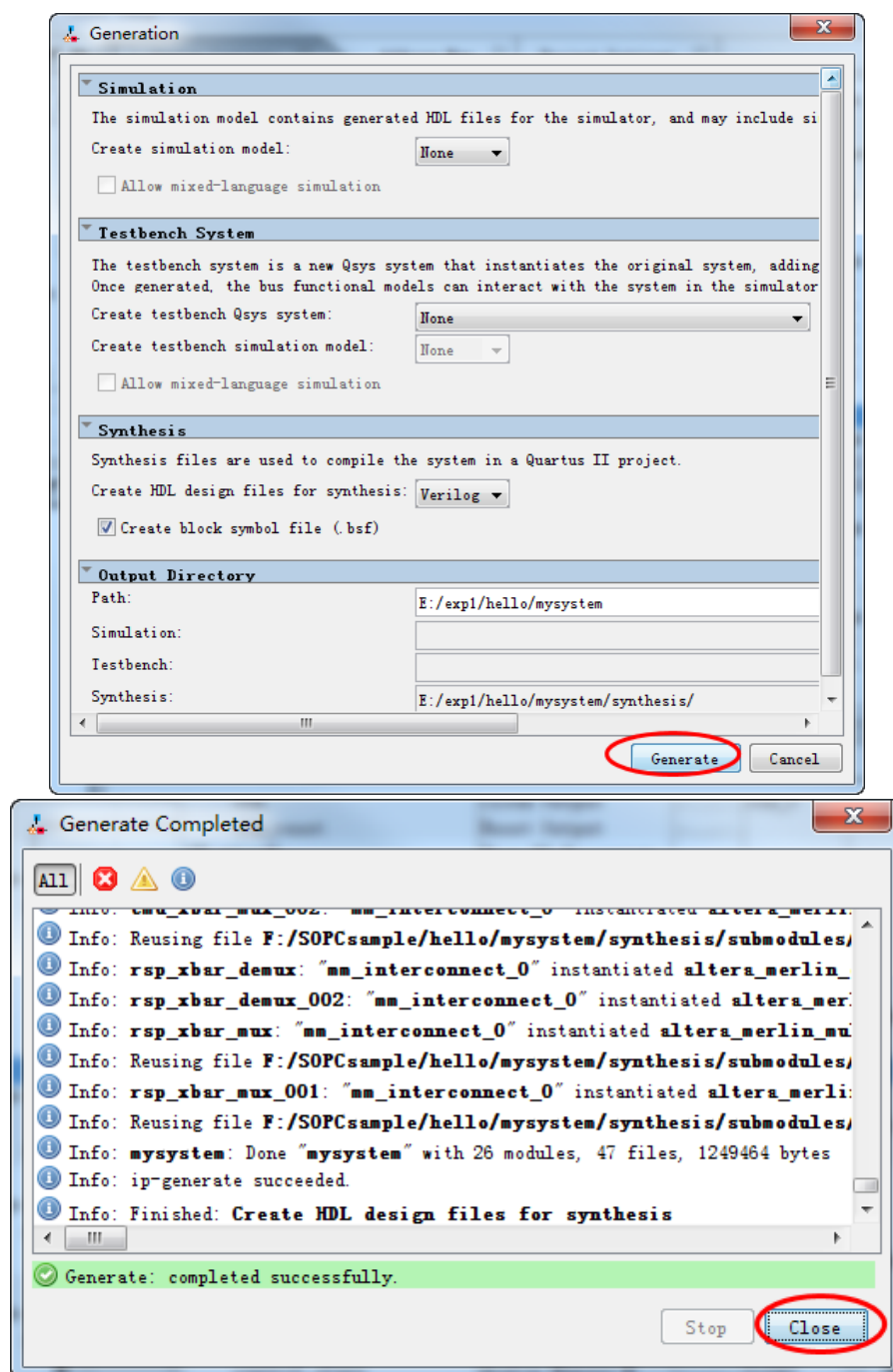
(9) 分配基地址

将 RAM 的基地址设为 0x0000, 并锁定。点击菜单栏中的【System】→【Assign Base Addresses】, Qsys 工具会自动为各个 IP 核分配基地址。



(10) 保存并生成 Qsys 系统

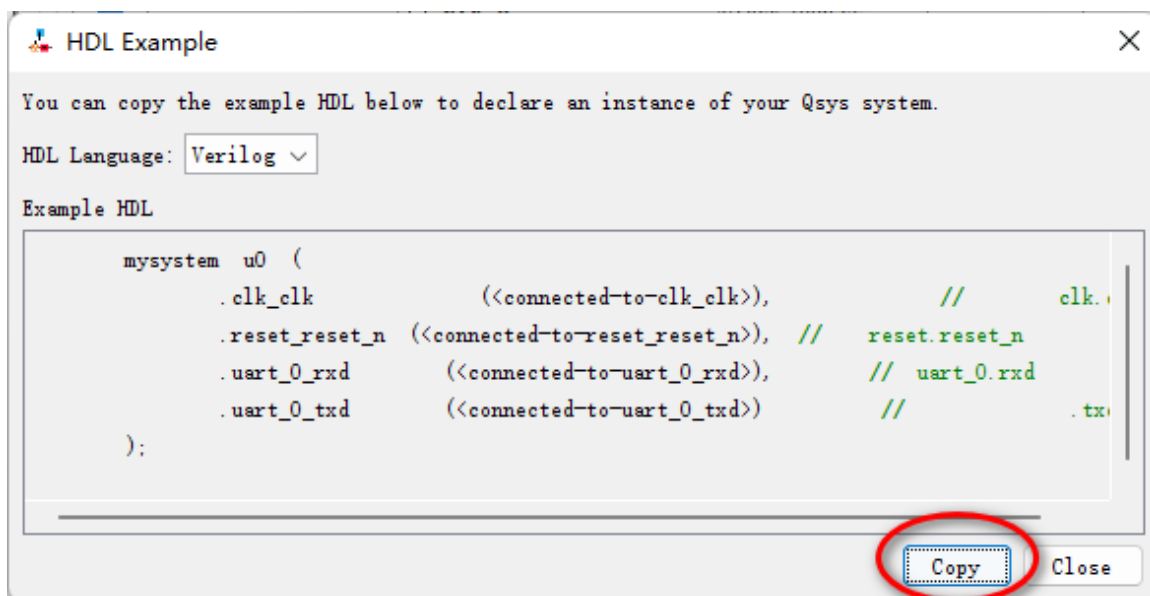
点击菜单栏中的【Save】, 在当前的工程目录下, 保存名为 mysystem.qsys。点击菜单栏中的【Generate】→【Generate...】选项, 弹出生成 Qsys 系统的设置界面。按照默认设置, 直接点击【Generate】。



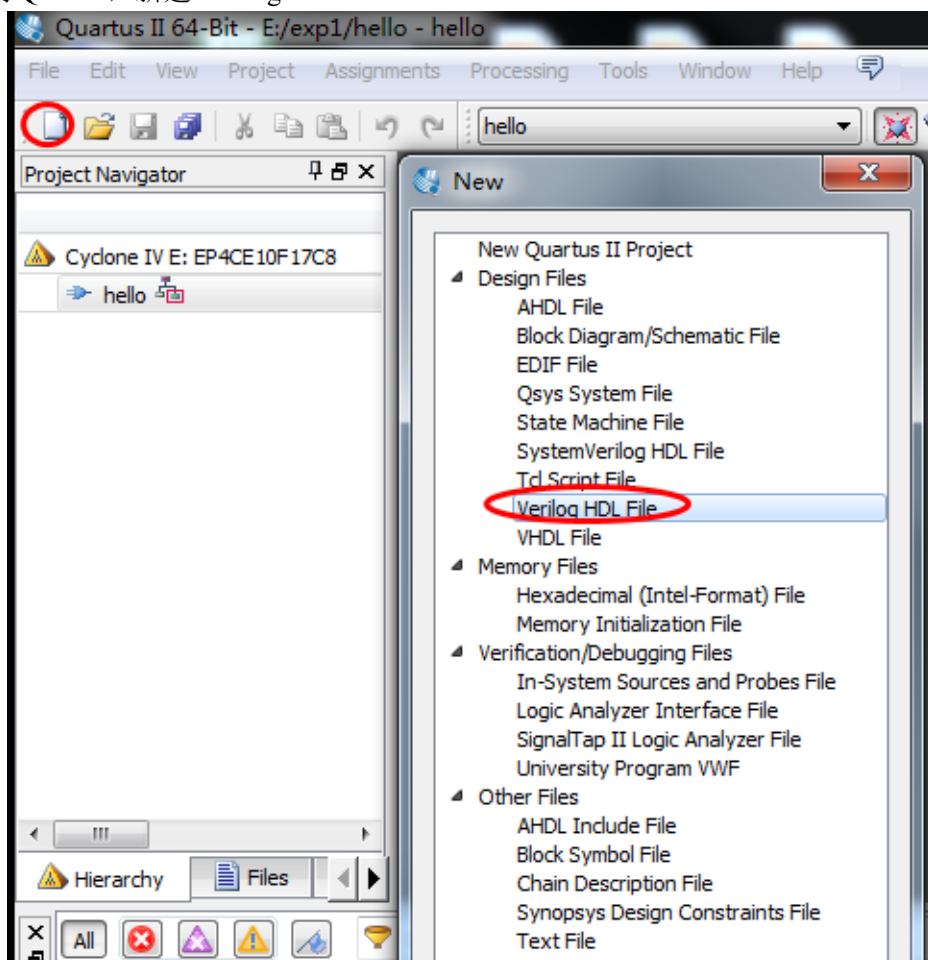
3.集成 Qsys 系统

(1) 例化 Qsys 系统

点击 Qsys 菜单栏的【Generate】→【HDL Example】。这是工具自动生成的 Qsys 系统例化代码，只需要点击【Copy】，就可以将其复制。图中的 clk_clk 端口是 Qsys 系统的时钟输入端口，应该给其输入 50Mhz 的时钟；reset_reset_n 是 Qsys 系统的复位端口，需要连接系统复位 sys_rst_n 上。



回到 Quartus，新建 Verilog HDL File。



然后在编辑窗口中粘贴，并修改成如下 Verilog 代码。最后保存名为 hello.v。

```
module hello(clk,rst,rxd,txd);
    input clk,rst,rxd;
    output txd ;
    mysystem u0 (
```

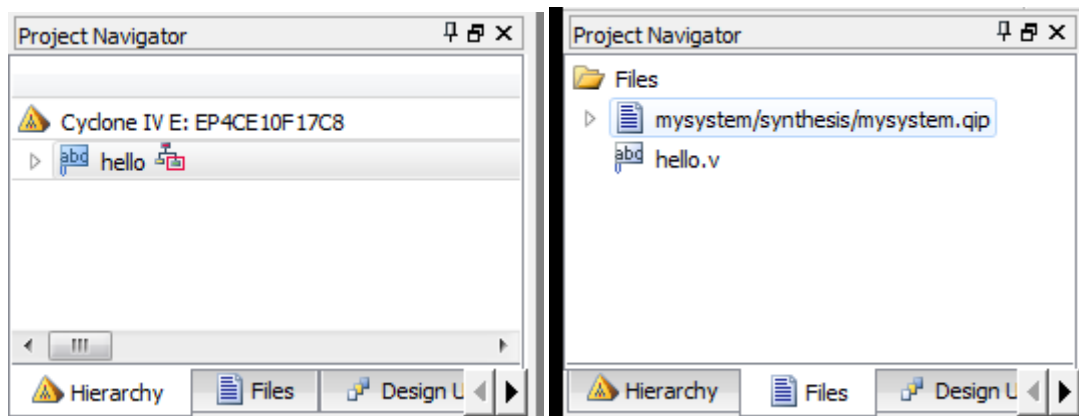
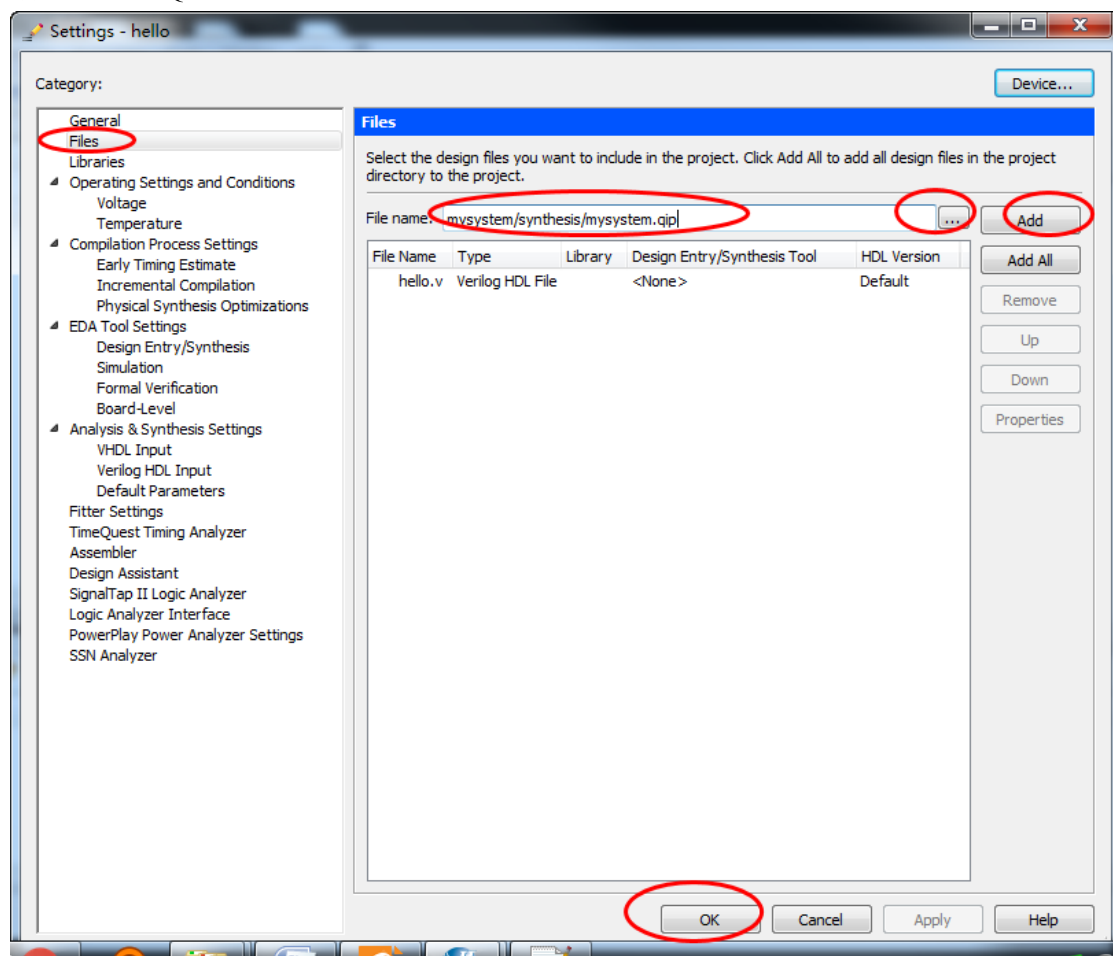
```

        .clk_clk      (clk),
        .reset_reset_n (rst),
        .uart_0_rxd    (rxd),
        .uart_0_txd    (txd)
    );
endmodule

```

(2) 添加 Qsys 系统

将 Qsys 生成的后缀名为“.qip”的文件添加到 Quartus II 工程里。点击 Quartus II 软件菜单栏【Assignment】→【Settings】进入设置页面。找到 E:\exp1\mysystem\synthesis 下的 mysystem.qip 文件，添加到 Quartus 工程。



(3) 分析综合与引脚锁定

编译的一个目的是为了查错，编译的另一个目的是为“引脚锁定”提供方便。选择菜单

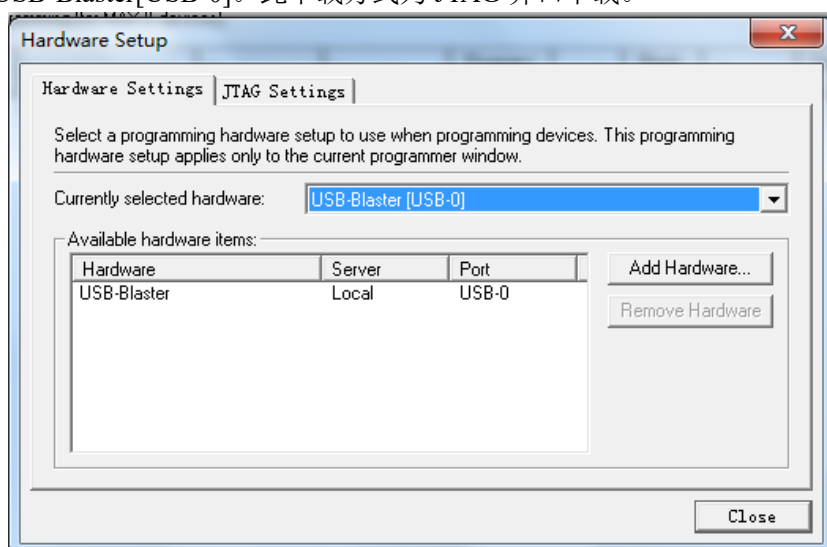
【Processing】->【Start】->【Start Analysis & Synthesis】或点击工具栏  图标，此时进行代码分析和综合。

引脚锁定是确定设计电路的每一端口对应目标芯片的具体引脚。选菜单【Assignments】->【Pin Planner】，在 Location 处输入对应引脚号（参考硬件结构图）。锁定的引脚可参考下图。

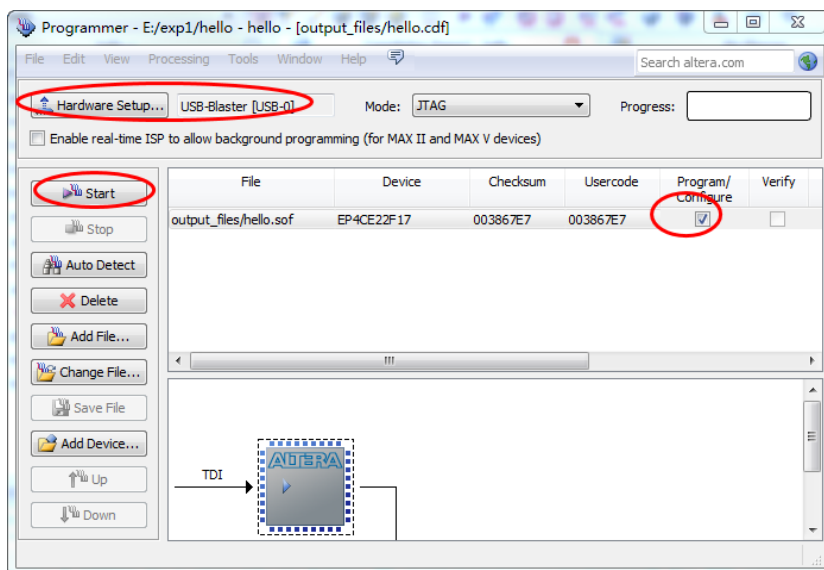
Named: *	Edit: 	clk
Node Name	Direction	Location
in altera_reserved_tck	Input	
in altera_reserved_tdi	Input	
out altera_reserved_tdo	Output	
in altera_reserved_tms	Input	
in clk	Input	PIN_E1
in rst	Input	PIN_E16
in rxd	Input	PIN_B5
out txd	Output	PIN_A6
<<new node>>		

(4) 全编译与下载

全编译的目的是生成与芯片相适应的时序仿真文件和编程下载文件。使用开发板提供的方口 USB 数据线连接开发板和电脑的 USB 口。选菜单【Processing】->【Start Compilation】全编译。选 Quartus II 菜单【Tools】->【Programmer】下载，点击【Hardware Setup...】按钮，在窗口中选择 USB-Blaster[USB-0]。此下载方式为 JTAG 并口下载。

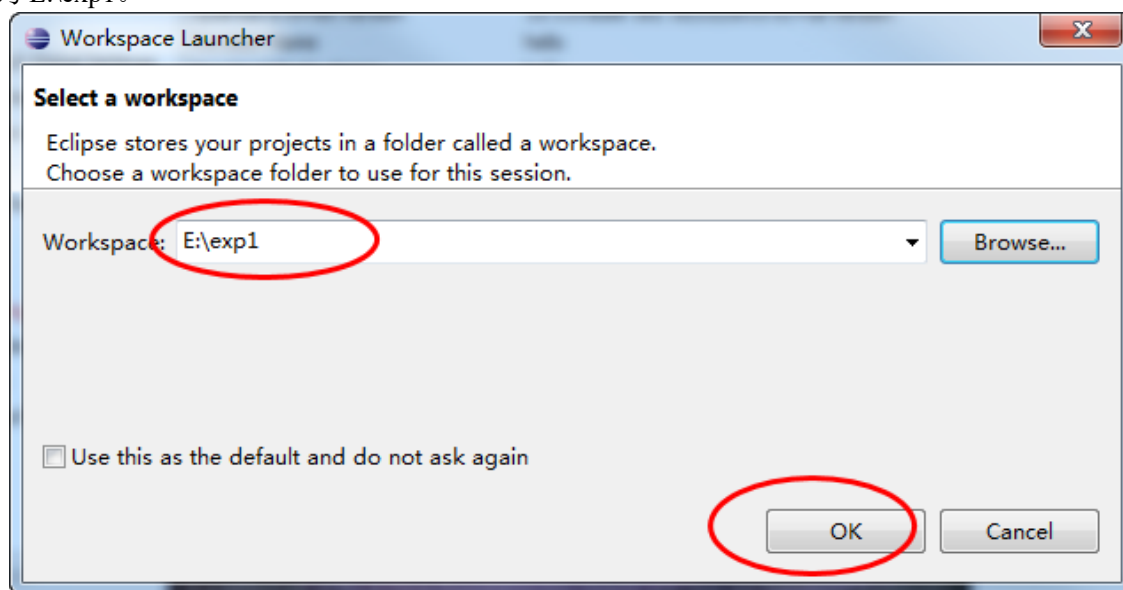


点击 Programmer 窗口中的【Start】按钮，进行配置下载。下载结束时，会给出成功或失败的提示信息。



四、软件设计

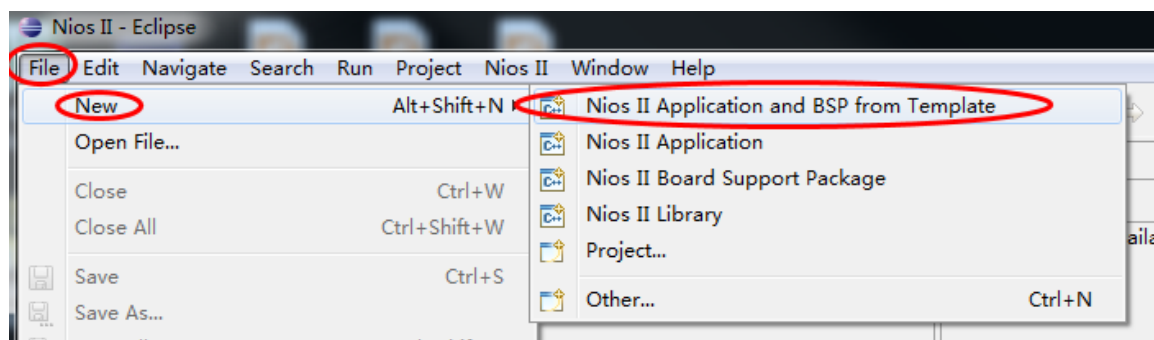
Quartus II 软件菜单栏中的【Tools】→【Nios II Software Build Tools for Eclipse】，来启动 Nios II SBT for Eclipse 软件。软件打开后，会弹出 Workspace Launcher 页面。这里将工作空间设置为 E:\exp1。



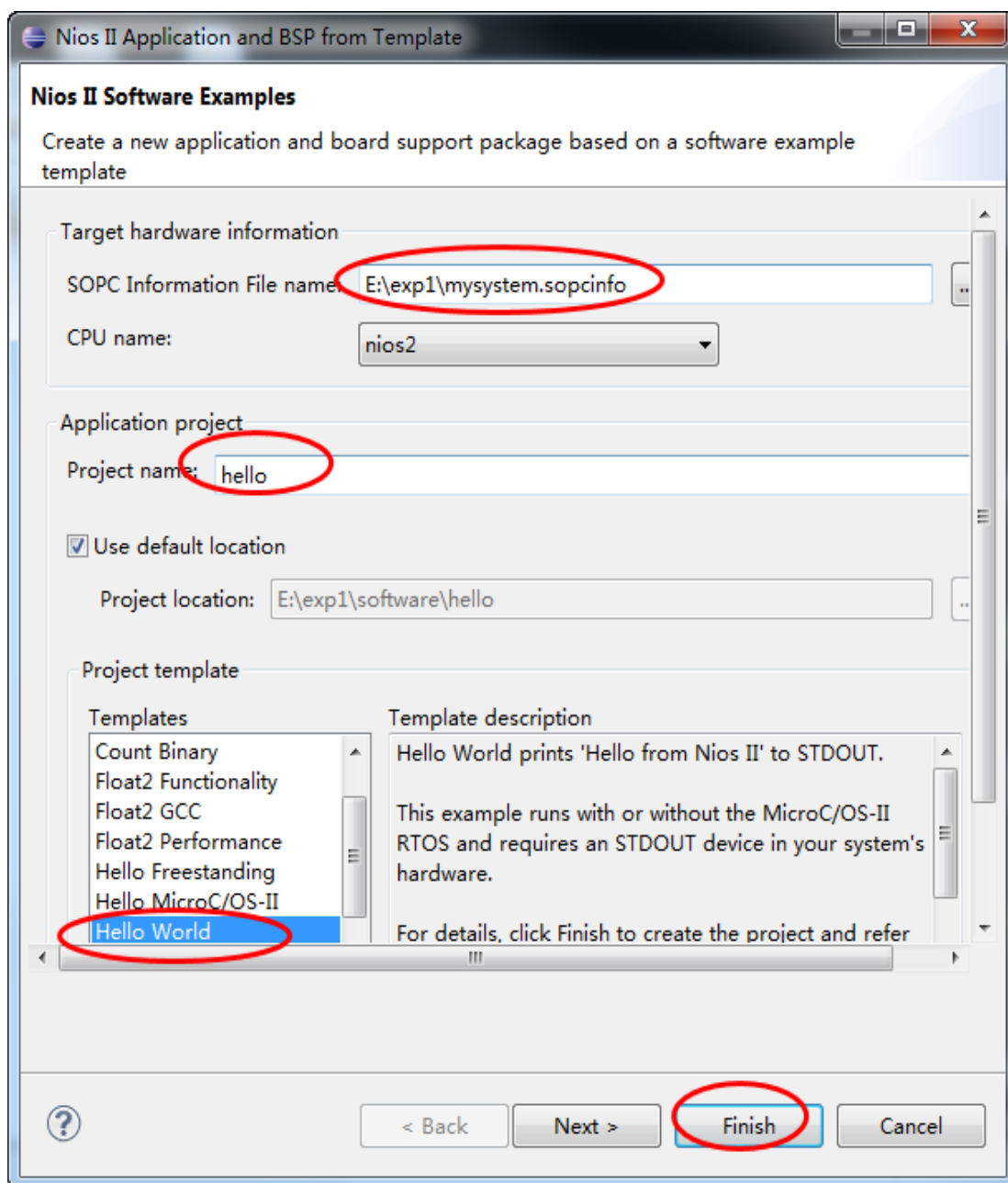
设置好工作空间后，点击【OK】进入 Nios II SBT for Eclipse 软件主界面。

1. 创建 Nios II 工程

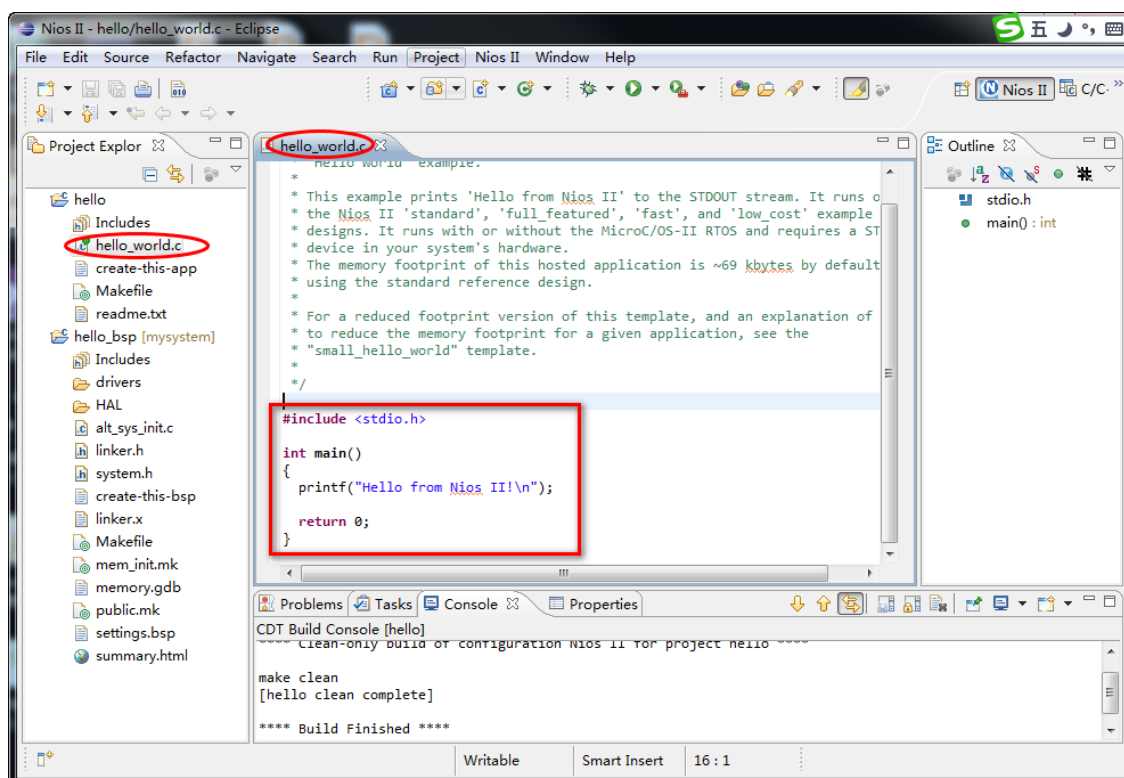
通过单击菜单栏中的【File】→【New】→【Nios II Application and BSP from Template】，来新建 Nios II 工程。



首先在“Target hardware information”一栏单击【...】按钮，选择 Nios II 工程所基于的目标硬件系统，mysystem.sopcinfo 文件。Nios II SBT for Eclipse 软件会自动识别 Qsys 系统中 CPU 名称，所以 CPU name 一项会自动生成。接下来，要给创建的 Nios II 工程命名（名称需要以英文字母开头，可以包含字母、数字和下划线，不能有中文及特殊字符），这里名为 hello。最后选择 hello world 工程模板。接下来点击【Next】，然后可以看到在创建应用工程(application project)的同时，会自动创建一个 BSP 工程（Board Support Package，板级支持包）。



在 Nios II SBT for Eclipse 软件的左侧“Project Explorer”窗口中可以看到两个工程：“hello”和“hello_bsp”。其中 hello 是 C/C++ 应用工程，而 hello_bsp 则提供了访问底层硬件（Qsys 系统）的函数库。打开 hello 工程里的“hello_world.c”文件。



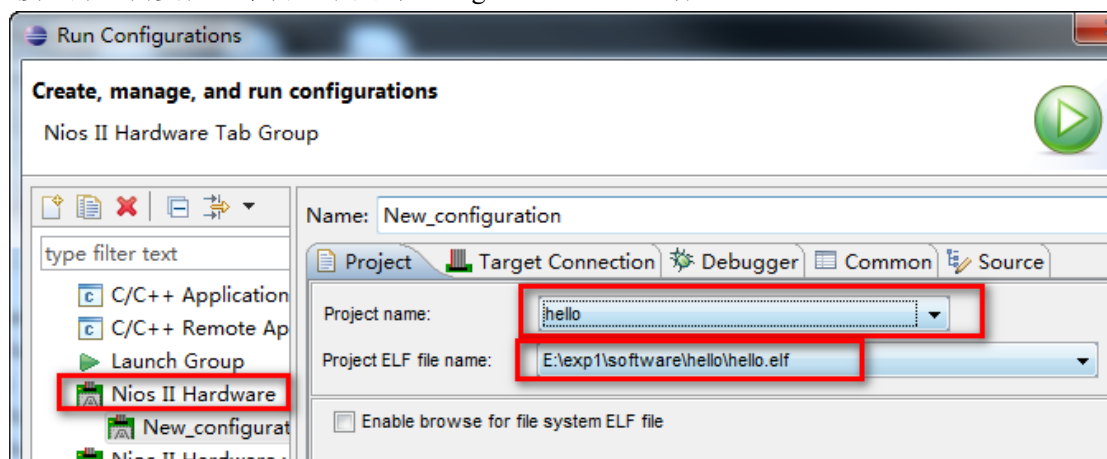
2. 编译 NiosII 工程

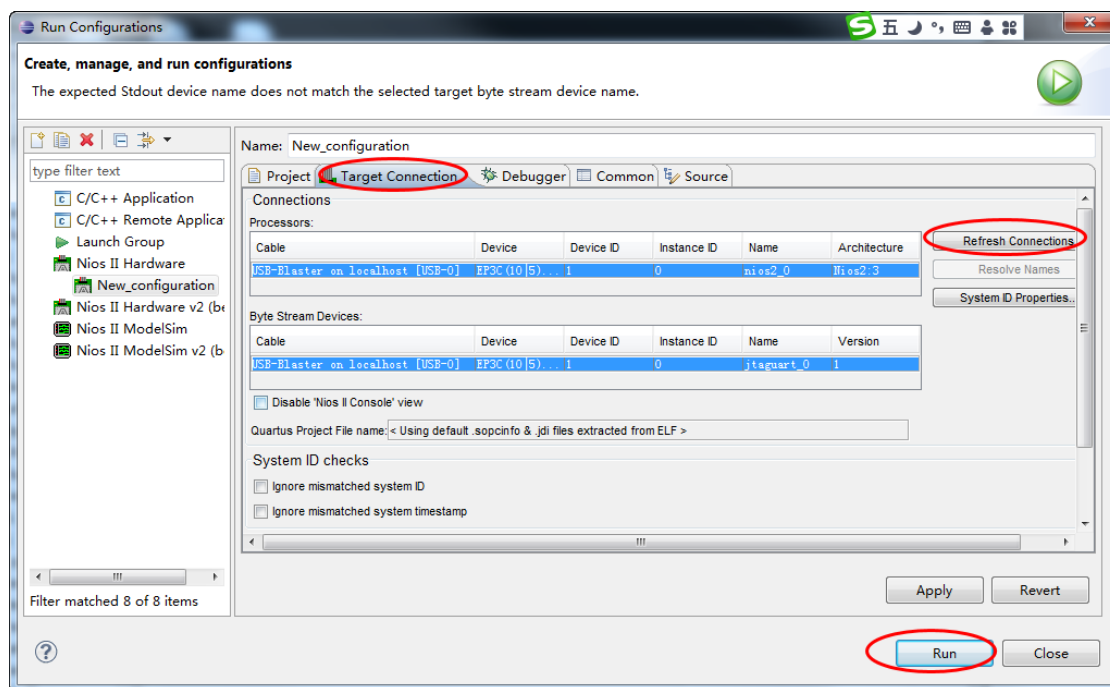
保存后，选择【Project】菜单的【Build All】来编译工程。

3. 下载验证

在 Nios II SBT for Eclipse 软件中右键 hello 工程，然后点击【Run As】→【Nios II Hardware】，将编译生成的可执行文件“hello.elf”下载到硬件系统中。

如果在程序下载的过程中弹出了“Run Configurations”窗口，提示找不到与 Nios 硬件系统的连接。那么需要在这个窗口中点击【Target Connection】标签。





接下来在“Target Connection”标签页中点击【Refresh Connections】按钮，这时软件便会自动识别开发板上的 Qsys 系统，并显示 Qsys 系统的相关信息。

打开串口调试助手，设置波特率 115200，其它默认。接着点击【Apply】→【Run】，将 hello.elf 文件下载至开发板中。下载结束后，程序自动开始运行。如果串口调试助手未接收到数据，可按动开发板的红色复位键。



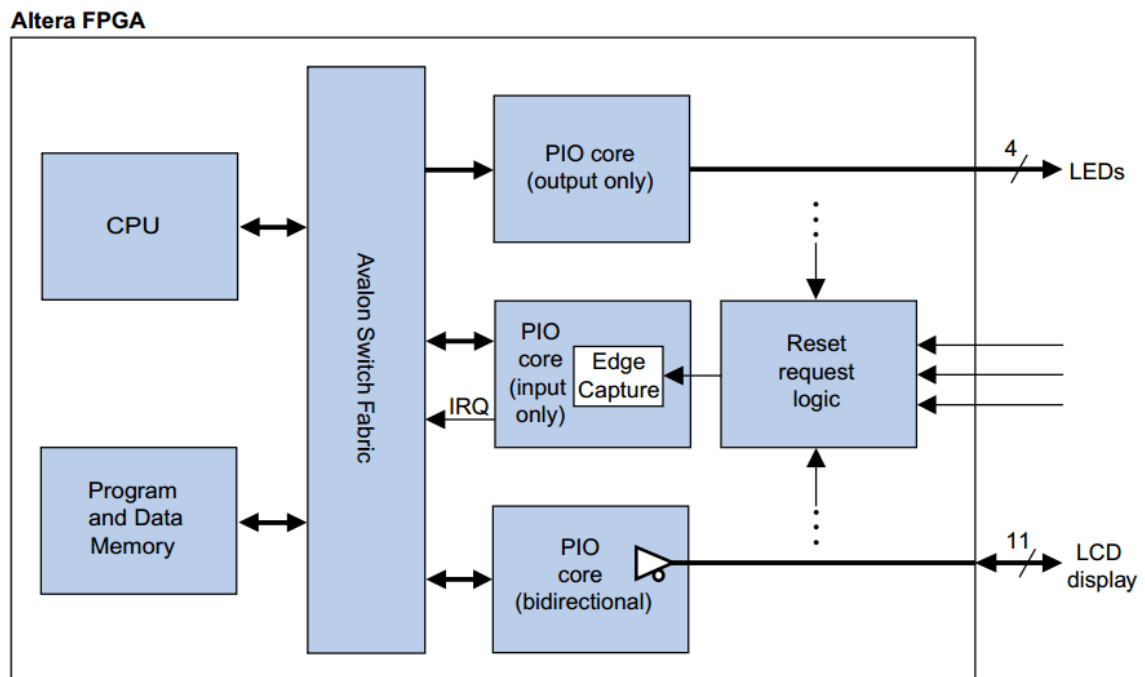
实验二 PIO IP 核

一、实验任务

1. Qsys 系统中加入 PIO IP 核，熟悉 PIO 核的功能。
2. 通过 2 个按键中断控制 4 个 LED 流水灯的运行状态。

二、实验简介

PIO IP 核为 Avalon-MM 从端口和通用 I/O 端口提供了一个存储器映射（memory-mapped）接口。PIO 的 I/O 端口可以连接到片内用户逻辑（Verilog 语言完成的电路部分），也可以连接到与外部器件相连的 FPGA 引脚。每个 PIO 核能提供最多 32 个 I/O 端口，用户可以在一个 Qsys 系统里添加多个 PIO IP 核。PIO 为用户逻辑或外围器件提供了简单的 I/O 控制，如控制 LED 灯、获取按键的电平变化信息、控制显示设备、与片外器件通信等。



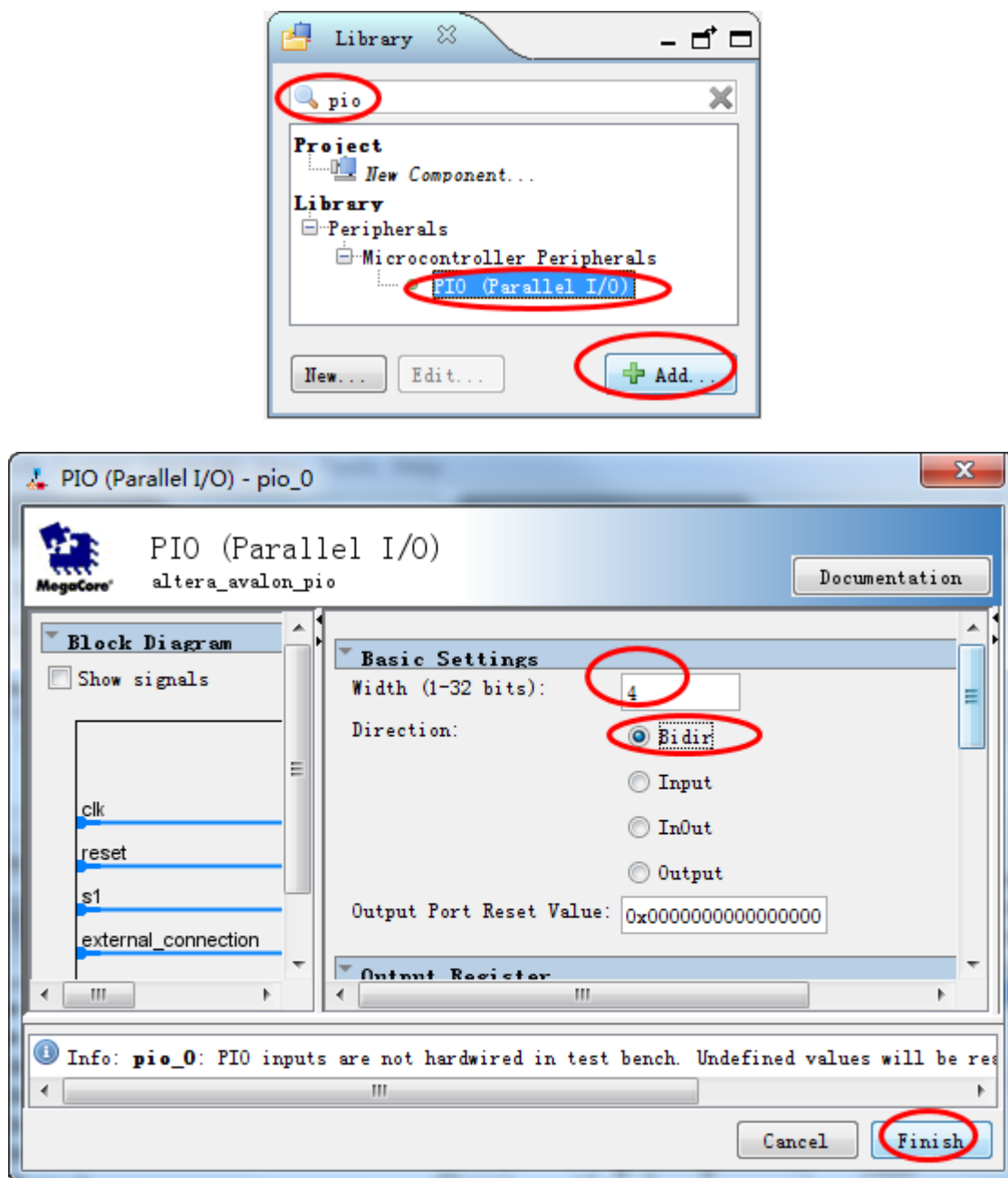
三、硬件设计

1. 创建 Quartus 工程

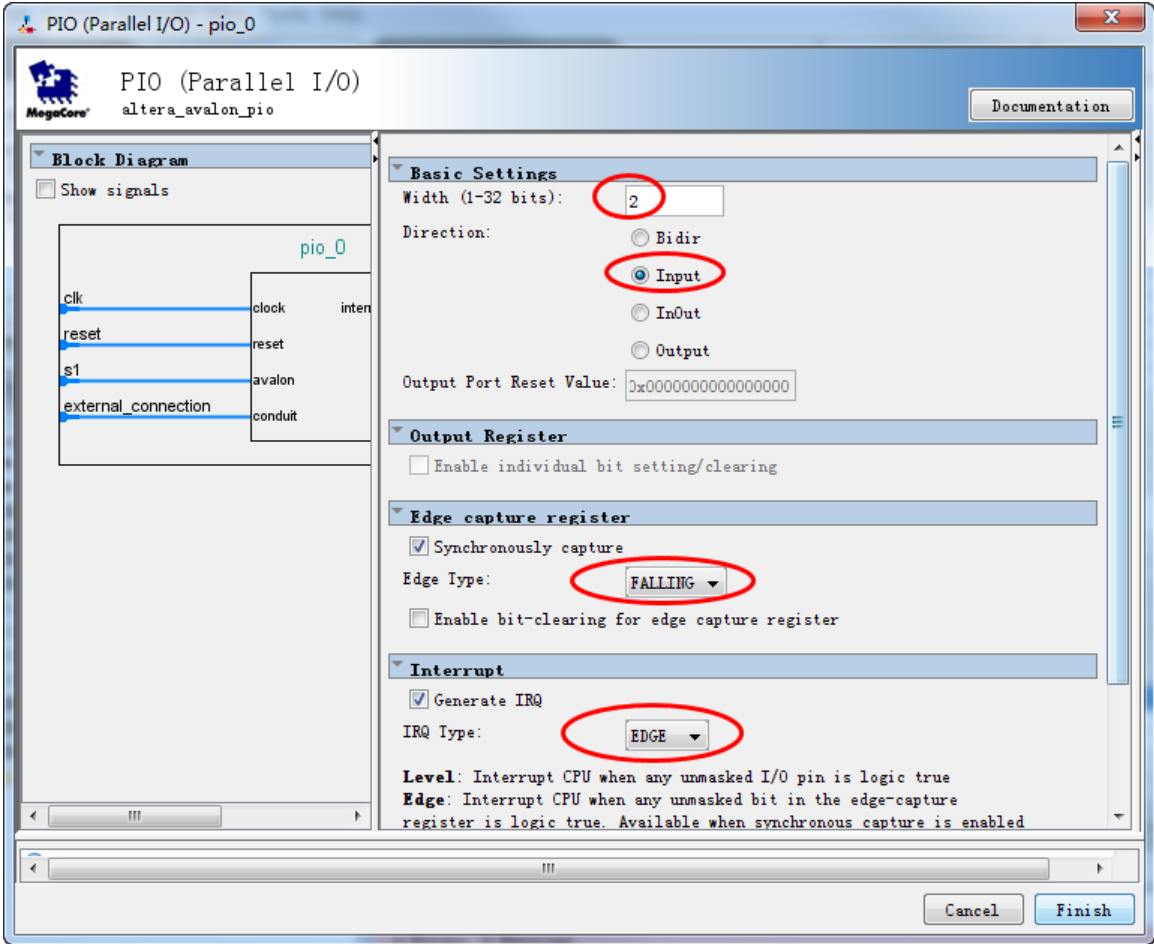
建立工程文件夹，比如 E:\exp2，工程名 pio，顶层设计实体 pio。

2. 创建 Qsys 系统

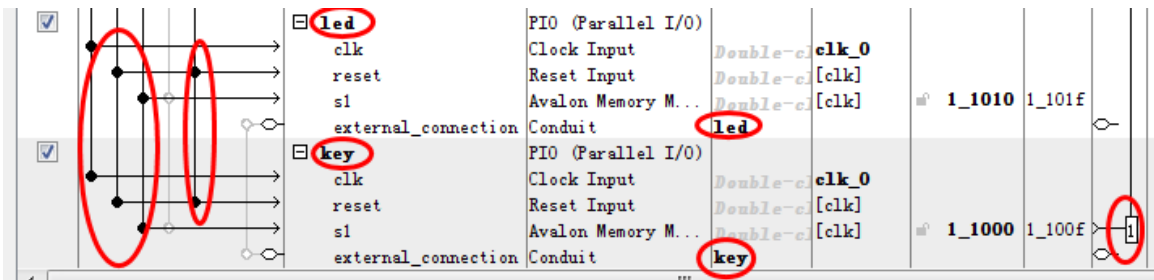
在实验 1 的基础上，删除 uart 核，然后添加两个 PIO 核。一个 PIO 核用于连接 LED 灯，设置为 4 位，双向端口。



另一个 PIO 核用于连接按键，设置为 2 位，输入端口，下降沿触发中断。



添加后，按图连线，分别重命名为 led 和 key，外部连接名为 led 和 key。另外 key 的中断号设置为 1。修改后重新分配基地址。



Qsys 总的硬件设计如下：

System Contents		Address Map		Project Settings					
Use	Connections	Name	Description	Export	Clock	Base	End	IRQ	Tags
☒		clk_0 clk_in clk_in_reset clk clk_reset	Clock Source Clock Input Reset Input Clock Output Reset Output	clk reset Double-cl Double-cl	exported clk_0				
☒		nios2 clk reset_n data_master instructio... jtag_debug... jtag_debug... custom_ins...	Mios II P... Clock Input Reset Input Avalon Me... Avalon Me... Reset Output Avalon Me... Custom In...	Double-cl Double-cl Double-cl Double-cl Double-cl Double-cl Double-cl	clk_0 [clk] [clk] [clk] [clk] [clk] [clk]	IRQ 0 IRQ 31			
☒		RAM clk1 s1 reset1	On-Chip M... Clock Input Avalon Me... Reset Input	Double-cl Double-cl Double-cl	clk_0 [clk1] [clk1]	0x0000 0x7cff			
☒		sysid_qsys clk reset control_slave	System ID... Clock Input Reset Input Avalon Me...	Double-cl Double-cl Double-cl	clk_0 [clk] [clk]	0x9020 0x9027			
☒		led clk reset s1 external_c...	PIO (Para... Clock Input Reset Input Avalon Me... Conduit	Double-cl Double-cl Double-cl Double-cl	clk_0 [clk] [clk] led	0x9010 0x901f			
☒		key clk reset s1 external_c...	PIO (Para... Clock Input Reset Input Avalon Me... Conduit	Double-cl Double-cl Double-cl Double-cl	clk_0 [clk] [clk] key	0x9000 0x900f			

保存，并重新生成 Qsys 系统。

3.集成 Qsys 系统

(1) 例化 Qsys 系统

回到 Quartus 中，添加 Qsys 生成的.qip 文件，再新建 verilog 文件，保存名为 pio.v。

```
module pio(clk,rst,led,key);
    input clk,rst;
    inout [3:0] led;
    input [1:0] key;
    mysystem u0 (
        .clk_clk    (clk),
        .reset_reset_n (rst),
        .led_export  (led),
        .key_export  (key)
    );
endmodule
```

(2) 分析综合与引脚锁定

设置 pio 为顶层设计实体。分析综合，引脚锁定可以参考下图。

Named: *  Edit:  		
Node Name	Direction	Location
altera_reserved_tck	Input	
altera_reserved_tdi	Input	
altera_reserved_tdo	Output	
altera_reserved_tms	Input	
clk	Input	PIN_E1
key[1]	Input	PIN_M16
key[0]	Input	PIN_E15
led[3]	Bidir	PIN_A3
led[2]	Bidir	PIN_A4
led[1]	Bidir	PIN_B3
led[0]	Bidir	PIN_A2
rst	Input	PIN_E16
<<new node>>		

(3) 全编译与下载

使用开发板提供的方口 USB 数据线连接开发板和电脑的 USB 口。选菜单【Processing】->【Start Compilation】进行全编译。Quartus II 菜单【Tools】->【Programmer】下载。

四、软件设计

将工作空间设置为 E:\exp2。点击【OK】进入 Nios II SBT for Eclipse 软件主界面。如果有工程，选中后，右键删除。

1. 创建 NiosII 工程

通过单击菜单栏中的【File】→【New】→【Nios II Application and BSP from Template】，来新建 Nios II 工程。取名 pio。修改 hello_world.c 的内容如下。

```
#include "stdio.h"
#include "altera_avalon_pio_regs.h"
#include "system.h"
#include "unistd.h"
#include "sys/alt_irq.h"
#include "alt_types.h"
alt_u8 data = 0;
volatile alt_u8 led_en = 0;
void key_isr() {
    IOWR_ALTERA_AVALON_PIO_IRQ_MASK(KEY_BASE, 0x00);
    data = IORD_ALTERA_AVALON_PIO_EDGE_CAP(KEY_BASE);
    if (data & 0x01) {
        led_en = 1;
    } else if (data & 0x02) {
        led_en = 0;
    }
    IOWR_ALTERA_AVALON_PIO_EDGE_CAP(KEY_BASE, 0x3);
    IOWR_ALTERA_AVALON_PIO_IRQ_MASK(KEY_BASE, 0x3);
}
```

```
int main() {
    int i;
    IOWR_ALTERA_AVALON_PIO_DIRECTION(LED_BASE, 0xF);
    IOWR_ALTERA_AVALON_PIO_EDGE_CAP(KEY_BASE, 0x3);
    IOWR_ALTERA_AVALON_PIO_IRQ_MASK(KEY_BASE, 0x3);
    alt_ic_isr_register(KEY_IRQ_INTERRUPT_CONTROLLER_ID,
        KEY_IRQ, key_isr, 0, 0);
    led_en = 0;
    while (1) {
        if (led_en) {
            for (i = 0; i <= 3; i++) {
                IOWR_ALTERA_AVALON_PIO_DATA(LED_BASE, ~(1<<i));
                usleep(200000);
            }
        } else {
            IOWR_ALTERA_AVALON_PIO_DATA(LED_BASE, 0xf);
        }
    }
    return 0;
}
```

2.编译 NiosII 工程

保存后，选择【Project】菜单的【Build All】来编译工程。

3.下载验证

选择【Run】→【Run Configurations...】，设置下载，按键，观察 LED 流水灯开启与停止。按复位键，观察效果。

实验三 UART IP 核

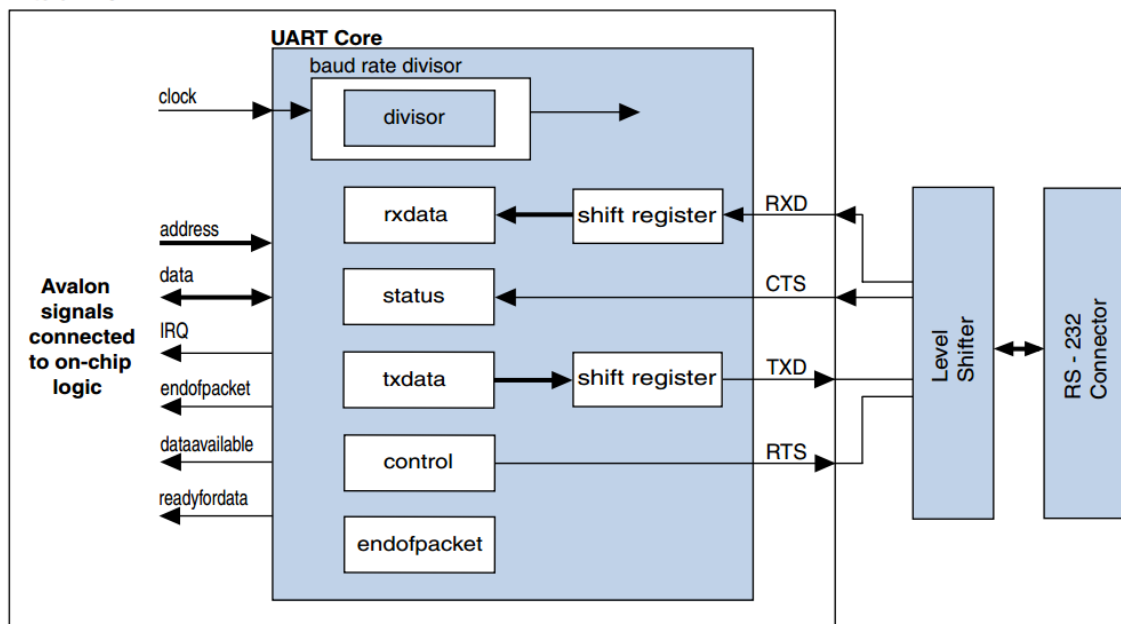
一、实验任务

1. Qsys 系统中加入 UART IP 核，熟悉 UART 核的功能。
2. 通过使用 UART IP 核和 PC 进行回环，将从 PC 收到的字符回送给 PC。

二、实验简介

UART 核提供了一个 Avalon 存储器映射（Avalon-MM）的接口，这个接口使得 Avalon-MM 的主要周边设备（例如 Nios II 处理器）通过读和写数据、控制寄存器，就能实现和 UART 核进行通信。

Altera FPGA



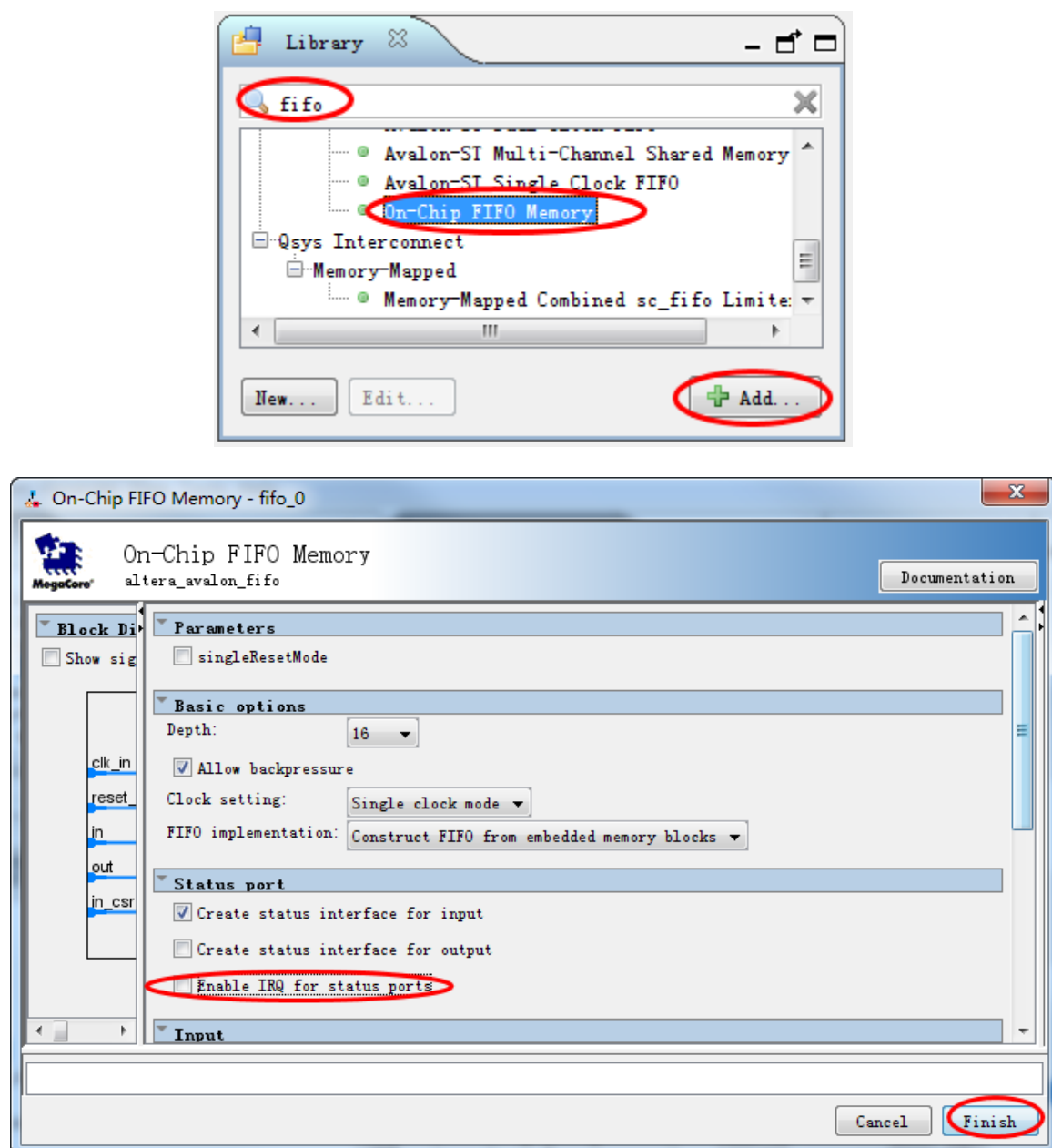
三、硬件设计

1. 创建 Quartus 工程

建立工程文件夹，比如 E:\exp3，工程名 uart，顶层设计实体 uart。

2. 创建 Qsys 系统

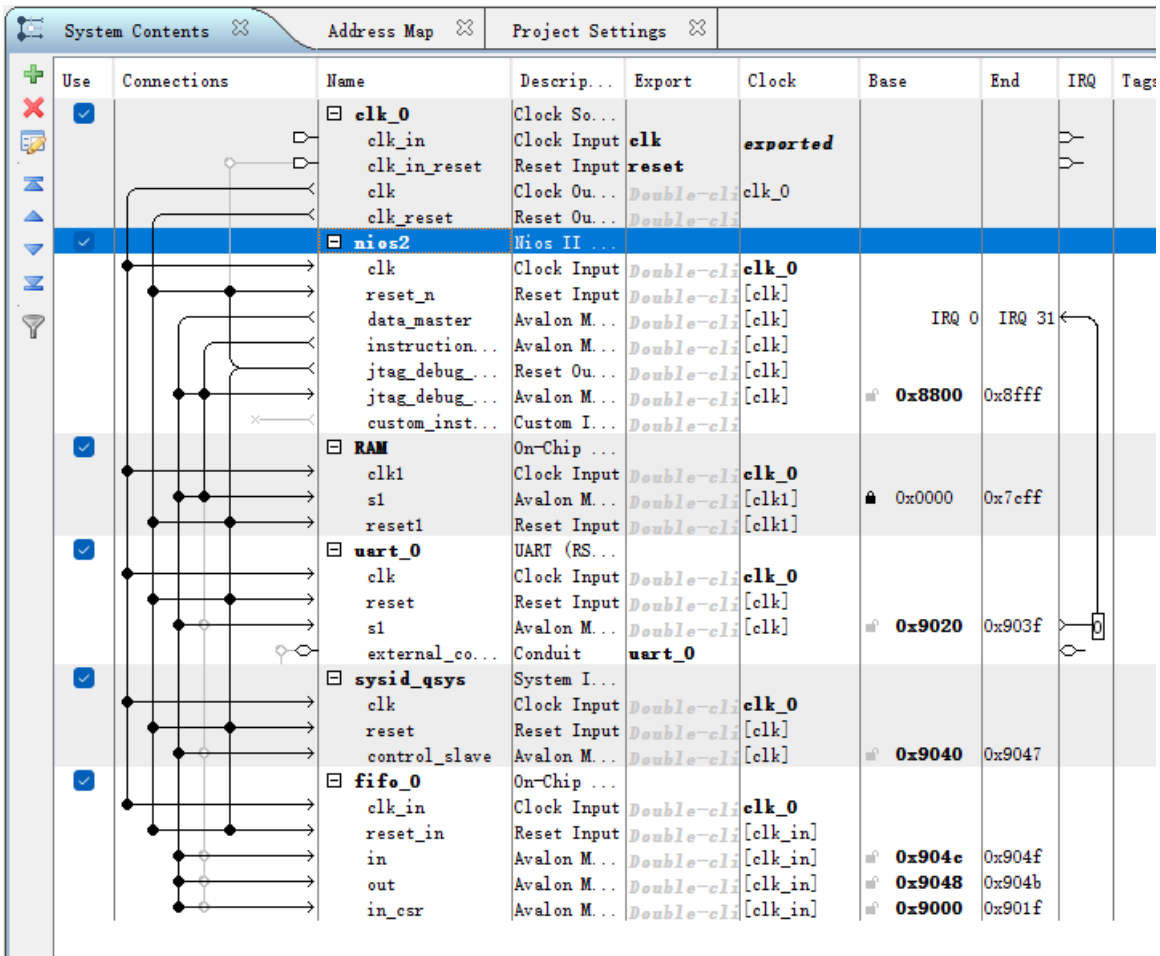
在实验 1 的基础上再添加 FIFO 核。UART 核用于异步串口通信，FIFO 核用于数据缓存。



添加后连线。修改后重新要分配基地址。

保存，并重新生成 Qsys 系统。

Qsys 总的硬件设计如下：



3.集成 Qsys 系统

(1) 例化 Qsys 系统

回到 Quartus 中，添加 Qsys 生成的.qip 文件，再新建 Verilog 文件，保存名为 uart.v。

```
module uart(clk,rst,rx,txd);
    input clk,rst;
    input rx;
    output txd;
    mysystem u0 (
        .clk_clk      (clk),
        .reset_reset_n (rst),
        .uart_0_rxd    (rx),
        .uart_0_txd    (txd)
    );
endmodule
```

(2) 分析综合与引脚锁定

设置 uart 为顶层设计实体。分析综合，引脚锁定可以参考下图。

Named: * Edit: clk		
Node Name	Direction	Location
in altera_reserved_tck	Input	
in altera_reserved_tdi	Input	
out altera_reserved_tdo	Output	
in altera_reserved_tms	Input	
in clk	Input	PIN_E1
in rst	Input	PIN_E16
in rxd	Input	PIN_B5
out txd	Output	PIN_A6
<<new node>>		

(3) 全编译与下载

使用开发板提供的方口 USB 数据线连接开发板和电脑的 USB 口。选菜单【Processing】->【Start Compilation】全编译。Queuatus II 菜单【Tools】->【Programmer】下载。

四、软件设计

将工作空间设置为 E:\exp3。点击【OK】进入 Nios II SBT for Eclipse 软件主界面。如果有工程，选中后，右键删除。

1.创建 NiosII 工程

通过单击菜单栏中的【File】→【New】→【Nios II Application and BSP from Template】，来新建 Nios II 工程。取名 uart。 修改 hello_world.c 的内容：

```
#include <stdio.h>
#include "system.h"
#include "altera_avalon_uart_regs.h"
#include "sys/alt_irq.h"
#include "alt_types.h"
#include "altera_avalon_fifo.h"
#include "altera_avalon_fifo_regs.h"
#include "altera_avalon_fifo_util.h"
alt_u8 rxdata, txdata;
void irq_uart() {
    if((IORD_ALTERA_AVALON_UART_STATUS(UART_0_BASE)
        & ALTERA_AVALON_UART_STATUS_RRDY_MSK)) {
        rxdata = IORD_ALTERA_AVALON_UART_RXDATA(UART_0_BASE);
        altera_avalon_fifo_write_fifo(FIFO_0_IN_BASE, FIFO_0_IN_CSR_BASE,
            rxdata);
    }
}
void irq_uart_init() {
    IOWR_ALTERA_AVALON_UART_STATUS(UART_0_BASE, 0);
    IOWR_ALTERA_AVALON_UART_DIVISOR(UART_0_BASE, 5000000/115200);
```

```

IOWR_ALTERA_AVALON_UART_CONTROL(UART_0_BASE,
    ALTERA_AVALON_UART_STATUS_RRDY_MSK);
alt_ic_isr_register(UART_0_IRQ_INTERRUPT_CONTROLLER_ID, UART_0_IRQ,
    irq_uart, 0, 0);
}
int main() {
    altera_avalon_fifo_init(FIFO_0_IN_CSR_BASE, 0, 0, 0);
    irq_uart_init();
    int level = 0;
    while (1) {
        if ((IORD_ALTERA_AVALON_UART_STATUS(UART_0_BASE)
            & ALTERA_AVALON_UART_STATUS_TRDY_MSK)) {
            level = altera_avalon_fifo_read_level(FIFO_0_IN_CSR_BASE);
            if (level != 0) {
                txdata = altera_avalon_fifo_read_fifo(FIFO_0_OUT_BASE,
                    FIFO_0_IN_CSR_BASE);
                IOWR_ALTERA_AVALON_UART_TXDATA(UART_0_BASE, txdata);
            }
        }
    }
    return 0;
}

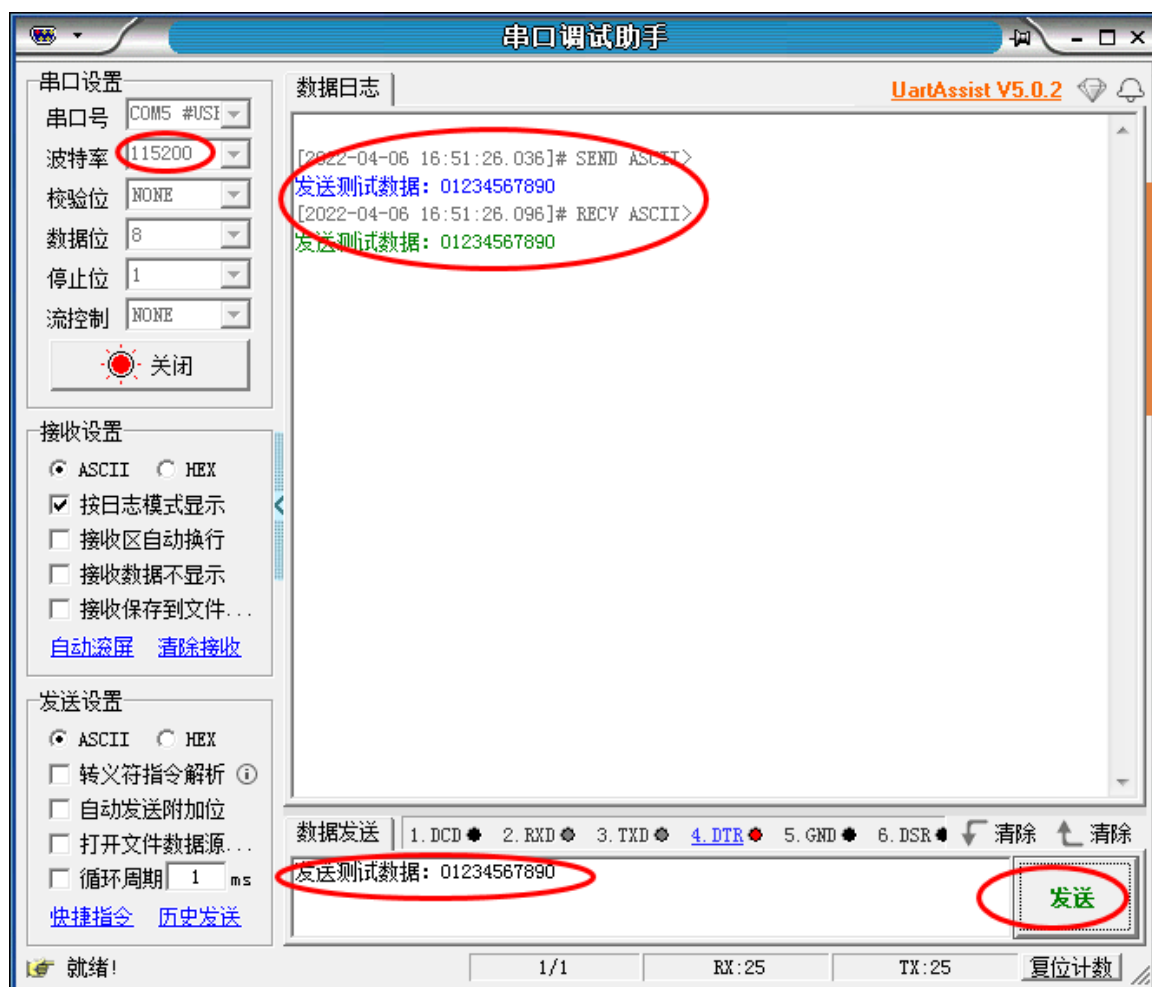
```

2.编译 NiosII 工程

保存后，选择【Project】菜单的【Build All】来编译工程。

3.下载验证

选择【Run】→【Run Configurations...】，设置下载。打开串口调试助手，设置波特率 115200，其它默认。从串口连续发送大量数据，观察返回的数据。



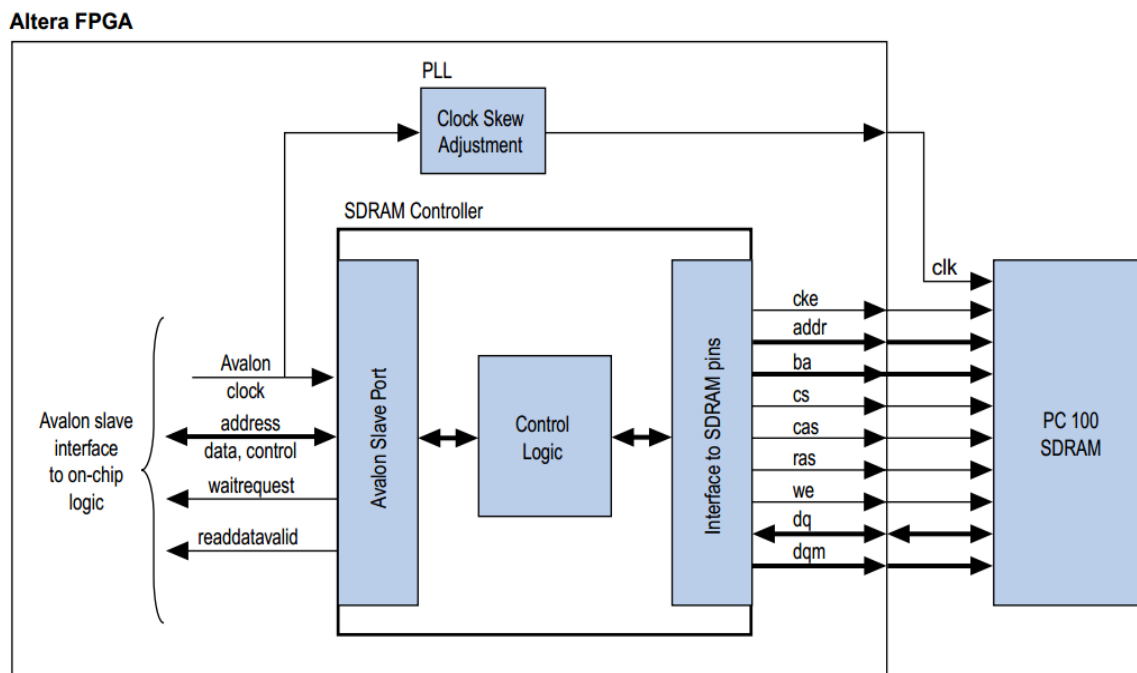
实验四 SDRAM IP 核

一、实验任务

1. Qsys 系统中加入 SDRAM 核，熟悉 SDRAM 核的功能。
2. SDRAM 读写实验，测试串口回环。

二、实验简介

SDRAM 控制器 IP 核能够处理所有的 SDRAM 协议要求，包括上电初始化、地址复用、刷新、读写时序等，极大方便了 SDRAM 的使用。下图为 SDRAM 控制器 IP 核和 SDRAM 芯片的连接框图。



开发板采用的 SDRAM 型号为 W9825G6KH-6， 4Mbit x 4banks x 16Bits I/O Synchronous DRAM。行地址: RA0~RA12, 列地址: CA0~CA8。芯片引脚描述如下。

PIN NUMBER	PIN NAME	FUNCTION	DESCRIPTION
23–26, 22, 29–36	A0–A12	Address	A0–A8 Multiplexed pins for row and column address. Row address: A0–A12. Column address: A0–A8. A10 is sampled during a precharge command to determine if all banks are to be precharged or bank selected by BS0, BS1.
20, 21	BS0, BS1	Bank Select	Select bank to activate during row address latch time, or bank to read/write during address latch time.
2, 4, 5, 7, 8, 10, 11, 13, 42, 44, 45, 47, 48, 50, 51, 53	DQ0–DQ15	Data Input/Output	Multiplexed pins for data output and input.
19	$\overline{\text{CS}}$	Chip Select	Disable or enable the command decoder. When command decoder is disabled, new command is ignored and previous operation continues.
18	$\overline{\text{RAS}}$	Row Address Strobe	Command input. When sampled at the rising edge of the clock, $\overline{\text{RAS}}$, $\overline{\text{CAS}}$ and $\overline{\text{WE}}$ define the operation to be executed.
17	$\overline{\text{CAS}}$	Column Address Strobe	Referred to $\overline{\text{RAS}}$
16	$\overline{\text{WE}}$	Write Enable	Referred to $\overline{\text{RAS}}$
15, 39	LDQM, UDQM	Input/Output Mask	The output buffer is placed at Hi-Z (with latency of 2) when DQM is sampled high in read cycle. In write cycle, sampling DQM high will block the write operation with zero latency.
38	CLK	Clock Inputs	System clock used to sample inputs on the rising edge of clock.
37	CKE	Clock Enable	CKE controls the clock activation and deactivation. When CKE is low, Power Down mode, Suspend mode, or Self Refresh mode is entered.
1, 14, 27	VDD	Power	Power for input buffers and logic circuit inside DRAM.
28, 41, 54	VSS	Ground	Ground for input buffers and logic circuit inside DRAM.
3, 9, 43, 49	VDDQ	Power for I/O Buffer	Separated power from VDD, to improve DQ noise immunity.
6, 12, 46, 52	VSSQ	Ground for I/O Buffer	Separated ground from VSS, to improve DQ noise immunity.
40	NC	No Connection	No connection.

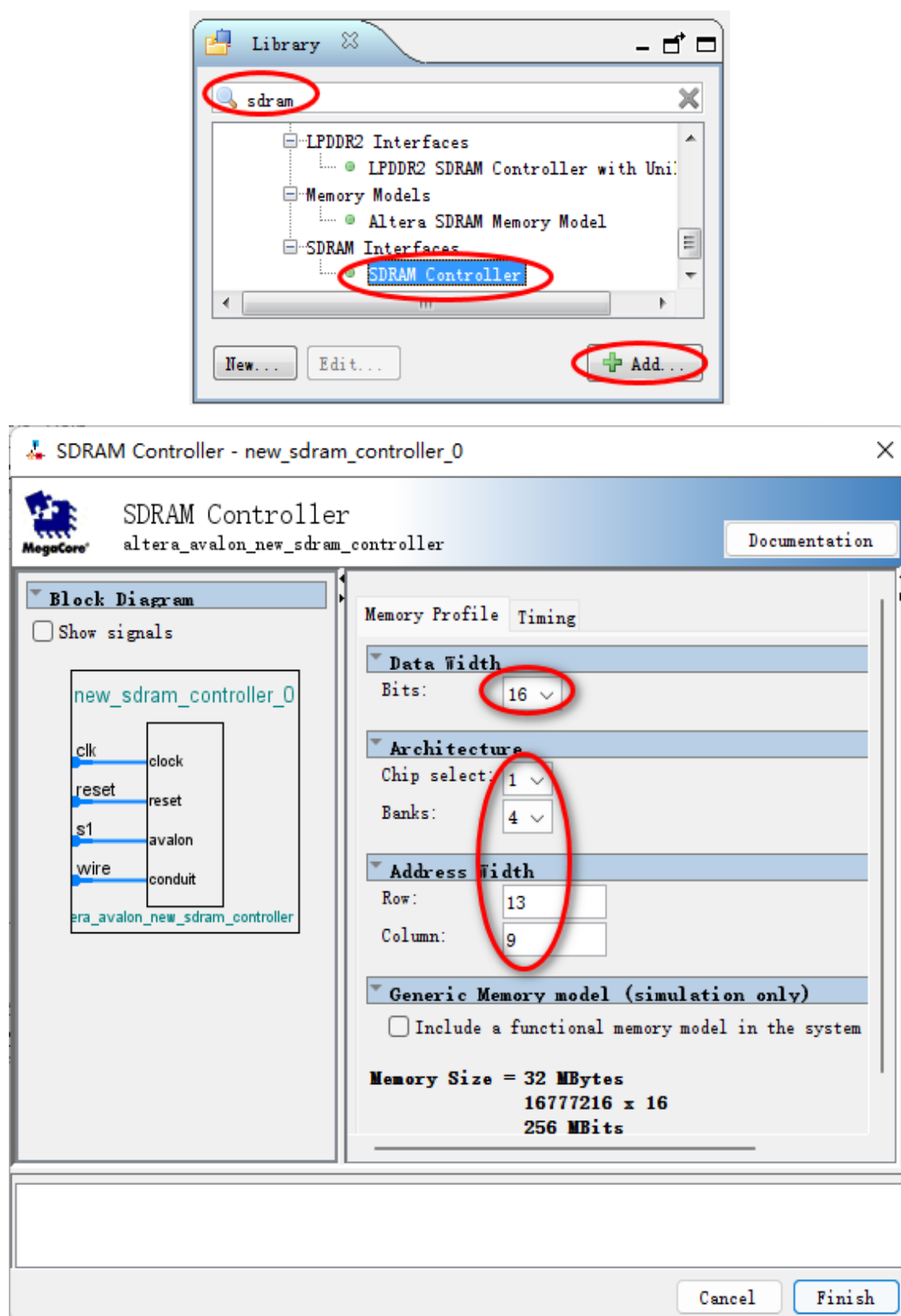
三、硬件设计

1. 创建 Quartus 工程

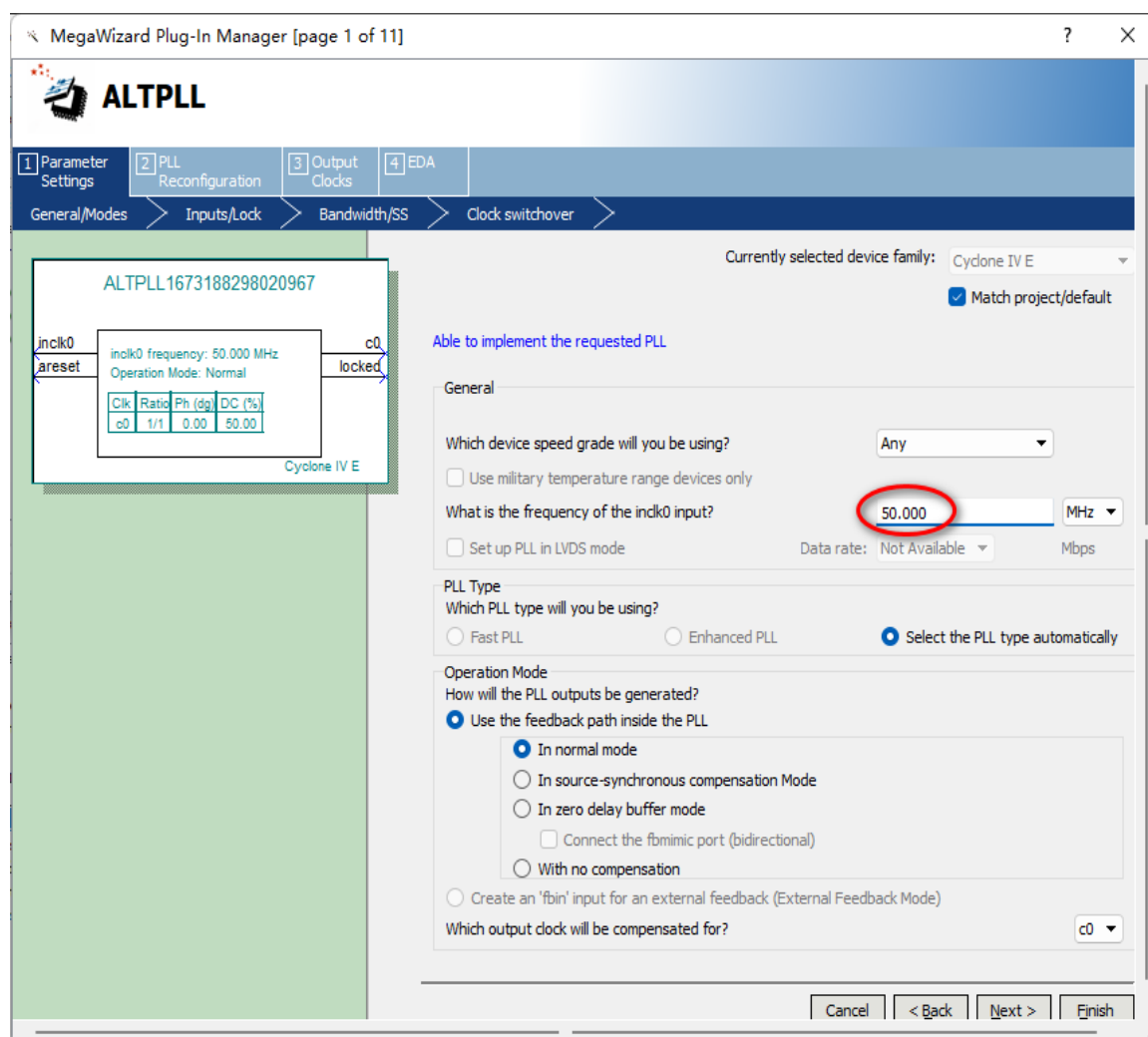
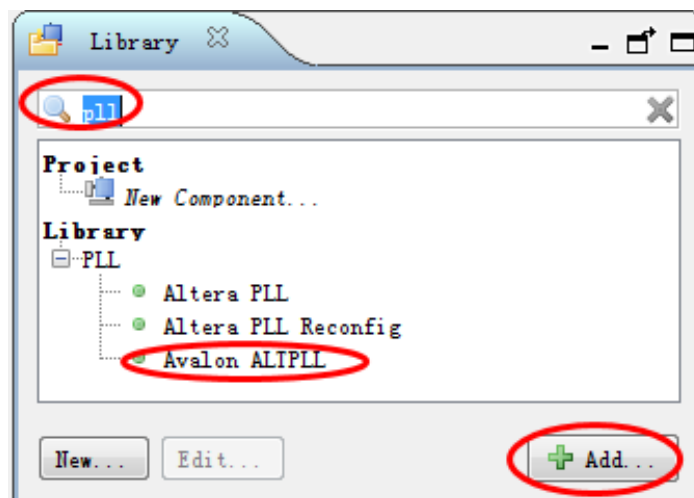
建立工程文件夹，比如 E:\exp4，工程名 sdram，顶层设计实体 sdram。

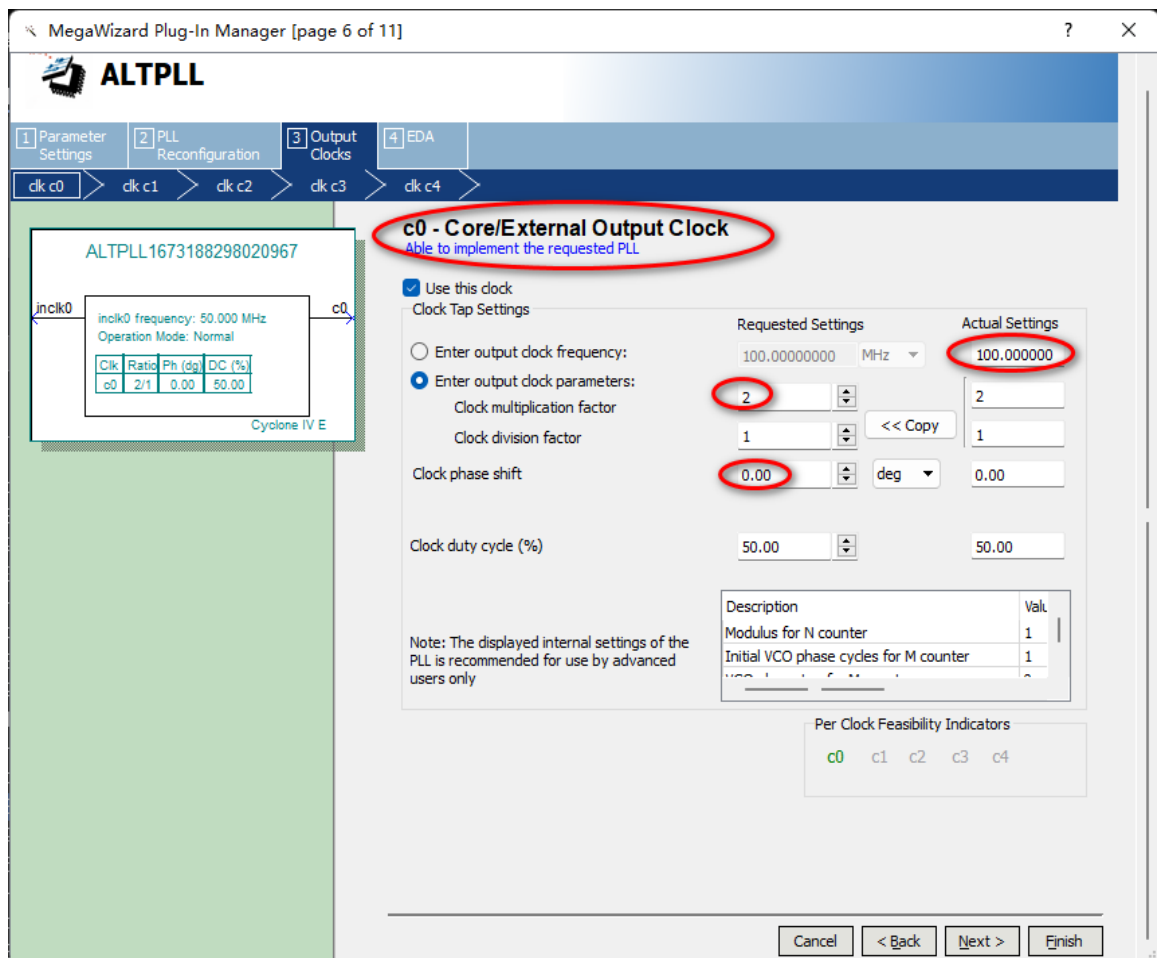
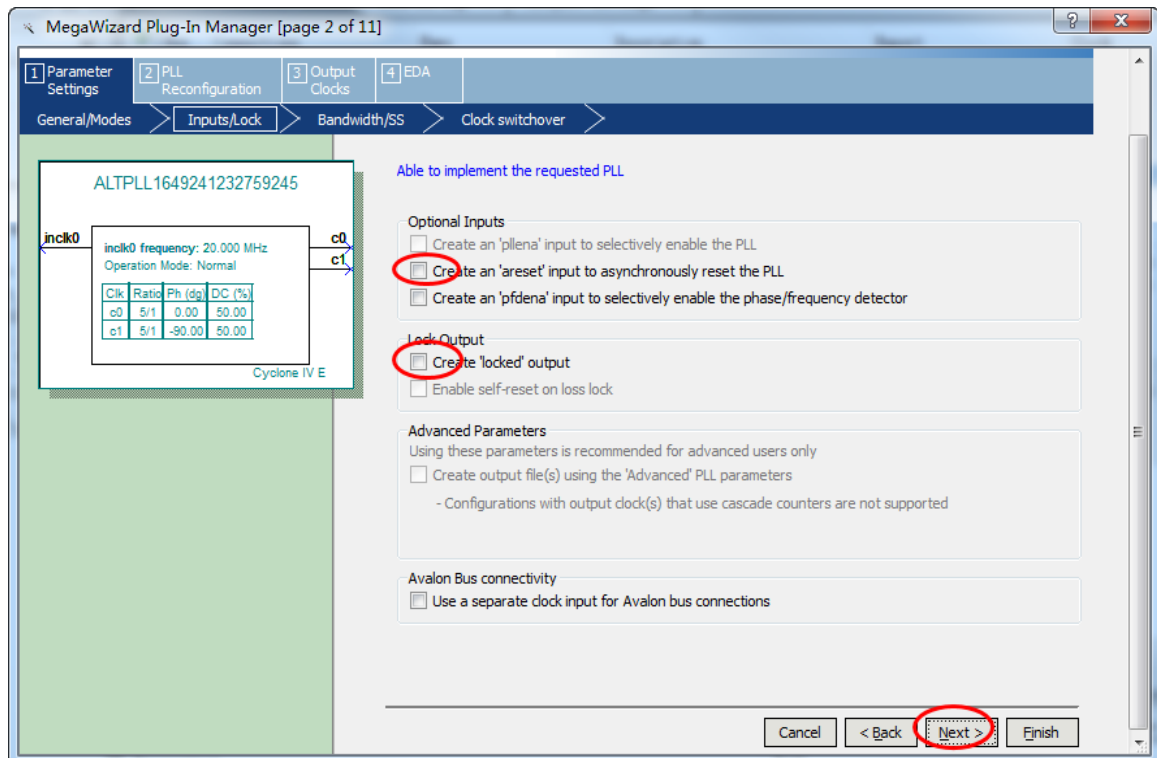
2. 创建 Qsys 系统

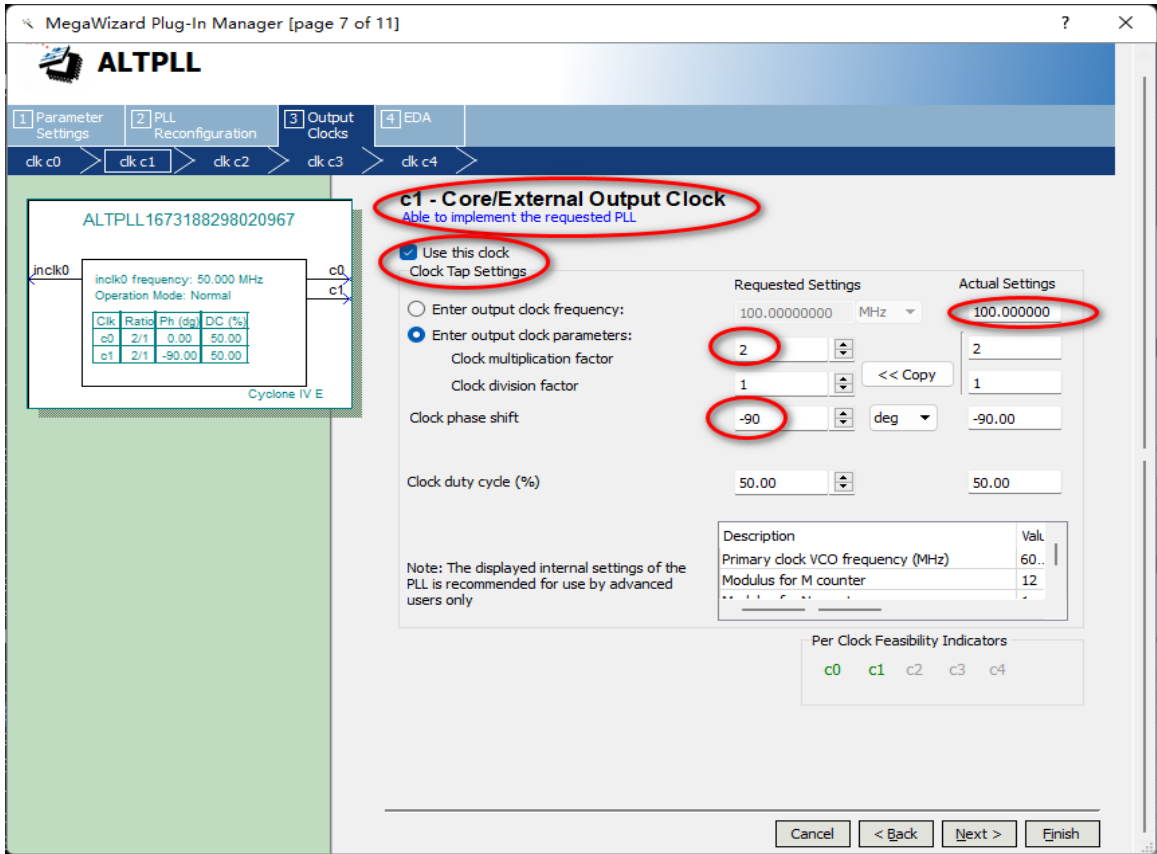
在实验 1 的基础上删除 RAM，然后添加 SDRAM 核和 PLL 核。UART 核用于异步串口通信，PLL 核用于产生 SDRAM 的时钟信号。



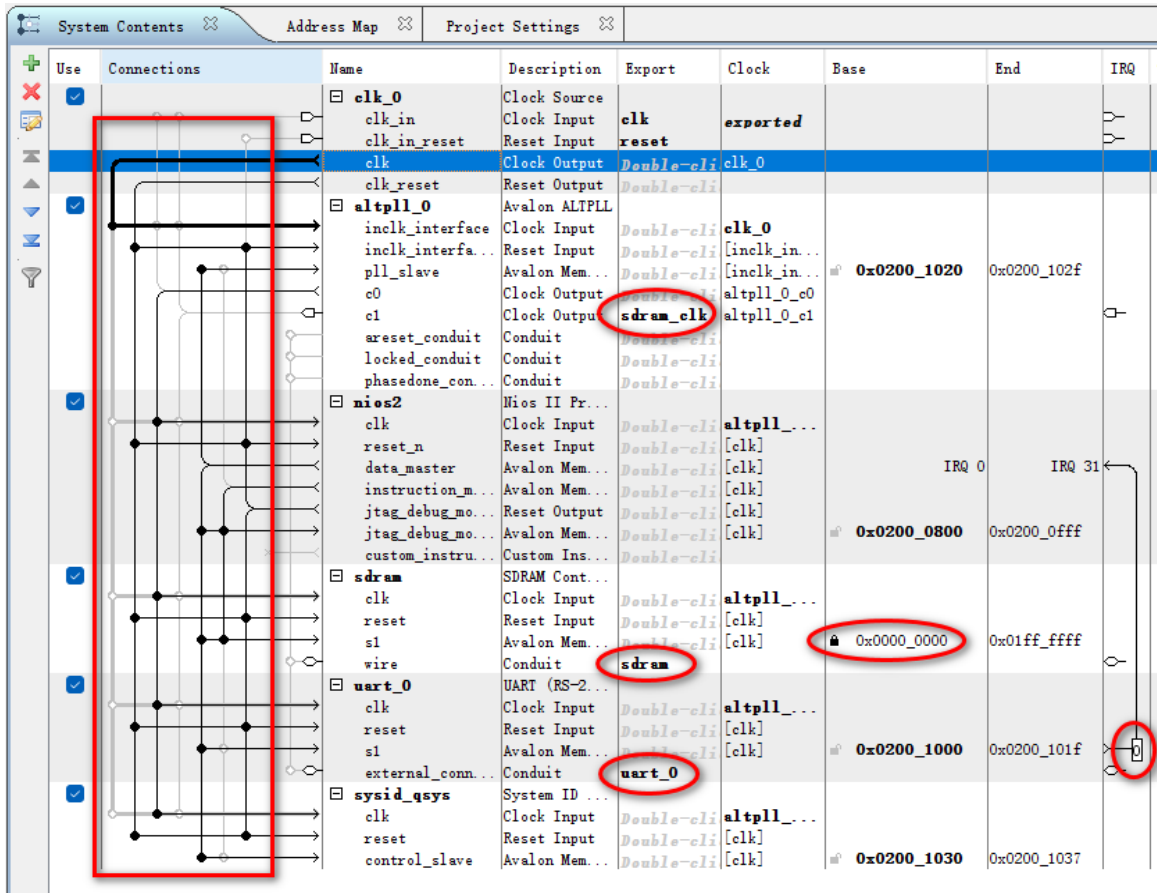
再添加 PLL 核。PLL 核输出两个 100M 时钟频率，相位相差 90 度。



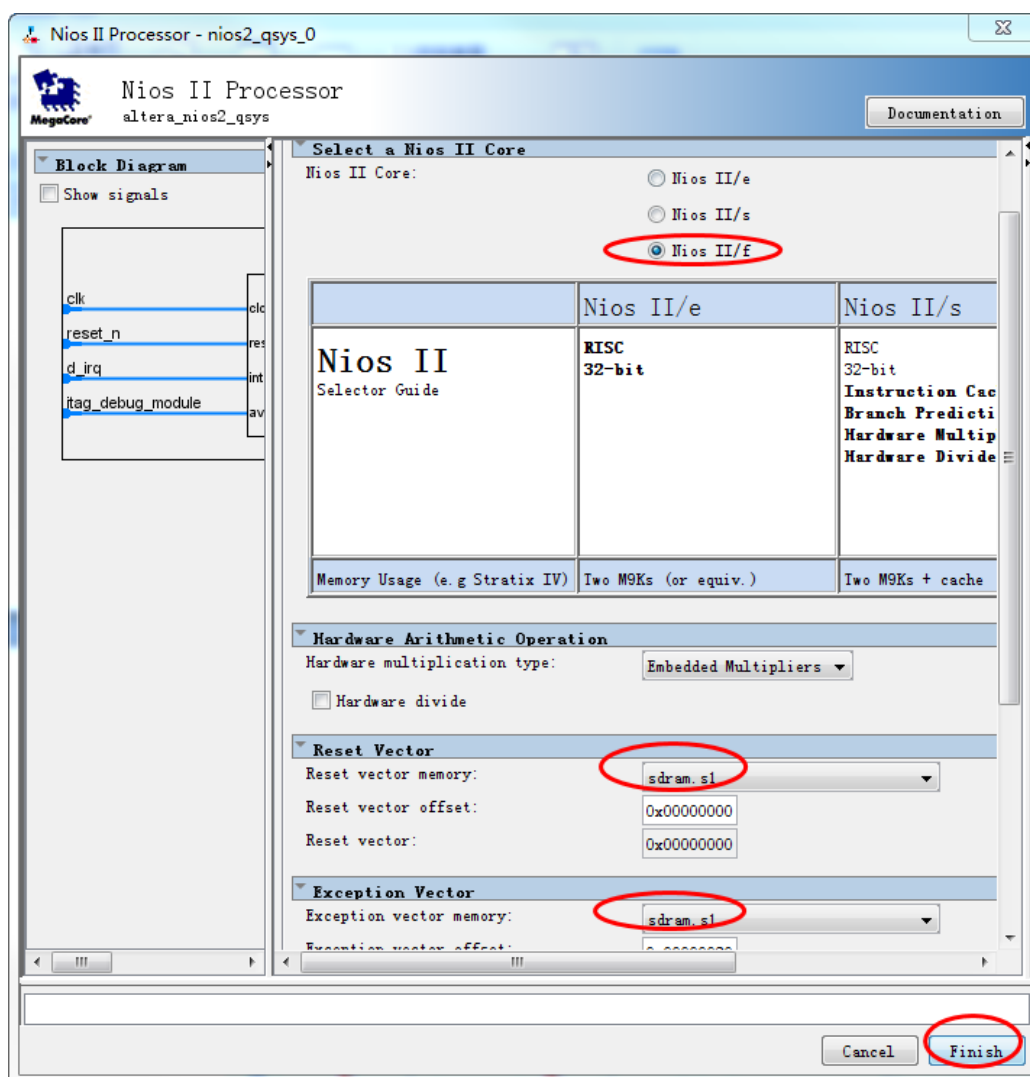




添加后连线，重命名为 sdrum，外部连接名 sdrum_clk。中断号设置。修改后重新分配基地址。



设置复位向量和异常向量。



保存，并重新生成 Qsys 系统。

3.集成 Qsys 系统

(1) 例化 Qsys 系统

回到 Quartus 中，添加 Qsys 生成的.qip 文件，再新建 Verilog 文件，保存名为 sdram.v。

```
module sdram(clk,rst,txd,rxd,
    sdram_clk,sdram_addr,sdram_ba,
    sdram_cas_n,sdram_cke,sdram_cs_n,
    sdram_dq,sdram_dqm,sdram_ras_n, sdram_we_n
);
input clk,rst; input rxd; output txd;
output wire sdram_clk;
output wire [12:0] sdram_addr;
output wire [1:0] sdram_ba;
output wire sdram_cas_n;
```

```

output wire      sdram_cke;
output wire      sdram_cs_n;
inout wire [15:0] sdram_dq;
output wire [1:0] sdram_dqm;
output wire      sdram_ras_n;
output wire      sdram_we_n ;

mysystem u0 (
    .clk_clk      (clk),
    .reset_reset_n (rst),
    .uart_0_rxd    (rxd),
    .uart_0_txd    (txd),
    .sdram_clk_clk (sdram_clk),
    .sdram_addr    (sdram_addr),
    .sdram_ba      (sdram_ba),
    .sdram_cas_n   (sdram_cas_n),
    .sdram_cke     (sdram_cke),
    .sdram_cs_n    (sdram_cs_n),
    .sdram_dq      (sdram_dq),
    .sdram_dqm     (sdram_dqm),
    .sdram_ras_n   (sdram_ras_n),
    .sdram_we_n    (sdram_we_n)
);
endmodule

```

(2) 分析综合与引脚锁定

设置 sdram 为顶层设计实体。分析与综合。在工程目录下，新建文件名为 AC620_SDRAM_pin_assignment.tcl, 内容如下。

```

# W9825G6KH
# 4Banks x 4M x 16Bit Synchronous DRAM
package require ::quartus::project
set_location_assignment PIN_E1 -to clk
set_location_assignment PIN_E16 -to rst
set_location_assignment PIN_B5 -to rxd
set_location_assignment PIN_A6 -to txd
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_addr[12]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_addr[11]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_addr[10]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_addr[9]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_addr[8]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_addr[7]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_addr[6]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_addr[5]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_addr[4]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_addr[3]

```

```

set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_addr[2]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_addr[1]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_addr[0]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_ba[1]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_ba[0]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_cas_n
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_cke
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_clk
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_cs_n
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_dq[15]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_dq[14]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_dq[13]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_dq[12]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_dq[11]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_dq[10]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_dq[9]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_dq[8]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_dq[7]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_dq[6]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_dq[5]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_dq[4]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_dq[3]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_dq[2]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_dq[1]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_dq[0]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_dqm[1]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_dqm[0]
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_ras_n
set_instance_assignment -name IO_STANDARD "3.3-V LVTTL" -to sdram_we_n
set_location_assignment PIN_P11 -to sdram_addr[0]
set_location_assignment PIN_L10 -to sdram_addr[1]
set_location_assignment PIN_P14 -to sdram_addr[2]
set_location_assignment PIN_T13 -to sdram_addr[3]
set_location_assignment PIN_N12 -to sdram_addr[4]
set_location_assignment PIN_M11 -to sdram_addr[5]
set_location_assignment PIN_L11 -to sdram_addr[6]
set_location_assignment PIN_T15 -to sdram_addr[7]
set_location_assignment PIN_R14 -to sdram_addr[8]
set_location_assignment PIN_T14 -to sdram_addr[9]
set_location_assignment PIN_M10 -to sdram_addr[10]
set_location_assignment PIN_R13 -to sdram_addr[11]
set_location_assignment PIN_N11 -to sdram_addr[12]
set_location_assignment PIN_T12 -to sdram_ba[0]
set_location_assignment PIN_M9 -to sdram_ba[1]

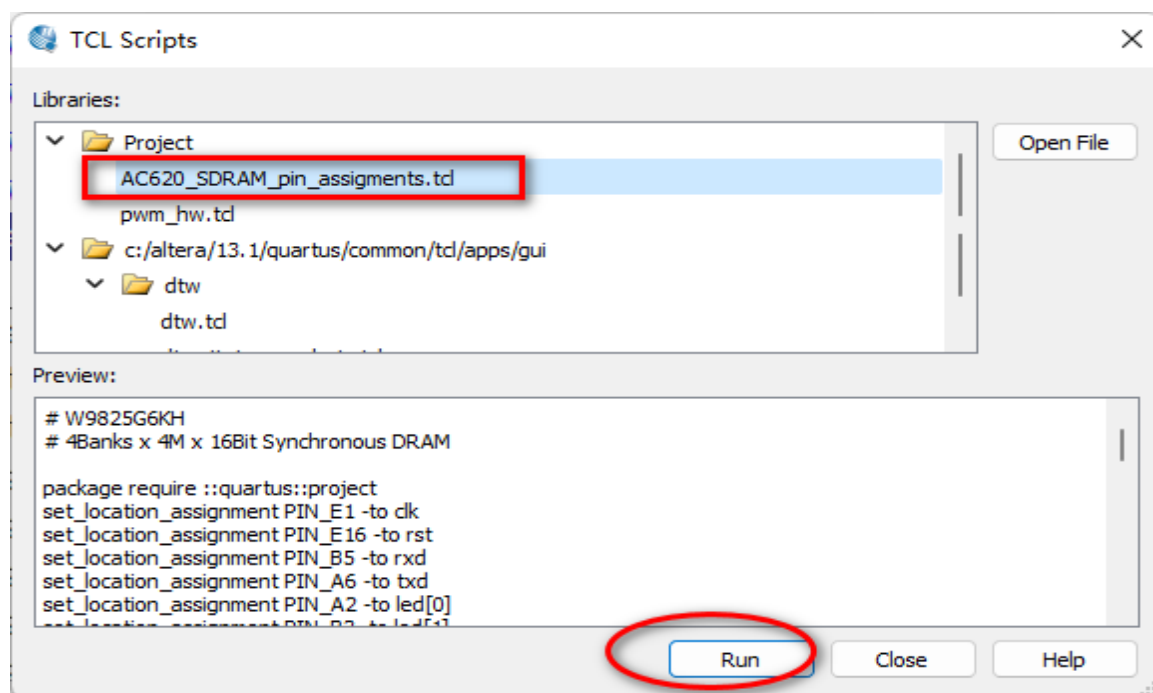
```

```

set_location_assignment PIN_R11 -to sdram_cas_n
set_location_assignment PIN_T11 -to sdram_cke
set_location_assignment PIN_T10 -to sdram_clk
set_location_assignment PIN_R12 -to sdram_cs_n
set_location_assignment PIN_T8 -to sdram_dqm[0]
set_location_assignment PIN_R10 -to sdram_dqm[1]
set_location_assignment PIN_R3 -to sdram_dq[0]
set_location_assignment PIN_T3 -to sdram_dq[1]
set_location_assignment PIN_R4 -to sdram_dq[2]
set_location_assignment PIN_T4 -to sdram_dq[3]
set_location_assignment PIN_R5 -to sdram_dq[4]
set_location_assignment PIN_T5 -to sdram_dq[5]
set_location_assignment PIN_R6 -to sdram_dq[6]
set_location_assignment PIN_R8 -to sdram_dq[7]
set_location_assignment PIN_R9 -to sdram_dq[8]
set_location_assignment PIN_K9 -to sdram_dq[9]
set_location_assignment PIN_L9 -to sdram_dq[10]
set_location_assignment PIN_K8 -to sdram_dq[11]
set_location_assignment PIN_L8 -to sdram_dq[12]
set_location_assignment PIN_M8 -to sdram_dq[13]
set_location_assignment PIN_N8 -to sdram_dq[14]
set_location_assignment PIN_P9 -to sdram_dq[15]
set_location_assignment PIN_N9 -to sdram_ras_n
set_location_assignment PIN_T9 -to sdram_we_n

```

Quartus 菜单【Tools】->【Tools script...】，选中 tcl 文件，然后执行，就会按脚本自动锁定引脚。



(3) 全编译与下载

使用开发板提供的方口 USB 数据线连接开发板和电脑的 USB 口。选菜单【Processing】->【Start Compilation】全编译。Queuatus II 菜单【Tools】->【Programmer】下载。

四、软件设计

将工作空间设置为 E:\exp4。点击【OK】进入 Nios II SBT for Eclipse 软件主界面。如果已有工程，选中后，右键删除。

1.创建 NiosII 工程

通过单击菜单栏中的【File】→【New】→【Nios II Application and BSP from Template】，来新建 Nios II 工程。取名 sdram。修改 hello_world.c 的内容。

测试代码 1: sdram 读写实验。

```
#include <stdio.h>
#include "system.h"
#include "alt_types.h"
#include "unistd.h"
#define sdram_rw_addr (alt_u64) (SDRAM_BASE+0x30000)
int main()
{
    alt_u64 *addr;
    int i;
    addr = sdram_rw_addr;
    for (i=0;i<100;i++){
        *(addr++)=i;
    }
    addr =sdram_rw_addr;
    for (i=0;i<100;i++){
        printf("%ld, %d\n",addr, *(addr++));
        usleep(10000);
    }
    return 0;
}
```

测试代码 2: 串口回环测试。

```
#include <stdio.h>
#include <string.h>
#include <fcntl.h>
#include "system.h"
#include "unistd.h"
int main(void) {
    int fd;
    char buf[100];
    fd = open(UART_0_NAME, O_RDWR, 0666); /* 以可读写方式打开*/
```

```
if (fd < 0) {  
    printf("Some error occurred...\n ");  
    return 1;  
}  
while (1) {  
    if (read(fd, buf, 1)>0 ){  
        write(fd, buf, 1);  
    }  
}  
close(fd);  
return 0;  
}
```

2.编译 NiosII 工程

保存后，选择【Project】菜单的【Build All】来编译工程。

3.下载验证

选择【Run】→【Run Configurations...】，设置下载。打开串口调试助手，设置波特率 115200，其它默认。分别测试代码 1 和代码 2，从串口发送数据，观察返回的数据。

实验五 uCOS 实验

一、实验任务

- 1. Qsys 系统中加入 timer 核，熟悉 timer 核的功能。
- 2.测试 uCOS 多线程任务和信号量共享资源。

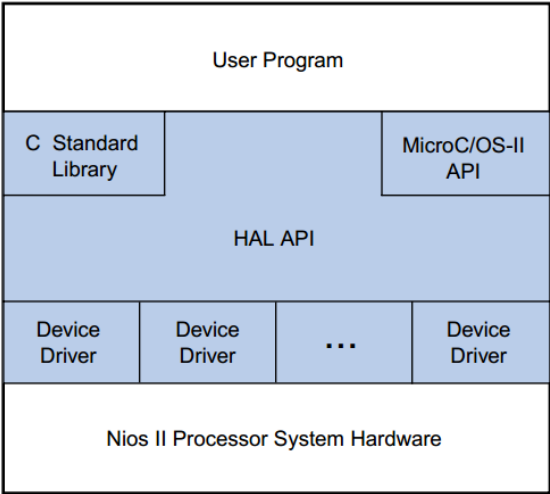
二、实验简介

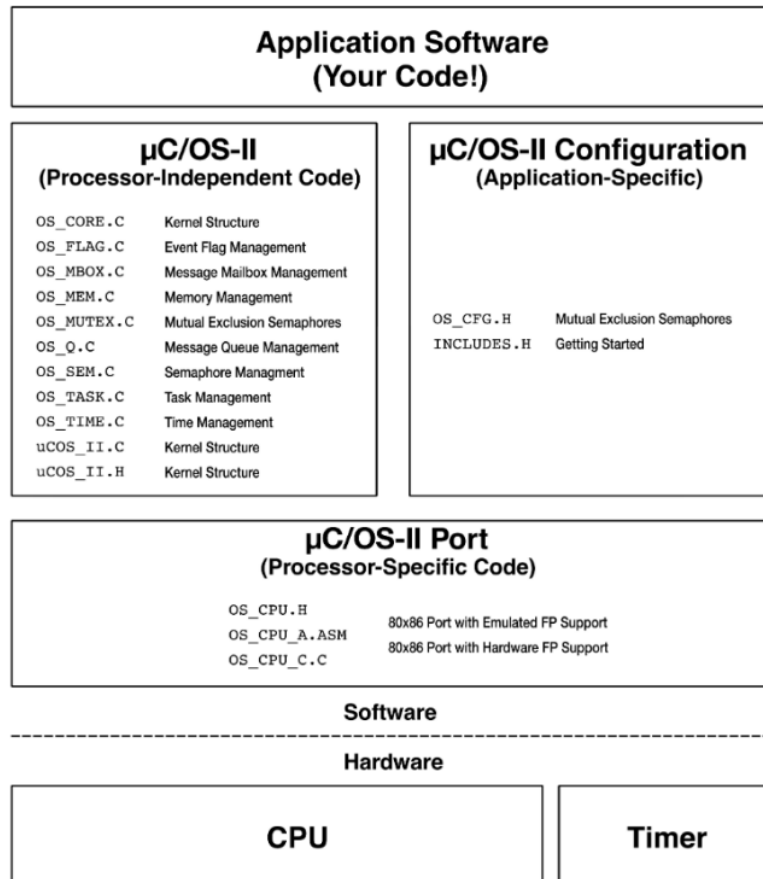
RTOS 全称为 Real Time Operating System，即实时操作系统。顾名思义，实时操作系统强调的是实时性。实时系统又分为硬实时和软实时，硬实时要求在规定的时间内必须完成操作，硬实时系统不允许超时，而对于软实时系统里面，超时的后果就没有那么严格。

uCOS 是 Micrium 公司出品的 RTOS 类实时操作系统，uCOS 目前有两个版本：uCOSII 和 uCOSIII。uCOSII 是用 C 和汇编来写的，其中绝大部分都是用 C 语言编写的，只有极少数的与处理器密切相关的部分代码是用汇编写的。

uCOSII 作为 RTOS 提供的服务有：多线程任务（Tasks）、事件标志组（Event flags）、消息传递（Message passing）、内存管理（Memory management）、信号量（Semaphores）、时间管理（Time management）等。操作系统允许多个任务同时运行，叫做多任务，实际上，一个处理器核心在某一时刻只能运行一个任务。操作系统中任务调度器的责任就是决定在某一时刻究竟运行哪个任务，任务调度在各个任务之间的切换。

Architecture of MicroC/OS-II Programs





uCOSII体系结构图

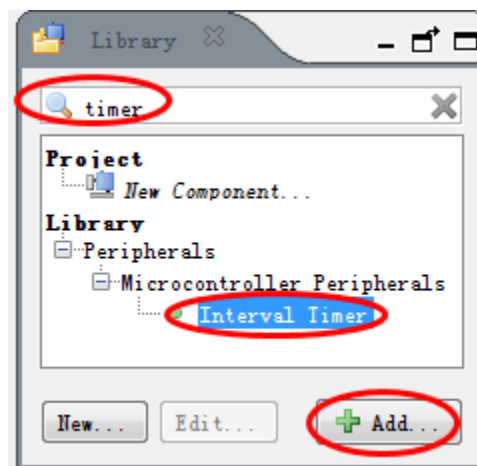
三、硬件设计

1. 创建 Quartus 工程

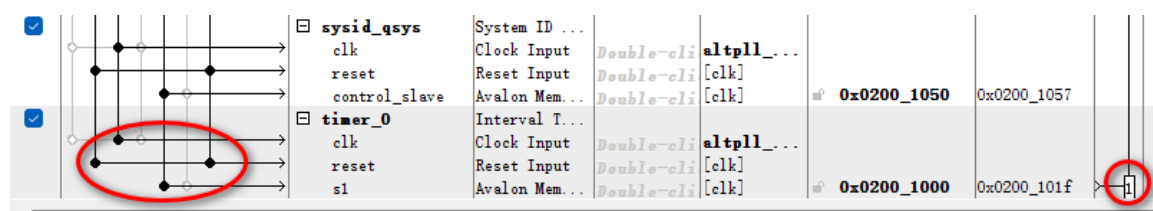
建立工程文件夹，比如 E:\exp5，工程名 uc0s，顶层设计实体 uc0s。

2. 创建 Qsys 系统

本实验是在实验 4 的基础上再添加 timer 核。timer 核用于 uc0s 的定时器。

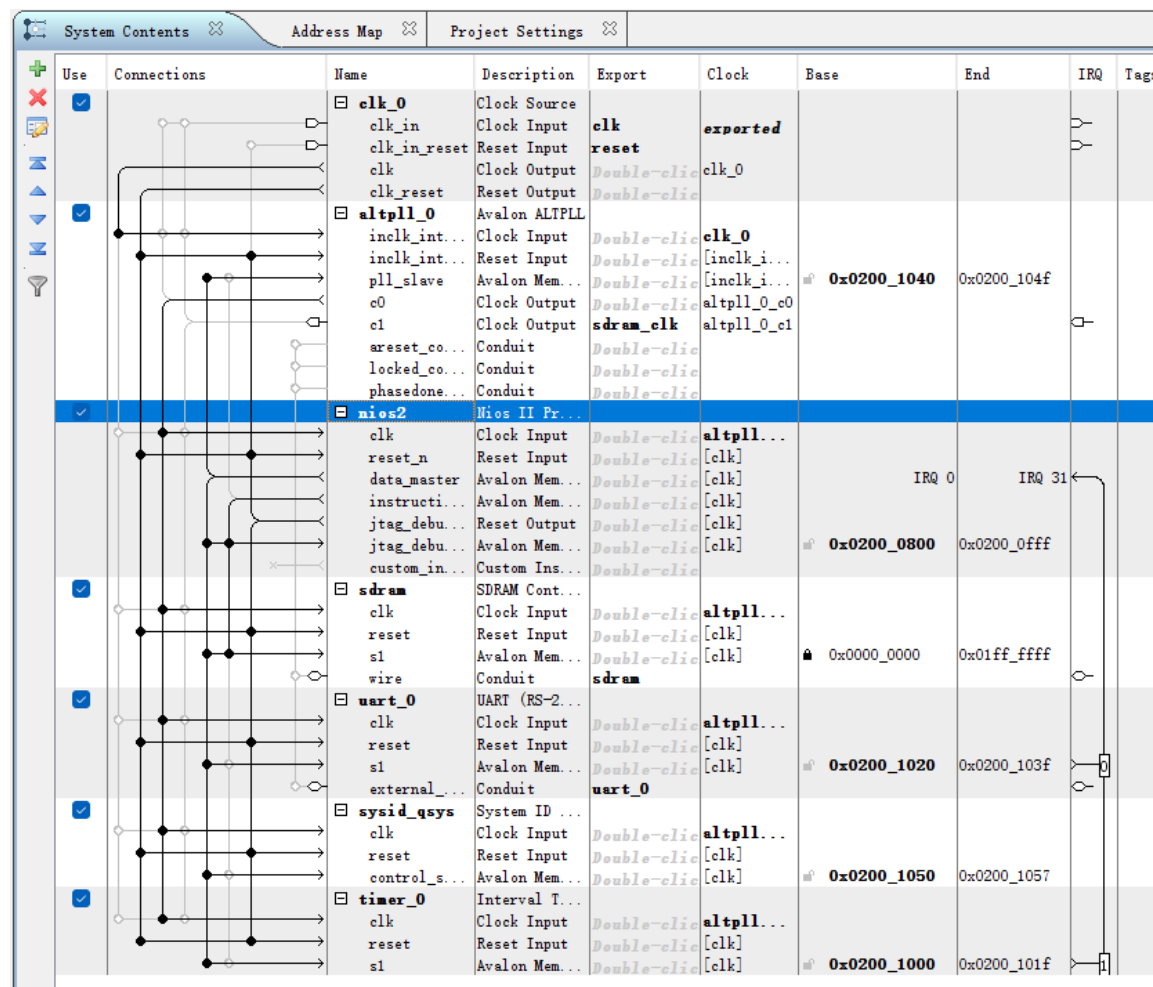


timer 核采用默认设置，不作修改。



添加后连线，中断号设置为1。修改后重新分配基地址。保存，并重新生成Qsys系统。

Qsys总的硬件设计如下:



3.集成 Qsys 系统

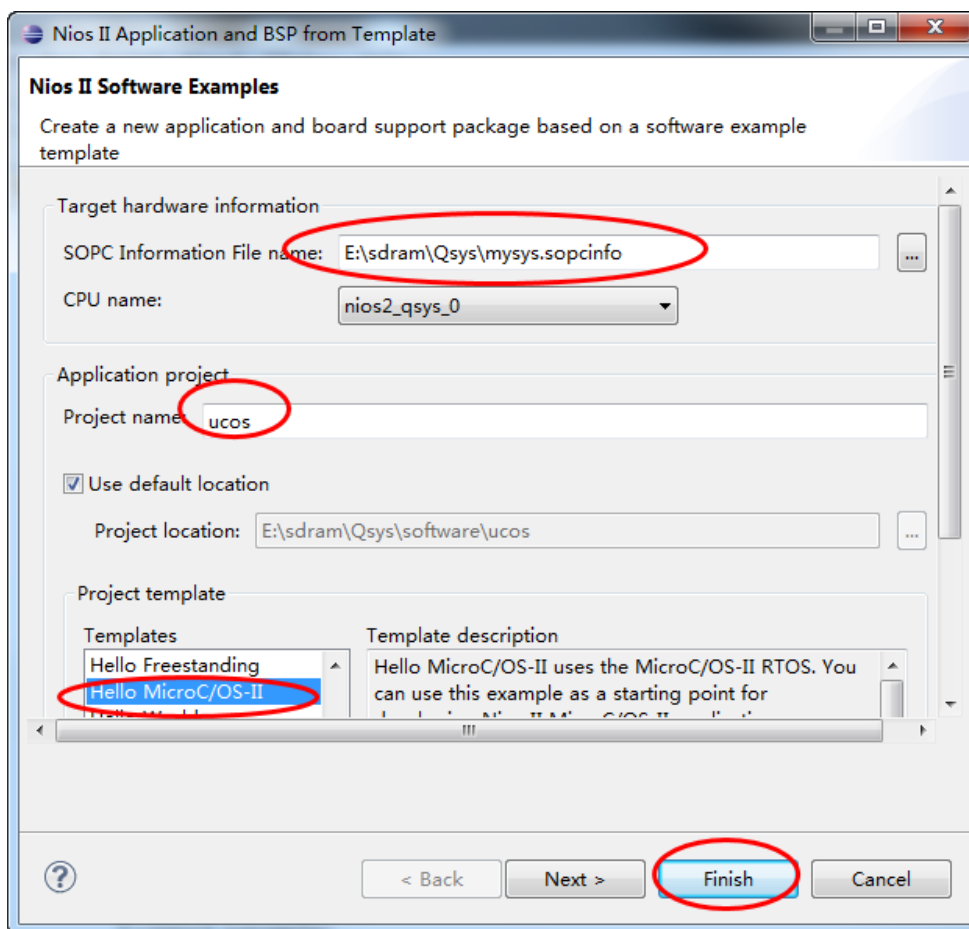
例化 Qsys 系统，实体名改为 uc0s，其它同实验四。使用开发板提供的方口 USB 数据线连接开发板和电脑的 USB 口。最后选菜单 Processing->Start Compilation 全编译。Queuarius II 菜单 Tools->Programmer 下载。

四、软件设计

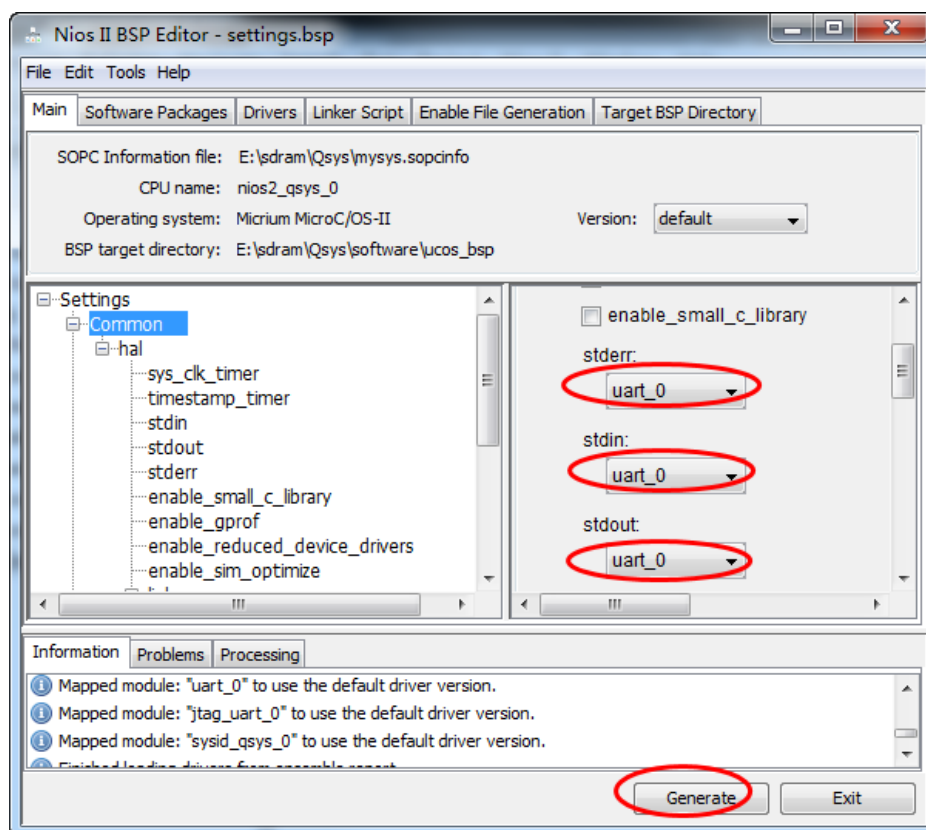
将工作空间设置为 E:\exp5。点击【OK】进入 Nios II SBT for Eclipse 软件主界面。如果有工程，选中后，右键删除。

1. 创建 NiosII 工程

通过单击菜单栏中的【File】→【New】→【Nios II Application and BSP from Template】，来新建 Nios II 工程。取名 ucos。模板选择“Hello MicroC/OS-II”。



bsp 工程中右键，选择【Nios II】->【BSP editor...】，修改标准输入输出为 uart_0。



测试代码 1：hello_ucosii.c 自带的 ucos 多任务测试代码。不需改动。

```
#include <stdio.h>
#include "includes.h"
/* Definition of Task Stacks */
#define TASK_STACKSIZE 2048
OS_STK task1_stk[TASK_STACKSIZE];
OS_STK task2_stk[TASK_STACKSIZE];
/* Definition of Task Priorities */
#define TASK1_PRIORITY 1
#define TASK2_PRIORITY 2
/* Prints "Hello World" and sleeps for three seconds */
void task1(void* pdata)
{
    while (1)
    {
        printf("Hello from task1\n");
        OSTimeDlyHMSM(0, 0, 3, 0);
    }
}
/* Prints "Hello World" and sleeps for three seconds */
void task2(void* pdata)
{
    while (1)
    {
```

```

    printf("Hello from task2\n");
    OSTimeDlyHMSM(0, 0, 3, 0);
}
}
/* The main function creates two task and starts multi-tasking */
int main(void)
{
    OSTaskCreateExt(task1,
                    NULL,
                    (void *)&task1_stk[TASK_STACKSIZE-1],
                    TASK1_PRIORITY,
                    TASK1_PRIORITY,
                    task1_stk,
                    TASK_STACKSIZE,
                    NULL,
                    0);
    OSTaskCreateExt(task2,
                    NULL,
                    (void *)&task2_stk[TASK_STACKSIZE-1],
                    TASK2_PRIORITY,
                    TASK2_PRIORITY,
                    task2_stk,
                    TASK_STACKSIZE,
                    NULL,
                    0);

    OSStart();
    return 0;
}

```

测试代码 2：信号量共享资源。

```

#include <stdio.h>
#include "includes.h"
//定义堆栈大小
#define TASK_STACKSIZE 2048
//定义任务堆栈
OS_STK start_stk[TASK_STACKSIZE];
OS_STK task1_stk[TASK_STACKSIZE];
OS_STK task2_stk[TASK_STACKSIZE];
//定义任务优先级
#define START_PRIORITY 4
#define TASK1_PRIORITY 5
#define TASK2_PRIORITY 6
//定义信号量 ,用作共享资源访问的信号量
OS_EVENT *shared_sem;
//共享内存单元

```

```

char shared_memory[40];
//任务 1 打印字符
void task1(void* pdata) {
    INT8U return_code = OS_NO_ERR; //系统调用的返回状态
    INT8U task1_str[] = "First Task Running";
    while (1) {
        printf("task1:\n");
        OSSemPend(shared_sem, 0, &return_code);
        memcpy(shared_memory, task1_str, sizeof(task1_str));
        usleep(200000);
        printf("%s\r\n", shared_memory);
        OSSemPost(shared_sem);
        OSTimeDlyHMSM(0, 0, 2, 0);
    }
}

//任务 2 打印字符
void task2(void* pdata) {
    INT8U return_code = OS_NO_ERR; //系统调用的返回状态
    INT8U task2_str[] = "Second Task Running";
    while (1) {
        printf("task2:\n");
        OSSemPend(shared_sem, 0, &return_code);
        memcpy(shared_memory, task2_str, sizeof(task2_str));
        usleep(200000);
        printf("%s\r\n", shared_memory);
        OSSemPost(shared_sem);
        OSTimeDlyHMSM(0, 0, 2, 0);
    }
}

//开始任务
void start_task(void* pdata) {
    initOSsemaphore();
    initCreateTasks();
    //任务自我删除
    OSTaskDel(OS_PRIO_SELF);
    while (1) ;
}

//初始化信号量
int initOSsemaphore(void) { //用作二值信号量时,信号量初始化为 1
    shared_sem = OSSemCreate(1);
    return 0;
}

//初始化任务函数
void initCreateTasks() {

```

```
//创建任务 1
OSTaskCreateExt(task1, NULL, (void *) &task1_stk[TASK_STACKSIZE - 1],
                TASK1_PRIORITY, TASK1_PRIORITY, task1_stk, TASK_STACKSIZE,
NULL, 0);
//创建任务 2
OSTaskCreateExt(task2, NULL, (void *) &task2_stk[TASK_STACKSIZE - 1],
                TASK2_PRIORITY, TASK2_PRIORITY, task2_stk, TASK_STACKSIZE,
NULL, 0);
}
//主函数
int main(void) {
    //创建开始任务
    OSTaskCreateExt(start_task, NULL, (void *) &start_stk[TASK_STACKSIZE -
1], START_PRIORITY, START_PRIORITY, start_stk, TASK_STACKSIZE, NULL, 0);
    OSStart();
    return 0;
}
```

2.编译 NiosII 工程

保存后，选择【Project】菜单的【Build All】来编译工程。

3.下载验证

选择【Run】→【Run Configurations...】，设置下载。打开串口调试助手，设置波特率 115200，其它默认。分别测试代码 1 和代码 2。

实验六 自定义 IP 核

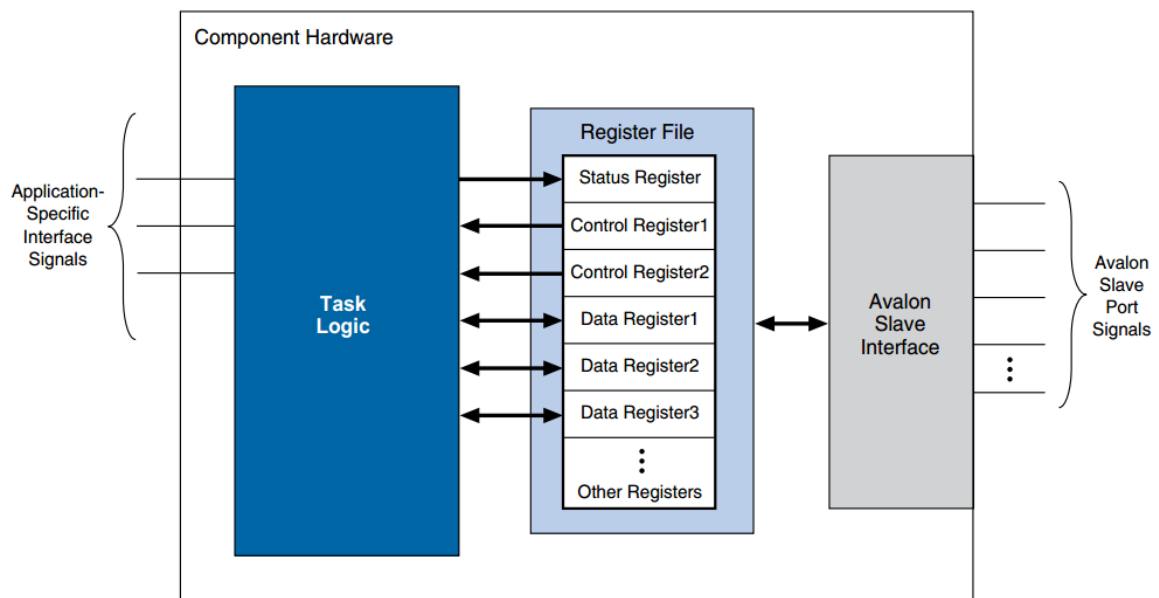
一、实验任务

1. Qsys 系统中加入自定义 PWM IP 核。
2. 使用自定义 PWM IP 核控制 LED 灯，使具有呼吸灯效果。

二、实验简介

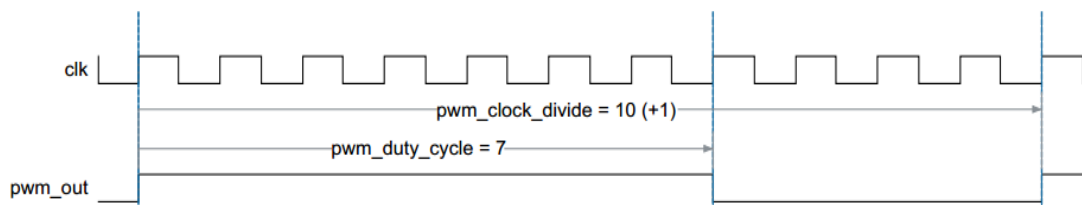
NiosII 是一个嵌入式软核处理器，除了可以根据需要任意添加已经提供的各种外设外，用户还可以通过定制自定义 IP 核的方式来满足各种应用需求。定制 IP 核是使用 NiosII 嵌入式软核处理器的一个重要特征。

Typical Component with One Avalon Slave Port

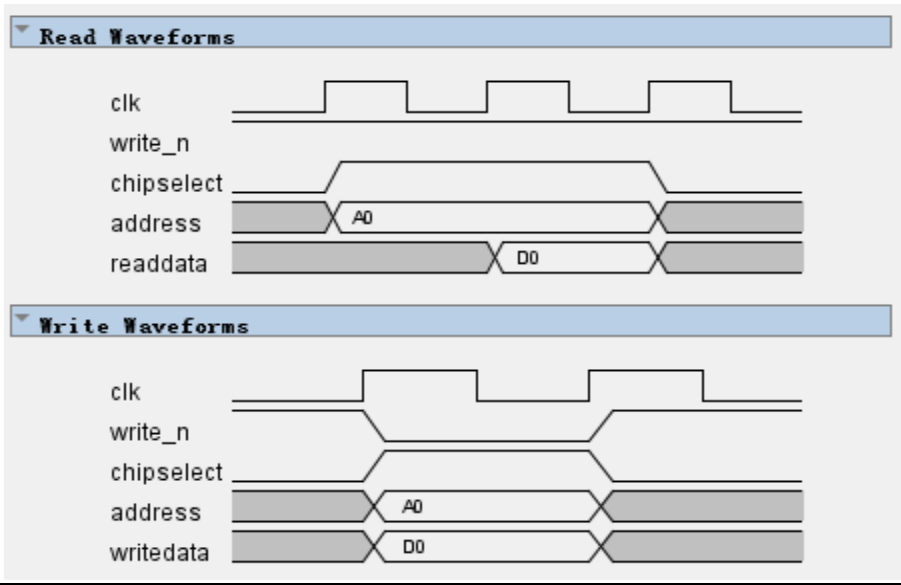
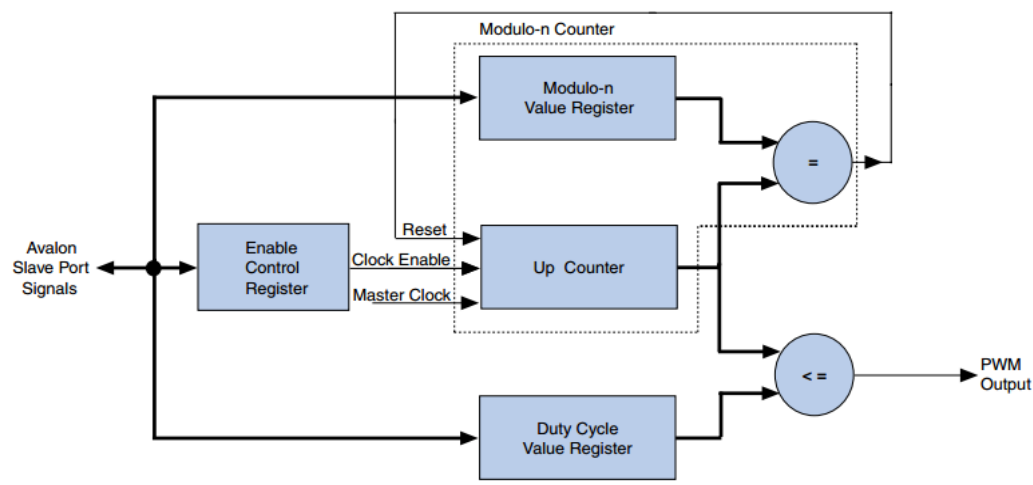


PWM (Pulse Width Modulation 脉冲宽度调制) 是通过对一系列脉冲的宽度进行调制。

Basic Pulse-Width Modulation Waveform



PWM Task Logic Structure



Avalon 信号类型	宽度	方向	描述
clk	1	input	同步时钟信号
reset_n	1	input	复位信号，低电平有效
chipselect	1	input	片选信号
address	2	input	2 位地址，译码后确定寄存器 offset
write_n	1	input	写使能信号
writedata	32	input	32 位写数据值
readdata	32	output	32 位读数据值

三、硬件设计

1.创建 Quartus 工程

建立工程文件夹，比如 E:\exp6，工程名 pwm，顶层设计实体 pwm。

在当前工程文件夹，新建 Verilog 文件，保存名为 mypwm.v。

```
module mypwm(clk,reset_n,chipselect,address,
write_n,writedata,readdata, pwm_out );
    input clk,reset_n;
    input chipselect;
    input [1:0] address;
    input write_n;
    input [31:0] writedata;
    output reg [31:0] readdata;
    output reg pwm_out;
    reg [31:0] pwm_period_reg;
    reg [31:0] pwm_duty_reg;
    reg pwm_enable;
    //读寄存器
    wire [ 31: 0] read_mux_out;
    assign read_mux_out = ({32 {(address == 0)}} & pwm_period_reg) |
({32 {(address == 1)}} & pwm_duty_reg) |
({32 {(address == 2)}} & pwm_enable);
    always @(posedge clk or negedge reset_n)
begin
    if (reset_n == 0)
        readdata <= 0;
    else
        readdata <= {32'b0 | read_mux_out};
end
    //写寄存器
    always @(posedge clk or negedge reset_n)
begin
    if (reset_n == 0)
        pwm_period_reg <= 0;
    else if (chipselect && ~write_n && (address == 0))
        pwm_period_reg <= writedata[31 : 0];
end
    always @(posedge clk or negedge reset_n)
begin
    if (reset_n == 0)
        pwm_duty_reg <= 0;
    else if (chipselect && ~write_n && (address == 1))
        pwm_duty_reg <= writedata[31 : 0];
end
end
```

```

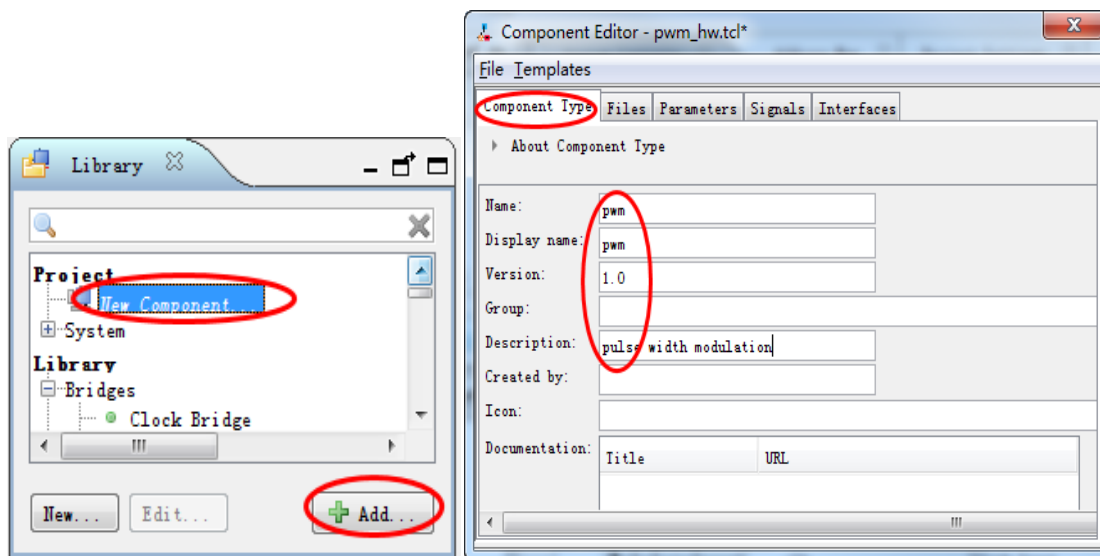
end
always @(posedge clk or negedge reset_n)
begin
    if (reset_n == 0)
        pwm_enable <= 0;
    else if (chipselct && ~write_n && (address == 2))
        pwm_enable <= writedata[0];
end
//逻辑功能
reg [31:0] counter;
always @(posedge clk or negedge reset_n)
begin
    if (reset_n == 0)
        counter <= 0;
    else if (counter < pwm_period_reg)
        counter <= counter+1;
    else
        counter <= 0;
end
always @(posedge clk or negedge reset_n)
begin
    if (reset_n == 0)
        pwm_out <= 0;
    else if (pwm_enable)
        begin
            if (counter < pwm_duty_reg)
                pwm_out <= 1;
            else
                pwm_out <= 0;
        end
    else pwm_out <= 0;
end
endmodule

```

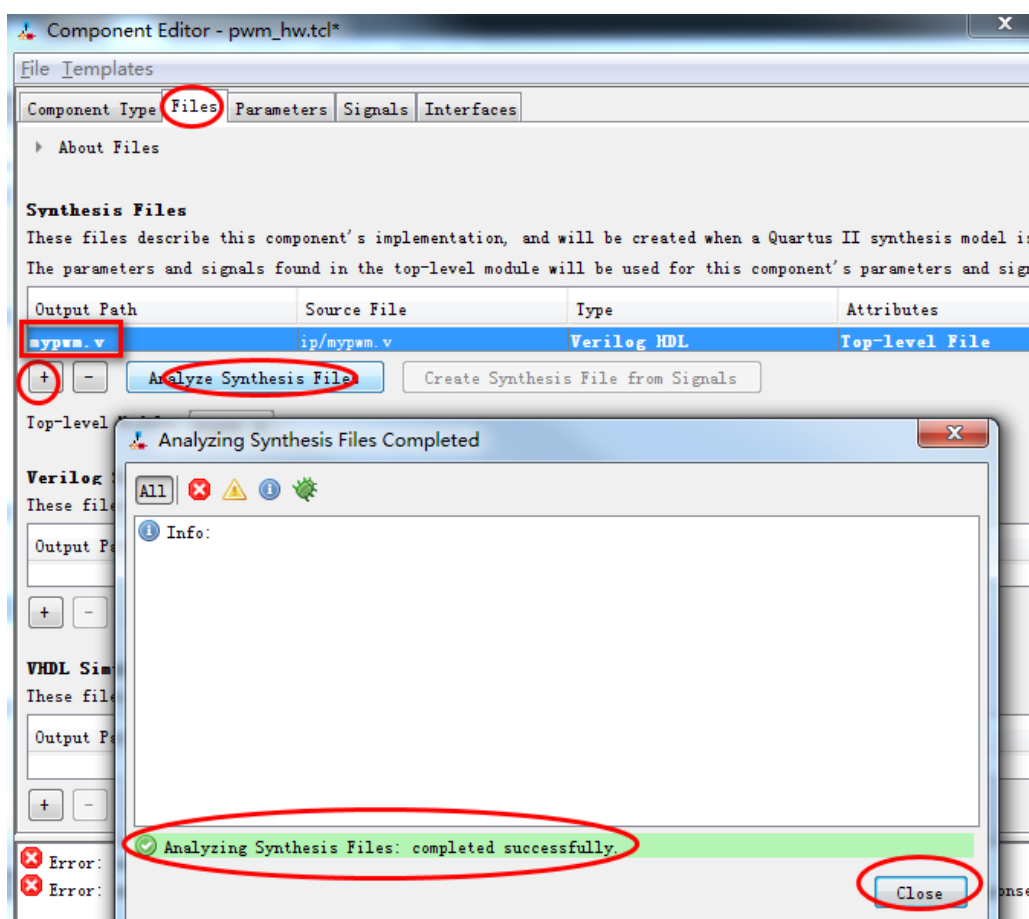
2.创建 Qsys 系统

在实验 1 的基础上添加自定义 pwm IP 核。用于控制 LED 灯的亮度。

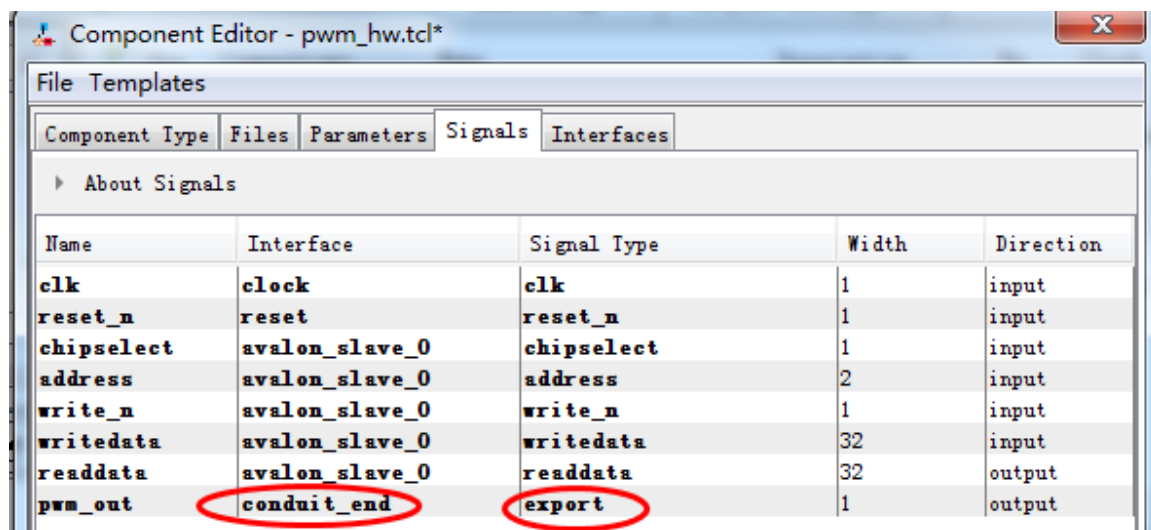
选择 Qsys 软件菜单栏中的【File】→【NewComponent…】将会出现如下图所示页面。在组件编辑器页面中给创建的 IP 核填写基本信息。



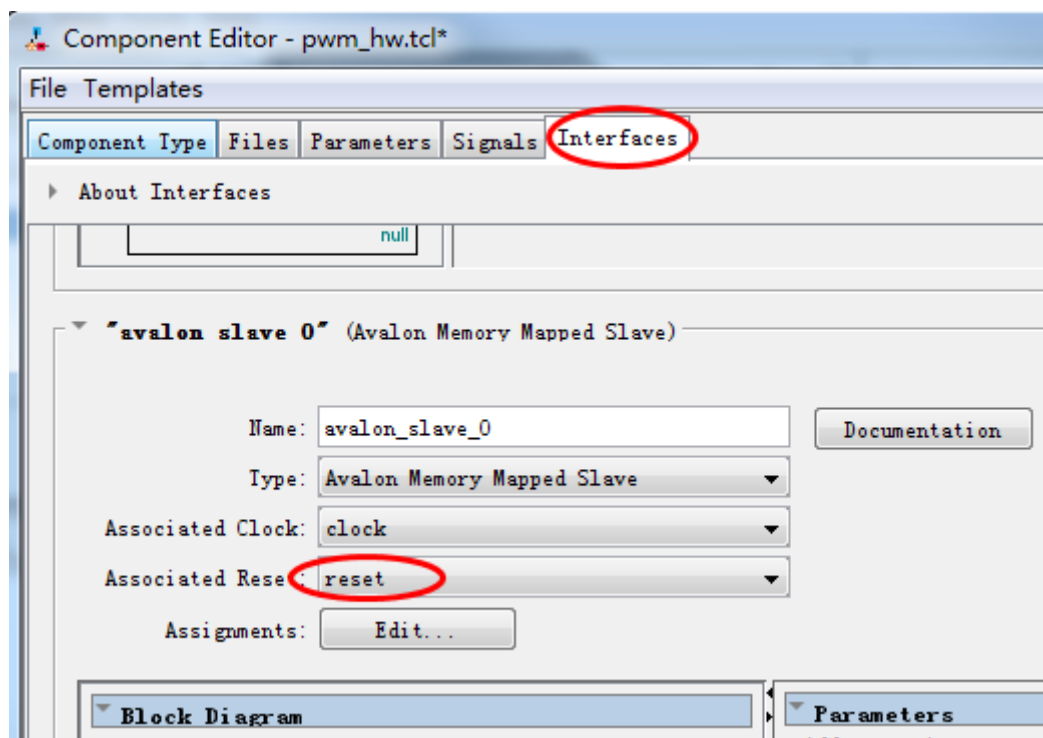
点击【+】按钮打开添加文件对话框，添加 mypwm.v 文件。在 Synthesis Files 栏中可看到刚添加的 mypwm.v 文件。点击【Analyze Synthesis Files】按钮来分析文件。

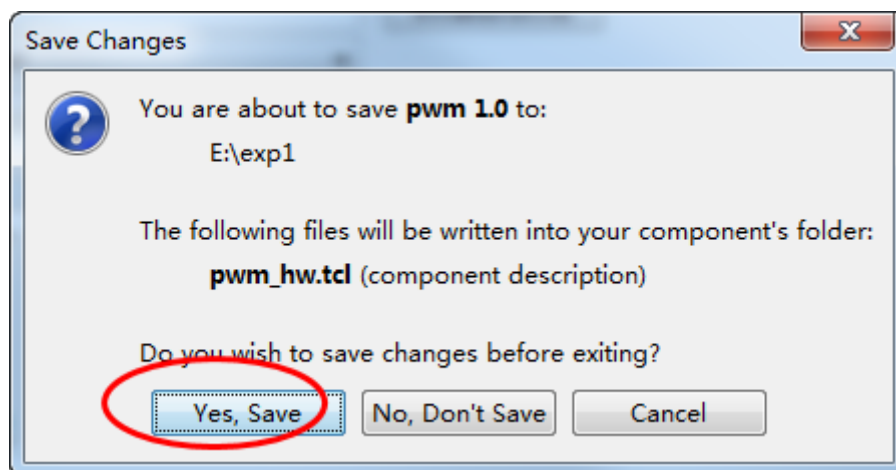


在 Signals 标签中，信号自动识别接口类型，信号类型，宽度和信号方向。但 pwm_out 识别有误，需要手动修改接口信号为：conduit_end，信号类型为 export。

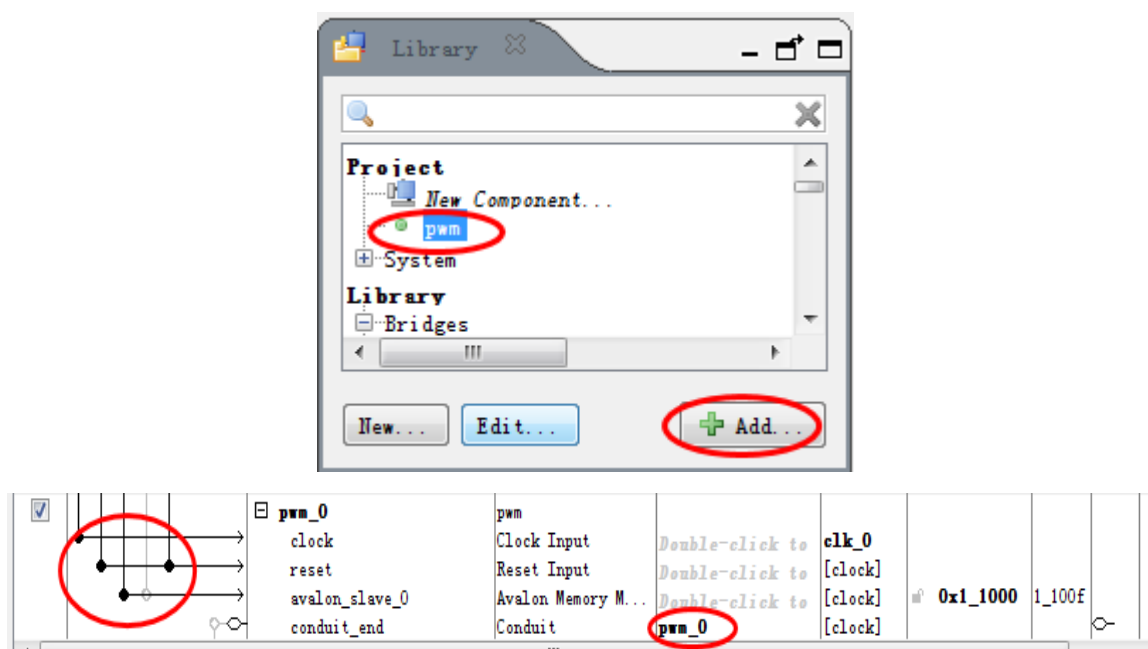


在 Interfaces 标签的 avalon slave 一栏中修改 Associated Reset 选项为 reset, 这时错误消失了。点击【Finish】按钮。





创建好的自定义的 pwm IP 核, 需要添加到 Qsys 系统中。添加后, 连线, 外部连接名为 pwm_0。修改后要重新分配基地址。



Qsys 总的硬件设计如下:

System Contents									
Address Map									
Project Settings									
Use	Connections	Name	Description	Export	Clock	Base	End	IRQ	Tags
☒		clk_0	Clock Source	clk	exported				
		clk_in	Clock Input	reset					
		clk_in_reset	Reset Input	Double-cl	clk_0				
		clk	Clock Output	Double-cl					
		clk_reset	Reset Output	Double-cl					
☒		nios2	Nios II Pro...						
		clk	Clock Input	Double-cl	clk_0				
		reset_n	Reset Input	Double-cl	[clk]				
		data_master	Avalon Memo...	Double-cl	[clk]			IRQ 0	IRQ 31
		instruction_...	Avalon Memo...	Double-cl	[clk]				
		jtag_debug_m...	Reset Output	Double-cl	[clk]				
		jtag_debug_m...	Avalon Memo...	Double-cl	[clk]	0x8800	0x8fff		
		custom_instr...	Custom Inst...	Double-cl	[clk]				
☒		RAM	On-Chip Mem...						
		clk1	Clock Input	Double-cl	clk_0				
		s1	Avalon Memo...	Double-cl	[clk1]	0x0000	0x7eff		
		reset1	Reset Input	Double-cl	[clk1]				
☒		sysid_qsys	System ID P...						
		clk	Clock Input	Double-cl	clk_0				
		reset	Reset Input	Double-cl	[clk]				
		control_slave	Avalon Memo...	Double-cl	[clk]	0x9030	0x9037		
☒		pwm_0	pwm						
		clock	Clock Input	Double-cl	clk_0				
		reset	Reset Input	Double-cl	[clock]	0x9020	0x902f		
		avalon_slave_0	Avalon Memo...	Double-cl	[clock]				
		conduit_end	Conduit	pwm_0	[clock]				

保存，并重新生成 Qsys 系统。

3.集成 Qsys 系统

(1) 例化 Qsys 系统

回到 Quartus 中，添加 Qsys 生成的.qip 文件，再新建 Verilog 文件，保存名为 hello.v。

```
module hello(clk,rst,led);
    input clk,rst;
    output led;
    mysystem u0 (
        .clk_clk      (clk),
        .reset_reset_n (rst),
        .pwm_0_export (led)
    );
endmodule
```

(2) 分析综合与引脚锁定

设置 hello 为顶层设计实体。分析和综合，引脚锁定可以参考下图。

Node Name	Direction	Location
altera_reserved_tck	Input	
altera_reserved_tdi	Input	
altera_reserved_tdo	Output	
altera_reserved_tms	Input	
clk	Input	PIN_E1
led	Output	PIN_A2
rst	Input	PIN_E16
<<new node>>		

(3) 全编译与下载

使用开发板提供的方口 USB 数据线连接开发板和电脑的 USB 口。选菜单【Processing】->【Start Compilation】全编译。Queuatus II 菜单【Tools】->【Programmer】下载。

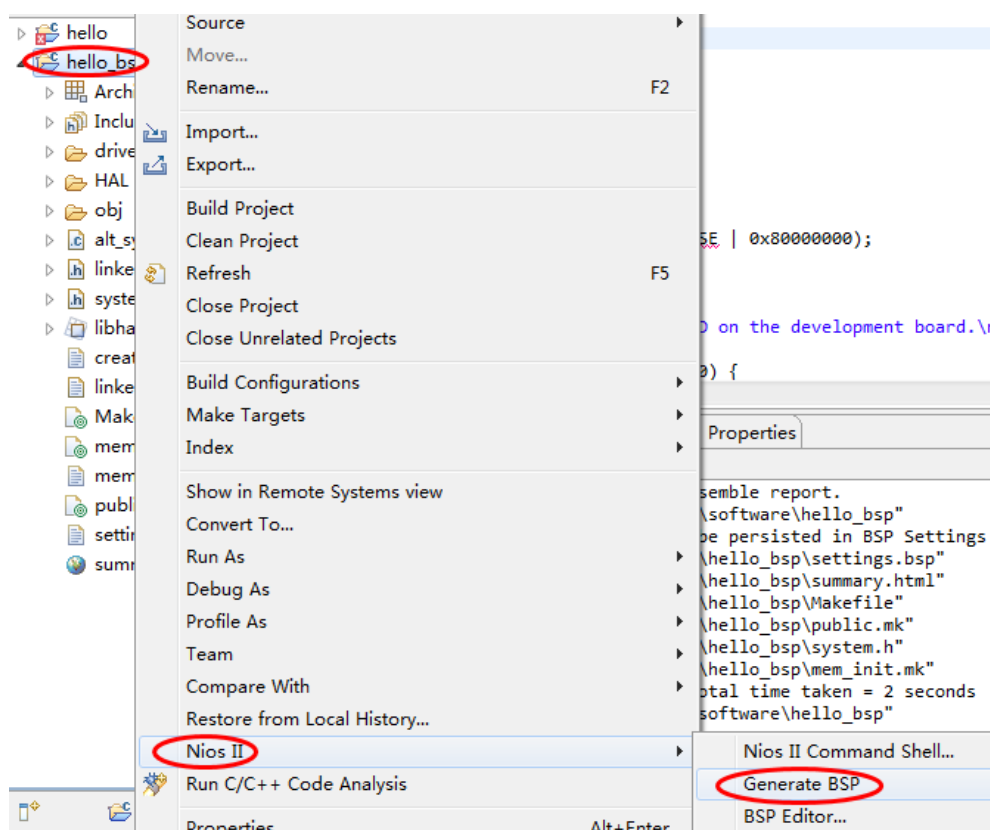
四、软件设计

将工作空间设置为 E:\exp6。点击【OK】进入 Nios II SBT for Eclipse 软件主界面。如果有工程，选中后，右键删除。

1.创建 NiosII 工程

通过单击菜单栏中的【File】→【New】→【Nios II Application and BSP from Template】，来新建 Nios II 工程。修改 hello_world.c 的内容：

```
#include <stdio.h>
#include <unistd.h>
#include "system.h"
typedef struct pwmstru {
    unsigned int period;
    unsigned int duty;
    unsigned int enable;
} pwm0;
int duty = 1;
int main() {
    printf("Notice the pulsing LED on the development board.\n");
    pwm0 * pwm = (pwm0 *) (PWM_0_BASE | 0x80000000);
    pwm->period = 100;
    pwm->duty = duty;
    pwm->enable = 1;
    while (1) {
        while (duty++ < 100) {
            usleep(5000);
            pwm->duty = duty;
        }
        while (duty-- > 0) {
            usleep(5000);
            pwm->duty = duty;
        }
    }
    return 0;
}
```



2. 编译 NiosII 工程

保存后，选择【Project】菜单的【Build All】来编译工程。

3. 下载验证

选择【Run】→【Run Configurations...】，设置下载。观察 LED 灯。